

VECTOR

APL92 Special Issue

- Dyalog APL for Windows Reviewed 55
- APL92 Conference Reports 66
- Ziemann on Smalltalk, APL and J 96
- Frey on OOPS, APL and J 116
- Smith on DDE from APL 128

... plus

- 16-page Educational Supplement 13
- Statistics Library News 29



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.9 No.2 October 1992

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible. Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/386.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1992-93

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail, not needed for Europe)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Student	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

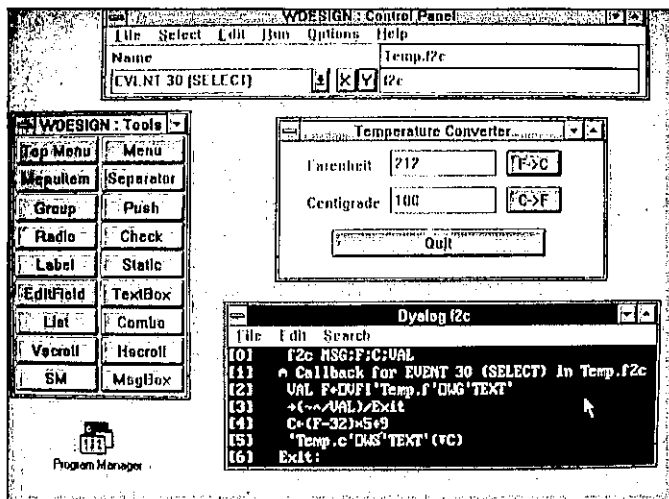
Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385.

CONTENTS

Editorial: APL, Windows and GUIs	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
General Correspondence		7
British APL Association News: News from Sustaining Members	Gill Smith	8
The Education Vector	Alan Mayer	13
The Random Vector	David Eastwood	29
ASLGREG reviewed	Jon Sandles	32
REVIEWS SECTION		
APL Product Guide	Gill Smith	41
Iverson's Introduction to J	Keith Smillie	53
Dyalog APL/W	Duncan Pearson	55
RECENT MEETINGS: APL92 Conference Reports		
APL 92 Reports	Anthony Camacho	66
BEKTOP at St Petersburg	Ray Cannon	88
Pictures at an Exhibition		93
Quotable Quotes from APL 92	John Searle	94
GENERAL ARTICLES		
Smalltalk, APL and J: an Exercise in Comparative Programming	Dave Ziemann	96
Object Oriented Extensions to APL and J	Robert J. Frey	116
APL92 Caption Contest		126
TECHNICAL SECTION		
Shared Variables and Windows DDE	Adrian Smith	128
Three Combinatoric Puzzles	Roger K.W. Hui	139
Index to Advertisers		143



APL for Windows™

Dyalog APL/W is a true 32-bit Windows application and it shows! Click on the APL icon and up pops a powerful multi-window APL development system, with a host of features designed to enhance your productivity and enjoyment. But that's just the start. APL system functions provide you with simple and elegant, yet powerful tools to drive the Windows Graphical User Interface (GUI). In one statement you can create a GUI object such as a PushButton, specify its appearance and behaviour, and attach a "callback" function to be run when the button is pressed. There's even a WDESIGN workspace to help you. Windows programming has never been easier - and your applications will run unchanged under OS/2 and on Unix workstations in future releases of Dyalog APL.

and it's friendly too

Dyalog APL/W supports Dynamic Data Exchange (DDE), using standard APL Shared Variables, so it can talk to any other Windows application that supports DDE. Can you imagine the power of APL combined seamlessly with your favourite databases, spreadsheets and other tools?

DYADIC

All trademarks acknowledged

Editorial: APL, Windows and GUIs

by Jonathan Barman

Exciting things are happening with APL and Windows. Dyalog/W APL and APL*PLUS II both have major advances in the integration of APL into the Windows environment. We have an extended review of the Windows features in Dyalog/W APL. In the next issue we plan to have an equivalent review of APL*PLUS II with Windows. Comparison between the two products will be quite interesting as the two vendors have approached the problem of integrating APL with Windows in completely different ways.

There is more to APL and Windows than just creating sexy looking dialog boxes. Sharing data between APL and other Windows applications is an incredibly powerful tool, and the article by Adrian Smith on Shared Variables in Dyalog/W APL shows just what can be achieved with a few simple functions. You can assign an APL variable and have the data immediately displayed in a chart. You can keep help text in APL variables and use a word processor to edit them directly. Entering data into a cell in an Excel spreadsheet can run an APL function with the results appearing immediately in the spreadsheet. The possibilities seem limitless.

Integrating APL with Windows could be carried out at the language level. APL is an old language, and was designed before any Graphical User Interfaces were thought of. The prime examples are `⎕` and `⎕` input which were originally designed in the heady days when connecting a typewriter to a computer was a wonderful achievement. It is amusing to speculate what APL would look like if it had been designed to be used with a GUI. Quad input would obviously open a dialog box. A standard interface would replace the hotch potch of session managers, function editors, ring editors, etc. that we have at the moment. However, a moment's reflection makes the GUI interface seem to be but froth on the surface in comparison with the solid achievement of a language with a consistent and elegant syntax. I am not sure that the existence of a GUI would have really made all that much difference to the design of APL; after all, the design of J seems to have ignored any GUI aspects.

Smalltalk was designed from the outset with a GUI. Dave Ziemann has written a fascinating article where he solves the same problem in Smalltalk, APL and J, so that the three languages can be compared. The object paradigm of Smalltalk would seem to make the handling of Windows events easier, where a lot of the detailed programming can be inherited from parent objects which are provided

as part of the language. In APL one needs to develop a library of functions to carry out the detailed work, and then life becomes simple again.

Aiming to get the maximum modularity and re-use of code is the same regardless of the language being used. In APL one uses functions and operators, and in Smalltalk one uses objects. Encapsulation seems to be more difficult in APL. Workspaces containing real applications usually have a plethora of global variables. Naming conventions are required to identify types of variables and to prevent conflict between utility functions and application specific functions.

Could APL be extended to include Object Orientation? My first reaction was that it would be impossible, but Robert Frey has written a paper suggesting extensions to J which seem to solve most of the problems. I find it difficult to assess the impact of suggestions like this without an implementation in which to experiment. The implementation of $\square NA$ in APL2 is a stepping stone towards Robert's suggestion. I have found it invaluable for improving encapsulation, and it completely removes the name clash problem.

APL must progress or die. Integration with the Windows GUI is a welcome indication that APL is alive and kicking. The ability to run APL functions and share data with other Windows applications seems to be a major advance.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association
c/o Gill Smith
Brook House, Gilling East,
YORK YO6 4JJ**

Price £12 per vol (UK); £14 (overseas), £20 (airmail)
including Postage & Packing.

Quick Reference Diary 1992-3

Date	Venue	Event
23 October	London	Field Trip to Morgan Stanley, at their new Canary Wharf offices
27 November	IEE London	Details to be Announced
12 February 1993	IEE London	
19 March	IEE London	
23 April	IEE London	
11 June	IEE London	AGM
August 15-19 1993	Toronto, Canada	APL93
17 Sept	IEE, London	Vendor Forum
22 October	IEE London	
26 November	IEE London	

All British APL Association meetings will be held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment. Meetings are normally held on Fridays throughout the autumn and spring.

Dates for Future Issues of VECTOR

	Vol.9 No.3	Vol.9 No.4	Vol.10 No.1
Copy date	1st Dec 92	1st March 93	1st June 93
Ad booking	8th Dec 92	12th March 93	10th June 93
Ad Copy	21st Dec 92	21st March 93	21st June 93
Distribution	January 93	April 93	July 93

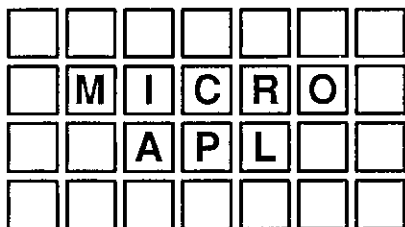
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006

General Correspondence

From: John Searle

26 July 1992

Thank you for improving upon my submitted function in the article *Six Months in the Life of Vector*. It was much more interesting than my original. Unfortunately, if you type it into IBM's APL2/PC in TryAPL2 it reveals a bug in their interpreter! Using the published example, the expression to the right of the $0\ 4\ 1$ should be integer 36, giving a result of 9 0. However, APL2 calculates an expression very close to 36, but gives a result of 8 4, and hence the function returns 8 5, not 9 1.

The IBM stand at APL92 was not impressed, but the Mangled-Logic APL*PLUS and Dyalogdic interpreters gave the "correct" results.

*ED: Thanks John, I should have written $\lfloor 10 \times A$ rather than relying on APL*PLUS to demote the result of the multiplication to integer. Not sure that this is a bug in any interpreter, as Encode is not $\square CT$ dependent and one should not depend on an interpreter demoting floating point numbers to integer. These little things are set to trap the unwary!*

From: David Crossley

July 1992

I am appalled to find that my BAA subscription for private membership costs £20 as I now live in France, as opposed to the UK subscription rate of £12. Why is this? Postal rates to EEC countries are the same as within the UK. So does the BAA have a policy to soak members living outside the UK? I look forward to a change in policy and a refund of £8.

ED: Sorry David, but the postage rates are not the same on packets, only on letters under 20gm (a 220gm Vector costs £0.88 airmail printed paper, as against £0.49 second class postage within the UK). However EC countries are considerably cheaper than the rest of the world, so perhaps the committee should take note.

News from Sustaining Members

Compiled by Gill Smith

Cocking & Drury

Cocking & Drury are delighted to welcome Manugistics as Sustaining Members of the British APL Association. Now news of the latest developments in the world of APL*PLUS will come directly from the horse's mouth — and won't depend on our rather vague attempts to meet the Editor's generous and often, but vainly, extended deadlines!

We at Cocking & Drury continue to be busy assisting with migrations from APL mainframe (mostly APL2) to APL*PLUS II for the 386/486. In one classic encounter a consultant managed to out-run the mainframe with one migrated workspace.

In the course of such work we have developed several tools to ease the work involved in migrating mainframe APL2 to APL*PLUS II for the 386/486. Anyone considering a similar move should call to enquire about features and prices. On October 9th Chris Burke, a UK qualified actuary working with Manugistics' Canadian dealer Strand Software, will be giving a talk on the use of APL in actuarial work. The event is free and attendees will be given copies of the Strand APL utility library which contains several standard actuarial utilities.

Later in October, Chris Lee of Manugistics will be presenting a preview session on Version 5 of APL*PLUS II for the 386/486. This event is also free. For more details and for reservations on either of these, call Beverley Satterley on 071-436 9481.

APL People Limited

Things are still pretty slow on the recruitment front, although we have noticed a slight increase in demand for contract staff. July and August are traditionally quiet months, so we hope that the Autumn will see an increase in the number of vacancies. Companies which are recruiting are taking their time in selecting the right candidate and there is a tendency for interview procedures to become more rigorous.

On the manufacturing side, APL People's sister company, APL Manufacturing, has at long last received an order from British Steel (it's only taken them two and a half years from their initial inquiry!). Orders are imminent from British Dental Floss and British Nuclear Fuels among others. Plans are afoot to re-locate from our present offices in Hounslow to something a bit more prestigious, to take advantage of the excellent value for money of commercial property at the moment.

Reuters APL Software Division

SHARP APL UNIX Version 4 is announced for delivery in the 4th quarter 1992. Supported hardware platforms include the IBM RS/6000 and Sun SPARCstation. This new SHARP APL UNIX version provides extensive facilities for UNIX-UNIX and UNIX-MVS communication. The SHARP APL Network Shared Variable Processor (NSVP) supports LU 6.2 based shared variable communication in between SHARP APL UNIX and SHARP APL MVS, and TCP/IP based shared variable communication for LAN-connected SHARP APL UNIX systems.

The SHARP APL Network Shared Variable Processor can be used for communication between SHARP APL applications and remote SHARP APL auxiliary processors. New stand-alone versions of SHARP APL MVS auxiliary processors are announced for use with SHARP APL UNIX Version 4. These stand-alone auxiliary processors can be installed on an MVS system without SHARP APL MVS, and accessed via NSVP from SHARP APL UNIX. Auxiliary processors available or planned in stand-alone versions include SSQL (for access to DB2), TSIO (for access to MVS QSAM data sets) and FCAP (for calling MVS software written in other languages than APL).

Using NSVP and stand-alone MVS auxiliary processors, SHARP APL UNIX can extract data from non-APL mainframe databases and transfer it to UNIX for processing. The Function Call Auxiliary Processor (FCAP) can be used from SHARP APL UNIX to access any MVS application with a well-defined Application Programming Interface.

Current SHARP APL MVS development activity is centred on performance improvement and connectivity. During 1992, a Performance Upgrade for SHARP APL MVS Version 20 increased performance significantly at large SHARP APL MVS sites, the largest of which support in excess of 1000 simultaneous users of SHARP APL applications.

Within Reuters, a major new SHARP APL MVS service, with world-wide network access, has been established in Frankfurt to support global risk management and trading applications.

For information about Reuters SHARP APL products, please contact Laurie Howard, Reuter Nederland BV, Amsterdam, Tel. +31 20 570 8733, fax +31 20 570 8758; or Larry Moore, Reuter Information Services (Canada) Limited, Toronto, Tel. 416 364 5361, fax 416 364 2910.

Dyadic Systems Ltd

As usual, Dyadic took a stand at the annual APL conference which was held this year in St Petersburg. Getting PCs in and out of Russia proved to be an "interesting" experience that our intrepid representatives will probably not wish to repeat. Nevertheless, everything was operational when the conference opened and the company was delighted by the enthusiastic response to the new version of Dyalog APL for Windows. This product actually began shipping during the conference and the reactions of early users has been equally gratifying. The DDE interface (which allows APL to talk to other

Windows applications) has proved to be a two-edged sword, as we now get support calls from users wanting help with Excell!

Back at base, there was no respite for the developers and work on the next release started immediately. This will include full colour support, object-oriented graphics, Dynamic Link Library (DLL) interface and a host of other improvements. This release will be supplied free of charge to existing customers, and is scheduled for November/December shipment. An interim "maintenance release" was distributed free of charge during September. Any customer who failed to receive this update is urged to return his/her registration card immediately.

Although the production of Dyalog APL/W has been top priority for several months, work has continued on the Unix version of the product. There are two new implementations available, one for SCO Unix (replacing the Xenix version), and one for Digital's RISC DECstations under Ultrix.

Dyadic takes this opportunity to remind VECTOR readers that it is a fully authorised IBM Business Associate (IBM-speak for Dealer) for the RISC System/6000. This now represents a wide range of computers from desktop workstations to large multi-user machines. In Dyadic's opinion, the RS/6000 represents the "best-buy" in the Unix market and is undoubtedly a superb machine for running Dyalog APL. Companies who are still running APL applications on their mainframe would do well to examine the possibilities of "downsizing" onto RS/6000. Not only will you save money, but your applications may actually run faster!. Contact Pauline Brand at Dyadic for further details.

Manugistics

Manugistics, Inc. is very pleased to join the other Sustaining Members of the British APL Association in supporting its many valuable contributions to APL. Among these we particularly value the publication of Vector, which many of us regard as the best, and certainly the most punctually published, journal of APL.

A great deal of activity has been occurring in our company in recent months, not least of which is our name change from STSC to Manugistics. Our new name reflects our application products for manufacturing, logistics and statistics, all of which have found gratifying commercial success. The name change is only that — we remain exactly the same employee-owned company as before, with the same people, the same products, and the same plans for the future.

There likewise need be no concern that our name change signals a change in our commitment to the APL*PLUS product line. Our APL business is growing and solidly profitable. Our last fiscal year produced record high sales throughout the company, and 75% of those revenues came from APL and APL-based application products. We have increased the size of our APL development department, and are looking forward to announcing major innovations in APL on these pages in the months to come.

February 1992 saw the appearance of version 4 of our flagship product, APL*PLUS II for the 386/486, and sales have far surpassed all prior releases in Europe and North America. As our warehouse staff struggled to fill the flood of orders, the development team scarcely paused for breath before setting to work on version 5, which we plan to ship before the end of 1992.

Version 5 will include major improvements to the Microsoft Windows interface introduced with version 4. Most notable of these is the introduction of APLGUI, a high-level object-oriented toolkit that makes Windows programming easy. In keeping with our philosophy of Open APL, APLGUI does not block access from the rest of the Windows software world. One can use a resource editor such as Borland's or Microsoft's to design APL dialog boxes, or build them entirely within APL. APLGUI is built as a layer on top of the APL-Windows interface, so applications that need to can directly access any of the 600+ Windows system calls as well as other vendors' products that are packaged as Dynamic Link Libraries. Those who have no desire to plumb the depths of Windows programming will be able to develop complete applications using only the easy high-level utilities.

Version 5 also will include utilities that simplify the use of Dynamic Data Exchange from APL and a rewrite of the APL-Windows communication facility that lets the DOS-hosted APL*PLUS II interpreter respond to Windows as briskly as if it were itself a Windows program.

Also coming in version 5 is a true debugger that is woven into the fabric of the interpreter itself. The APL*PLUS II debugger permits visual tracing of program code line by line, statement, or individual primitive. Breakpoints can be set on individual statements, and watchpoints can be set on variables with optional APL test expressions. The full capabilities are too numerous to mention, so we recommend that UK readers ring Cocking & Drury (071-436-9481) for details, plus a list of the other delights coming in the new release.

On other fronts, version 11 of APL*PLUS/PC is coming soon, featuring the Screen Interface Toolkit and a facility for converting workspaces into .EXE files, with no runtime interpreter or royalties involved. The UNIX version of APL*PLUS II is now available on Hewlett-Packard's breathtakingly-fast 700 and 800 series workstations and minicomputers. Details on any APL*PLUS product are available from Cocking & Drury. Outside the UK, please contact our International Sales department (Tel: +1-301-984-5412; Fax: +1-301-984-5094).

MicroAPL Ltd

We usually feature news on our various APL interpreter products in Vector; as a change we will be discussing some of the consultancy work we are doing at the moment. We provide consultancy services in three broad areas, strategic management consultancy, applications consultancy for windows-based projects and APL applications consultancy. Overall, our consultancy services account for over half our revenue.

As a company, MicroAPL has now built up some 8 years experience in programming graphical user environments starting with our early work on the Apple Macintosh and coming right up to date with X-Windows and Windows 3. At present the computer industry is experiencing a period of exceptional change with new alliances emerging and difficult choices to be faced over the choice of future hardware and software. Over the last few months we have conducted a number of studies advising clients on their strategic options for IT, focussing particularly on the choice of development tools for new software projects. We have also undertaken an extensive review of the ever growing number of packaged Executive Information Systems.

On the applications consultancy front our non-APL work is becoming progressively more important and we have undertaken work recently using a range of development tools starting with Visual Basic for prototyping and for some simple applications and then moving on to products such as Gupta's SQLWindows or PowerBuilder from Admiral Software, both driving a variety of database servers.

Our APL consultancy work divides between support services for a number of well-established APL systems and a variety of new APL applications. As one would expect the sort of application being written in APL varies widely — we have recently completed work on communications software for the Apple Macintosh; an interpreter to convert Hewlett Packard HPGL instructions into PostScript; and software to format and forward data for an electronic parts ordering system.

I have not included our systems programmers in this analysis since they are primarily involved in development of our software products such as APL.68000. In recent months work here has been focussed on MicroAPL's 'Portable Assembler' product which is now being sold to developers who are moving applications onto the new range of RISC-based Macintoshes which are under development.

THE EDUCATION VECTOR

October 1992

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.9 No.2. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	14
Exploring MAGIC	David Ziemann	16
Some Notes on Direct Definition	Alan Mayer	27

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

Welcome to Education Vector. I note, with some surprise, that I am embarking upon my second year as Editor. My first year has been one in which much has occurred in the world at large, and many plans have been laid. I am grateful to so many readers who have contributed their work, their views and their ideas. My receipt of "email" in particular has increased considerably since I took over the editorship of this journal!

I am particularly pleased with the response to my request for material for use in education. Several people have responded with ideas and offers of help. My own inability to devote enough time to the task has tended to make progress slower than it should be, but I shall endeavour to clear the back-log and make some of the material available as soon as possible. I am still very keen to hear from any schools that would be interested in trying out APL.

In the last issue, Bill McLean contributed a game called MAGIC and issued a challenge to readers: "Any offers of an algorithm (in APL of course) to solve MAGIC problems?". Dave Ziemann has taken up that challenge, and a good deal of this issue of Education Vector is devoted to his solution. Dave's approach to programming by direct definition is instructive, and his article demonstrates very well how this form of APL can be used to "mimic" the thought processes involved in problem-solving. His approach is fundamentally second generation, and he uses APL*PLUS II.

I have added a note of my own on direct definition, including a workspace to provide direct definition facilities, which I hope will help those who do not have this mode of input implemented in their APL, or who have not come across this method before.

Alain Delmotte has asked me to include a small correction to his paper on "Workspace Listing in IAPL". On page 38 of the last issue, the functions $\Delta\Delta HEADINGS$ and $\Delta\Delta LIST$ were "glued" together. I think the line-numbering makes it clear where function $\Delta\Delta HEADINGS$ ends and $\Delta\Delta LIST$ begins. Those few lines should read:

```
[8]   $\Delta\Delta PRINT$  ''
      v

      v  $\alpha$   $\Delta\Delta LIST$  w
[1]  * Successively lists the variables and functions
```

Free Subscriptions to Education Vector!

If you are reading someone else's copy, or know of someone who would like to receive Education Vector, please let the Editor know. Education Vector is distributed free of charge to over a thousand readers each quarter.

If you are interested in APL, you may like to join the British APL Association. Members receive four issues of the journal Vector (144 pages per issue) of which Education Vector is a part. To obtain an application form, write to Rowena Small, 8 Cardigan Road, London E3 5HU. The current subscription is £12 per annum (UK private membership). Student membership is half price.

A final thought. A subscription to the BAA makes an ideal Christmas present for an APLer!

New Email Discussion List

For those who have not come across them before, an Email Discussion list is a forum in which anyone with an EMAIL address can take part. You "sign on" by sending an Email in a specified format to the "list server" address, after which you will automatically receive any correspondence sent to the list. If you wish, you can contribute items yourself. You can, at any time, "sign off" as easily as you signed on.

A list called APL-L has existed for some time, and includes discussion on all sorts of aspects of APL. I am grateful to Alain Delmotte for drawing my attention to a new list, which was created following a workshop at APL92. It is called APLEDU-L, and is specifically concerned with teaching APL.

To subscribe to either list, send a message to:

LISTSERV@UNB.CA (on INTERNET) ... or
LISTSERV@UNBVM1 (from BITNET/EARN)

The message should have no name or subject, and the first and only line should read:

SUBSCRIBE APL-L your-name ... or
SUBSCRIBE APLEDU-L your-name

information about the list and instructions for signing off will be received automatically.

Exploring MAGIC

by Dave Ziemann

Introduction

Thanks to Bill McLean for suggesting an interesting game in July's Education Vector, Vector Vol.9 No.1. In the game of MAGIC a randomly generated matrix of 0s and 1s is displayed, and the player is asked to select from a standard set of matrices (also 0s or 1s). The selected matrix is used to complement the values in the original matrix. The aim is to end up with the target matrix, a matrix of 1s with a lone 0 in the middle.

Bill presented a program that lets the user play MAGIC, and he finished by inviting an APL algorithm that solves MAGIC puzzles.

I wanted to use direct definition to express my ideas, and the problem also suggested the use of nested arrays, so I used APL*PLUS II with a direct definition handler I developed a few years ago.

I had great fun programming my thoughts, the results of which I submit for your consideration. I don't pretend that this flow-of-consciousness coding style has resulted in either a definitive or a concise solution, but it does demonstrate that direct definition can be used to explore ideas very rapidly, without interruptions caused by the superfluous syntax of traditional function definition methods. A global index origin of zero ($\square IO \leftarrow 0$) is used throughout.

Representing MAGIC Boxes and Keys

The nine boxes, which I have renumbered 0-8, are as follows:

1 1 0	1 1 1	0 1 1
1 1 0	0 0 0	0 1 1
0 0 0	0 0 0	0 0 0
0	1	2
1 0 0	0 1 0	0 0 1
1 0 0	1 1 1	0 0 1
1 0 0	0 1 0	0 0 1
3	4	5
0 0 0	0 0 0	0 0 0
1 1 0	0 0 0	0 1 1
1 1 0	1 1 1	0 1 1
6	7	8

A short MAGIC game can be summarised by the following sequence, which begins with a random initial box, and ends with the winning target box. The numbers between the boxes are the key numbers required to effect the transitions:

0 0 0		1 1 0		1 1 1		1 1 1
1 0 0	0	0 1 0	5	0 1 1	6	1 0 1
0 0 0		0 0 0		0 0 1		1 1 1

First we need to be able to enter the binary representation of each of the nine keys. The following functions do that:

```
boxes012: 3 3 3p'1'='110110000111000000011011000'
boxes345: 3 3 3p'1'='100100100010111010001001001'
boxes678: 3 3 3p'1'='000110110000000111000011011'
```

These are examples of niladic directly defined functions; they do not take any arguments, but return constant results. The nine boxes can be structured into a vector of nine matrices as follows:

```
boxes: +~++boxes012,boxes345,boxes678
boxes
1 1 0 1 1 1 0 1 1 1 0 0 0 1 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 1 1
0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 1 1 1 0 1 1
```

Although we could deal with binary matrices directly, I have chosen to use a different representation, namely the decimal value of the nine-bit number that represents the matrix. (This is discussed further in Appendix A.) We will need two utilities for converting between the forms:

```
bd: 21,w
db: 3 3p(9p2)Tw
db 123
0 0 1
1 1 1
0 1 1
```

The omega symbol represents the function right argument. Now we can represent every state as an integer scalar from 0 through 511. Let's convert the winning state matrix into a decimal value:

```
win: bd 3 3p'1'='111101111'
win
495
```

We can convert the integer values to binary matrices when we want to perform the complementing operation which is at the heart of this game:

```
stateComp: bd (db α)≠(db ω)
123 stateComp 42
81
db"123 42 81
0 0 1    0 0 0    0 0 1
1 1 1    1 0 1    0 1 0
0 1 1    0 1 0    0 0 1
```

A function that returns the decimal value of the box represented by each key number will be useful:

```
key: bd"boxes[ω]
key :9
432 448 216 292 186 73 54 7 27
```

Generating the State Table

Now we can generate the state table for this system. It has one row per input state, and one column per selected key. Each cell in the result contains the target state reached by applying each key to the corresponding input state. The state table is easily constructed by applying the *stateComp* function between all combinations of input state and key number:

```
stateTableΔ: (1512)*.stateComp key 19
pstateTableΔ
512 9
```

You may not have noticed the delay just then, but the above expression took 12 seconds to execute on a 33Mhz 486. To speed things up a bit, we can write a cover function:

```
stateTable: StateTa+stateTableΔ : 2=□nc'StateTa' : StateTa n/G=StateTa
```

This uses the three segment form of direct definition. If the middle segment, the condition segment, evaluates to 1 then the right segment is executed. On the other hand, if the condition segment evaluates to 0 then the left segment is executed.

The *n/G=* is a little trick that tells the direct definition handler to leave the named variable as global (the handler normally localises all variables to the left of an assignment arrow).

Although *StateTa* is technically a global variable, it is not referred to elsewhere. It is actually being used to emulate "state memory", a kind of name scope which is not available in APL, where the function remembers the value of the variable from the previous time it was called. From now on, executions of this function will result in an immediate response.

Now we can search the state table for any state we are interested in, returning the input states (row numbers) of the state table that lead to the argument state:

```
statesFrom: where stateTablev.εw
where: w≠pw : 2=⊂nc'α' : w≠α
statesFrom win
47 95 203 311 341 422 473 488 500
```

The dyadic form of the *where* utility will be used later. The alpha in the definition of *where* represents the function's left argument, if supplied.

Generating the State Transition Vector

Our algorithm will be to use *statesFrom* to trace backwards from the winning state, through each prior state, until all 512 states have been accounted for. On the way we can build a vector of state vectors, which will be as long as the longest minimal winning sequence.

So we must now build on *statesFrom* as follows:

```
stepBack: (statesFrom first w) without flatten w
without: α~w
first: w
flatten: ⊃./w : 0εpw : 10
stepBack win
47 95 203 311 341 422 473 488 500
```

Notice that *stepBack* assumes that its argument is a vector of vectors, the growing vector of state vectors that will be our result. The *without* utility is used to ensure that any states we have already visited are not duplicated in the result, and so avoids a potentially infinite loop in the recursive function that we must now grit our teeth and write. Here it is:

```
traceBack: traceBack w append1 stepBack w : 0εpstepBack w : w
append1: {w},α
```

When *stepBack* has visited every state, it returns an empty result, and this is the terminating condition for the *traceBack* function. Until this condition is met, the function will continue to append enclosed vectors of states to the left

end of the result. Let us "burn in" the expression that computes the complete set of state transitions, from the winning state to the earliest state, into a function, and then with mounting excitement try it:

```
stateTransΔ: traceBack ,c,win
p"stateTransΔ
10
```

That took 9 seconds to run, so before looking at the result, we can use the same technique as before to hide the delay:

```
stateTrans: StateTr+stateTransΔ : 2=Unc'StateTr' : StateTr a/G=StateTr
p"stateTrans
1 9 36 84 126 126 84 36 9 1
```

Each item of this 10 element vector contains states in a single "pool". To play a successful game of MAGIC it is necessary to select a key which moves us from a given state pool into the next one to the right, until we reach the target. We can see that the longest possible game involves nine transitions, from a single state in the starting pool, to a single state in the final pool. The distribution of states in the transition vector is symmetrical, which is to be expected from the commutative nature of the complementing (not-equals) function.

There is only room to view a fragment of the state transition vector here:

```
table: ((pw),1)pw
12 80+* table"stateTrans

186      0      7      18      3      1      2      19      47      495
      98      27      23      6      4      8      22      95
      140     43      24      9      5     13      25     203
      161     46      28     12     10     17      40     311
      189     54      29     16     11     20      52     341
      243     73      41     21     14     30      68     422
      266     84      44     26     15     32      88     473
      378     94      45     31     34     37     102     488
      414    101      48     33     35     51     105     500
           121      49     36     38     56     113
           139      53     42     39     61     130
           151      69     50     58     67     135
```

We can look at the bit patterns for the first and last states:

```
stateTrans[0 9]
186 495

db"stateTrans[0 9]
0 1 0      1 1 1
1 1 1      1 0 1
0 1 0      1 1 1
```

It is also possible to verify that every possible state is represented exactly once in the state transitions vector:

```
sort: ω[Δω]
^/{1512}=sort flatten stateTrans
1
```

Generating the Key Transition Vector

The question remains, which keys do we need to press to effect the transition from one state pool to the next? This depends on the input and target states. We have to find the column number in the state table that contains the target state in a row corresponding to the input state:

```
keyBetweenStates: where"↑stateTable[α;]εω
186 keyBetweenStates 1
```

This last expression produces an empty result because it is not possible to get from state 186 to state 1 in a single transition (see the state transition vector fragment above). Using states in contiguous state pools should give us a result:

```
186 keyBetweenStates 0
4
186 keyBetweenStates 98
2
```

The function will work for vector arguments, returning for each state in the left argument a vector of key numbers that will move us into the state pool on the right:

```
table 0 98 keyBetweenStates 43 73 101 121
5
5 7 8
```

This tells us that state 0 needs key 5 to gives us one of the states in the right argument (we don't know or care which), and that state 98 can get there via any of the keys 5, 7 or 8.

We can apply this function between a pair of state pools using APL's reduction operator:

```
table >keyBetweenStates/stateTrans[1 2]

0 1 2 3 5 6 7 8
0 1 3 4 5 6 7 8
0 1 2 3 4 5 7 8
0 1 2 3 4 5 6 7
0 1 2 3 4 5 6 8
0 1 2 3 4 6 7 8
1 2 3 4 5 6 7 8
0 2 3 4 5 6 7 8
0 1 2 4 5 6 7 8
```

Notice that there is only one "wrong" key you can select for each state in pool 1. Similarly we can find that there are only two wrong keys that can be pressed to move from pool 2 to pool 3:

```
ρ↑↑keyBetweenStates/stateTrans[2 3]
36 7
```

To find the key numbers that can be used to effect all of the state transitions, we must apply the function between each contiguous pair of vectors in the state transition vector, as follows:

```
keyTransΔ: keyBetweenStates/9 2ρ(1 8 1/1 2 1)/stateTrans
keyTrans: KeyTr←keyTransΔ,c,⊖0 : 2=⊖nc'KeyTr' : KeyTr a/G=KeyTr
```

The expression in brackets duplicates each vector in *stateTrans* except for the first and last. The reshape turns the argument into a 2-column matrix, each row being a pair of contiguous state pools. These two expressions are here left ungeneralised for clarity. Users with APL systems that include a dyadic N-wise reduction operator could code this more directly. A final empty element is catenated onto the result in the *keyTrans* cover function in order to make it conform with the *stateTrans* vector.

We can pair each state transition vector with its corresponding key transition vector, and compactly display them using *fmtTrans*.

There is only room here for a few rows of the first 6 elements of the result:

```
fmtTrans: stateTrans pair"keyTrans
pair: α,fmt"table ω
fmt: 1 0↖ω
```

```

      +12+4DEB*6+fmtTrans
186 012345678 0 01235678 7 0123568 18 345678 3 12457 1 0134
      98 01345678 27 0123567 23 024678 6 01347 4 1245
      140 01234578 43 0134678 24 124578 9 34567 5 1678
      161 01234567 46 1245678 28 012356 12 02467 10 0578
      189 01234568 54 0123578 29 013478 16 02468 11 0246
      243 01234678 73 0123678 41 124568 21 34568 14 3456
      266 12345678 84 0134578 44 013468 26 01348 15 2378
      378 02345678 94 0245678 45 012357 31 12458 34 2367
      414 01245678 101 0134568 48 013467 33 02478 35 3458
      121 0134567 49 012358 36 34578 38 0248
      139 0123458 53 124567 42 01235 39 0567
      151 0123457 69 024567 50 12456 58 0247

```

DEB is a utility function that deletes extraneous blank columns. It was copied from the APL*PLUS II distributed workspace ASMFNS. Notice that the number of "correct" keys we can select is always the same in any given state pool, and decreases by one in subsequent state pools. We can incidentally compute the total number of correct possible moves in each state pool by multiplying the number of states in the pool by the number of keys that can be pressed to move us into the next pool:

```

      ~1+(⊖⊖stateTrans)*p~stateTrans
9 72 252 504 630 504 252 72 9

```

Solving MAGIC Puzzles

A function to generate the key numbers that must be selected for a given state is:

```

      keysAfterState: keyTrans pickWith (stateTrans pickIndex w)
      pickWith: w⊃α
      pickIndex: (α find w),(α pickWith α find w)⊃w
      find: where wε"α
      keysAfterState 186
0 1 2 3 4 5 6 7 8
      keysAfterState 42
0 1 2 3 5
      keysAfterState 500
8

```

We don't really care what state we get into when we select a key, so long as it's getting us into the next state pool, one step nearer to the target. So when we have a choice of key, let us arbitrarily agree to use the one with the lowest key:

```

      keyAfterState: pickOne keysAfterState w
      pickOne: ⊃w
      keyAfterState 186
0

```

The *pickOne* function could be modified to pick a different key, as required. Now we can trace forward through the state transitions, computing a key that

will get us into the next state pool. It will be more useful to develop a function that returns two numbers, a key number and a target state, given an input state:

```

afterState: w afterKey keyAfterState w
afterKey: w, a stateComp key w
186 afterKey 0
0 266
  afterState 186
0 266

```

By tracing forward through the transitions vectors we can generate the key number/state pair for every transition, starting from a given state:

```

traceFromState: traceForw <afterState w : win=w : 10
traceForw: traceForw w appendr afterState final w :
win=final w : w
  appendr: a, c w
  final: last flatten w
  last: => φ, w

traceFromState 186
0 266 1 202 2 18 3 310 4 396 5 453 6 499 7 500 8 495

```

We can write a formatting utility to make this result look a little more presentable:

```

fmtKeyState: table (first w)(db key first w)(db last w)
fmtKeyStates:(<table('')(db w)),fmtKeyState"traceFromState w

fmtKeyStates 32
      0      5      6
0 0 0    1 1 0    0 0 1    0 0 0
1 0 0    1 1 0    0 0 1    1 1 0
0 0 0    0 0 0    0 0 1    1 1 0
      1 1 0    1 1 1    1 1 1
      0 1 0    0 1 1    1 0 1
      0 0 0    0 0 1    1 1 1

```

The first row of the result contains key numbers, the second contains input states, and the third contains target states achieved by the corresponding key presses. The leftmost box in the second row is the initial state, 32 in this case. The final box in the last row is the winning target state.

A more compact form of display is required to show the longest winning line:

```

show: fmt""(c<db w),fmtKeyState"traceFromState w
show 186
010  0      1      2      3      4      5      6      7      8
111
010  110    111    011    100    010    001    000    000    000
     110    000    011    100    111    001    110    000    011
     000    000    000    100    010    001    110    111    011
     100    011    000    100    110    111    111    111    111
     001    001    010    110    001    000    110    110    101
     010    010    010    110    100    101    011    100    111

```

Notice that you can get there by pressing each key once, in the order 0 to 8. We can write a function to apply a series of keys to an initial state:

```

applyKeys: *\\db"α, key"w
186 applyKeys 0 1 2
0 1 0    1 0 0    0 1 1    0 0 0
1 1 1    0 0 1    0 0 1    0 1 0
0 1 0    0 1 0    0 1 0    0 1 0

```

and then use it to actually play MAGIC, and verify that the solution is correct for any starting state:

```

checkSolution: fmt"w applyKeys first"traceFromState w
checkSolution 186
010  100  011  000  100  110  111  111  111  111
111  001  001  010  110  001  000  110  110  101
010  010  010  010  110  100  101  011  100  111

```

Having finished this exercise, I am left with the feeling that there is probably a very simple way of knowing what key to press from any given state, but it eludes me at the moment...

Appendix A – Representation of State

It is possible to retain the 3 by 3 binary matrix representation of MAGIC states throughout the computations, rather than use the scalar decimal value, as described above. As far as the code is concerned this would make the conversion utilities *bd* and *db* unnecessary, and in turn the *stateComp* function as well. The *stateTable* function could then use a not-equals outer product directly, which would speed up the calculation of the state table.

However, the state table would now also dramatically increase in size. From an 18k integer matrix to a very nasty 165k matrix of enclosures. Furthermore, the current algorithm is made simpler because the state matrix is self-indexable, that is, the rows of the state matrix can be indexed by the decimal representation of any state within the state matrix.

Appendix B — Alphabetic Function Listing by Category

Generating the State Table

```
stateComp : bd (db a)*(db w)
stateTable : StateTa+stateTableΔ : 2=□nc'StateTa' : StateTa
             a/G=StateTa
stateTableΔ : (1512)*.stateComp key 19
```

Generating the State Transition Vector

```
stateTrans : StateTr+stateTransΔ : 2=□nc'StateTr' : StateTr
             a/G=StateTr
stateTransΔ : traceBack ,c,win
stepBack : (statesFrom first w) without flatten w
traceBack : traceBack w appendl stepBack w : 0cstepBack w : w
statesFrom : where stateTablev.cw
```

Generating the Key Transition Vector

```
keyBetweenStates : where"+stateTable[a;]cw
keyTrans : KeyTr+keyTransΔ,c,c10 : 2=□nc'KeyTr' : KeyTr
             a/G=KeyTr
keyTransΔ : keyBetweenStates/9 2p(1 8 1/1 2 1)/stateTrans
```

Solving MAGIC puzzles

```
afterKey : w,a stateComp key w
afterState : w afterKey keyAfterState w
applyKeys : *db"a,key"w
keyAfterState : pickOne keysAfterState w
keysAfterState : keyTrans pickWith (stateTrans pickIndex w)
traceForw : traceForw w appendr afterState final w : win=final w : w
traceFromState : traceForw cafterState w : win=w : 10
```

General Utilities in this Application

```
appendl : (cw),a
appendr : a,cw
final : last flatten w
find : where wc"a
first : >w
flatten : >,/w : 0cpw : 10
fmt : 1 0w
last : >φ,w
pickIndex : (a find w),(a pickWith a find w)w
pickWith : w>a
sort : w[Δw]
table : ((pw),1)pw
where : wfpw : 2=□nc'a' : w/a
without : a~w
```

Application Utilities

```

bd      : 21,ω
boxes   : ↑↑↑boxes012,boxes345,boxes678
boxes012 : 3 3 3p'1'='1101100001110000000011011000'
boxes345 : 3 3 3p'1'='100100100010111010001001001'
boxes678 : 3 3 3p'1'='000110110000000111000011011'
db      : 3 3p(9p2)τω
checkSolution: fmt"ω applyKeys first"traceFromState ω
fmtKeyState : table (first ω)(db key first ω)(db last ω)
fmtKeyStates : (ctable('')(db ω)),fmtKeyState"traceFromState ω
fmtTrans    : stateTrans pair"keyTrans
key         : bd"boxes[ω]
pair        : α,fmt"table ω
pickOne     : ωω
show        : fmt""(←db ω),fmtKeyState"traceFromState ω
win         : bd 3 3p'1'='111101111'

```

Some Notes on Direct Definition

by Alan Mayer

Many readers of Education Vector use Direct Definition routinely to input their APL functions, and will probably wonder why I have bothered to include these notes.

Now, for those who are still with me ... Direct Definition is an alternative way of entering APL functions into a workspace. In style it is similar to formal functions in mathematics, and it is popular with people who have been trained in mathematics, or whose brain happens to work that way. Personally I think it is well worth exploring — like APL itself, you don't know what you're missing until you've tried it.

Some APL interpreters support direct definition as standard, others do not have it at all. For the latter case, I include at the end of this article a "DIY" DD Workspace (based on one by Ken Iverson, as found in 'A Source Book in APL' reprinted from the 1980 IP Sharp Users' Meeting in Toronto, but modified by various people since). DD functions are presumed to produce a result, and the left and right arguments (if they exist) are referred to as α and ω . To enter DD mode, first type *DEF* (in some versions this may be *DEFINE*). Then type in the name of the function you want to define, followed by a colon (:). After the colon, type in the APL formula which has to be evaluated to produce the required result, using α s and ω s as necessary.

Examples

(a) to calculate a square root:

```

DEF          <===>          ∇ Z+SQRT X
SQRT: ω*÷2          [1]      Z+X*÷2
                      ∇

```

(b) to round a value to a specified number of places:

```

DEF          <===>          ∇ Z+N DP X
DP: (⌊.5+(10*α)×ω)÷10*ω    [1]  Z+(⌊0.5+(10*N)×X)÷10*N
                      ∇

```

It is quite easy to translate back from DD to regular function definition. First extract the function name, and look for α s and ω s. If there are any α s, you want a dyadic function; if there are ω s only, you want a monadic one; and if there are neither, the function is niladic (no arguments). Accordingly, your header will be $Z+A$ FUNCTION B , or $Z+FUNCTION$ B , or just $Z+FUNCTION$. The formula must then be expressed in APL, using A for α , B for ω and assigning the result to Z . If your system does not support DD, you can build your own workspace as follows:

```

∇ DEF;∇IO
[1] ∇IO←1
[2] ∇FX F9 ∇
∇

∇ D←F9 E;F;I;K
[1] F←(, (E='ω')×.÷5+1)/,E,(ϕ4,ρE)ρ' Y9 '
[2] F←(, (F='α')×.÷5+1)/,F,(ϕ4,ρF)ρ' X9 '
[3] F←1+ρD+(0,+/-6,I)+(-(3×I)++I+':=F)ϕF,(ϕ6,ρF)ρ' '
[4] D←3ϕC9[1+(1+'α'∈E),I,0;],ϕD[;1,(I+2+iF),2]
[5] D[1;]+(1+ρD)†(¯2×-( 'ω'∈E)∨'α'∈E)+D[1;]
[6] K←K+2×K<1ϕK+I∧K∈(> 1 0 ϕ'+∇'×.÷E)/K+∨-I+E∈A9
[7] F←(0,1+ρE)[ρD+D,(F,ρE)+ϕ 0 ¯2 +Kϕ' ',E,[1.5]';'
[8] D←(F+D),[1]F[2]†'α',E
∇

A9←'0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ∇Δ'
C9← 5 11 ρ'ΔΔΔZ9+ΔΔΔΔΔΔY9Z9+ΔΔΔΔΔΔY9Z9+X9ΔΔΔ)/3+(0=1†,ΔΔΔΔ+0,0ρZ9+

```

Note: in the definition of $C9$ the symbol Δ represents a space. These spaces must be counted carefully! (A small prize is offered to anyone who gets this right in 1!)

With these four objects in your workspace, you will have direct definition available for any APL environment. Only one restriction — you can't name objects DEF , $F9$, $A9$ or $C9$.

THE RANDOM VECTOR

The Newsletter of the APL Statistics Library
Editor David Eastwood

October 1992

Contents

Editorial	David Eastwood	30
ASL - The Future	Jake Ansell	31
ASLGREG reviewed	Jon Sandles	32
Review comments	Alan Sykes	34
ASLGREG corrections	Alan Sykes	35
A Volume for the Reliability Shelf	Jake Ansell Mulhim Al-Doori	37

Editorial

by David Eastwood

In this issue of The Random Vector, we have a mixture of articles, reviewing the current state of the ASL and introducing some ideas for future developments. Jake Ansell, the ASL Chairman, gives a brief review of the current status of the project and also introduces a proposal for another volume for the ASL based on some work he and a colleague are carrying out at the moment.

Now that two volumes of the ASL are on general release, we are beginning to get some useful feedback on the software. I am pleased to include a review of ASLGREG, the Linear Regression volume, which has been submitted by Jon Sandles of Nestlé-Rowntree. Alan Sykes has submitted a number of minor corrections to the ASLGREG software and has also responded to some comments made in the review of ASLGREG.

If you would like to order copies of the ASL software, order forms are available from both the ASL Management Committee and from the BAA Administration (Rowena Small).

As usual, I give contact addresses for the ASL Management Committee:

Chairman: Jake Ansell, Business Studies, University of Edinburgh
William Robertson Bldg, 50 George Square, Edinburgh EH8 9JY
Tel: 031-650-3806

Deputy Chairman: Alan Sykes, European Business Management School,
University of Wales, Singleton Park, Swansea, SA2 8PP
Tel: 0792-295296

BAA Representative: David Eastwood, MicroAPL Ltd, South Bank Technopark,
90 London Road, London SE1 6LN Tel: 071-922-8866

ASL - The Future

by Jake Ansell

The experience gained over the production of ASL software shows that 'voluntary' software development is a slow process. This is not only true of ASL for it occurs with other software. For example users of GLIM are still awaiting the imminent arrival of GLIM 4.

The problems that arise are manyfold. The initial burst of software follows the interest of the author. Hence often a volunteer will provide the interesting element where thought is necessary. The difficulty arises of getting the other functions, which are more basic, written. These are often supposedly trivial functions which are required to provide either general utility or a minor support function.

ASL was originally fortunate that both Alan Sykes and Tony O'Hagan had the desire and commitment to complete their tasks. Hence we have the two volumes: ASLGREG and ASLGRAPH. Unfortunately the Basic Statistics Volume of the Bottom Shelf has existed in embryonic form for too long. Hopefully by the time this edition of Vector is published it too will be available.

It is time to stress though that the ASL project continues and that there will be future releases of software. In this edition of Vector an article appears detailing a suggested volume of the Reliability shelf. There are other shelves in the process of production.

Beyond this there are a number of possible shelves to be constructed/developed. However, these shelves will only be added if individuals are willing to come forward to assist. In the past there has been considerable effort given on a voluntary basis and many need to be thanked. But the project will require further assistance.

At the present time ASL has developed an in-house coding standard. It also has had a team of willing software testers who have helped considerably the production of the current ASL software. If there are statisticians or others with code which they think would be appropriate for ASL then they should contact either Alan Sykes or myself at addresses given above.

THE SUCCESS OF THE PROJECT IS IN YOUR HANDS.

ASLGREG — The Regression and Generalised Linear Models Workspace

reviewed by Jon Sandles

Having only worked in an APL environment for just under a year I feel I can only really review this package on a statistical level. I have not yet developed enough prejudices in the APL world to inflict on you all. (Some might argue otherwise!) This workspace is the first of the regression volumes and contains 'functions that perform GLIM type calculations in a way that allows them to be incorporated into user functions.'

Data Entry

There are no utilities included within the workspace to help with data entry. But remember that the package is written as a set of independent utilities. Hence, you can bolt on your own preferred data management utilities. All the package expects is a set of APL vectors for analysis.

Data Analysis

The documentation for this workspace starts off with an excellent introduction to regression which I would recommend to anybody. Then we are presented with some examples of the code working on a real problem. Working through this, the function names seem to have a Genstat type feel to them, but without some of the tedious features of that package (e.g. the `term` command). You simply declare the dependent variable using `YVAR varname`. You can then try to fit explanatory variables using `FIT 'GM+VAR1+VAR2....etc'`. The term `GM` is the intercept coefficient and is required if you are not trying to fit a line through the origin (referring to Grand Mean vector). A fit produces results of the form:

<i>Source</i>	<i>ss</i>	<i>df</i>	<i>ms</i>
----	--	--	--
<i>Due to model</i>	9864.6	1	9864.6
<i>Residual</i>	12576	27	465.78

<i>Total corrected</i>	22441	28	801.45
 <i>Percentage variation accounted for =</i>			43.96
<i>F-statistic</i>			21.18
<i>p-value =</i>			0.00

Presentation of results throughout the system is kept to the bare minimum of requirements — after all we can all add our own little display functions if we want.

The precision throughout the system is controlled by `PP`. Estimate values can be displayed, as can fitted values and t-residuals. Quadratic relationships can be fitted. Qualitative variables can be fitted by declaring a factor using *FACTOR varname*. The results of fitting a parameterisation are displayed using corner-point parameterisation, which again is similar to Genstat. As the documentation points out, it is a piece of cake to convert these into group mean estimates using a little APL.

Advanced features

A function called *TESTHYP* allows you to test several hypotheses at once and judge whether the reduction in residual sum of squares due to the inclusion of extra variables is significant. For those of us who want all the work done for us (that's me) there is a *STEPWISE* function which runs through the potential explanatory variables selecting/rejecting the ones which explain most/least of the variation in the dependent variable, depending on the F-levels that you pass it.

More advanced features include weighted least squares, leverage, cooks distance, which allow us to progress even when some of the Normal assumptions start to collapse. There are nine link functions (Genstat had eight) and six error distributions (Genstat had 5). But the beauty of these is that with a little help from the example in the documentation it becomes very easy to customise your own link functions.

Results

How does it all work in practice? On the whole it all seems to work correctly. The type of environment feels similar to interactive Genstat. Of course not all the functionality of that package is contained within this one volume; bits missing are the prediction-type functions and the fitcurve-type functions. But again it is possible to bolt these features on yourself. Missing values do not seem to be supported at all. Genstat allows regression on data sets with missing values in, although whether this is really required can be argued. It would certainly be worthwhile considering missing values at this stage in the evolution of ASLGREG as the method of representation could be a contentious issue.

Summary

Using a set of regression functions from within APL seems to me to be the ideal environment for manipulating data. In the past I have felt frustrated at not being able to get quite what I want from existing packages. These functions (and the documentation) are at a low enough level to build on if you require a more advanced interface — for example graphics etc., but they are also at a high enough level for them to run unaided and produce the required results on the screen. One slight gripe is that there seem to be a few hardcoded lengths to format results on the screen, and if you run the functions with pathological data it can crash trying to put pathological results on the screen ... some would say it is the best way to draw attention to such data errors!

Availability

ASLGREG is available from the British APL Association, c/o Rowena Small at the address on the inside back cover. The cost is £10 plus £2 postage and packing (UK). For order forms, please contact Rowena.

Comments on the ASLGREG Review

by Alan Sykes

I am grateful to Jon Sandles for his careful review of ASLGREG. I think it is fair to say that his views on the software have been echoed by other people who have used it, which is very heartening to those of us who have been involved in its production! It has now been available for some 10 months, and in that time only five bugs have materialised. They are easily put right and the corrections are listed below.

Jon raises a number of important points that deserve some response. On the question of prediction, I do have a beta-test version of a function *PREDICT* which allows the user to obtain predicted values corresponding to given values of predictor variables. (This is not a simple function to write in view of the generality of model specification!) Further work might perhaps incorporate confidence intervals. Such a function would be useful in producing graphical output e.g. plotting fitted values. (On this subject Norman Thomson has sent me some first thoughts on graphical tools for ASLGREG, but I have not had time to do further work in this direction as yet.)

I agree that there may well be problems of formatting and welcome specific examples to aid in correcting this. (Note however that the user does have the option of outputting straight unformatted numbers to the screen.)

Finally, on the question of dealing with missing values, it is not quite true to say that there are no provisions. Suppose that all your regression data was contained in a matrix M with missing values indicated by 9999. Then APL comes to the rescue to create the zero-one vector

$$WT \leftarrow \wedge / M \neq 9999$$

and this can be declared as a priorweight vector so that the regression analysis is performed only on the data points for which all information is available. However, I agree that no further missing data facilities such as those in GENSTAT are available.

Corrections to Known Bugs in ASLGREG (September 1992)

by Alan Sykes

Since its release, five software defects have been discovered in ASLGREG. These may be easily rectified by following the instructions below. (Not necessarily all the defects will be pertinent to your copy, and copies dated September 1992 will incorporate all the changes.)

1. *YVAR*

The function *YVAR* contained a local variable ' S '. All references to it (including the header) should be replaced by the variable name ' ΔS ' so that a statement such as

```
YVAR 'S'
```

will work as required.

2. *ORTHOG*

The function *ORTHOG* in its original form is a complete mystery to me! Somehow it got corrupted: fortunately it is short so here is the correct version.

```

      VA←ORTHOG A;M;K;T;TT
[1]  M←(ρA)[2]
[2]  K←1
[3]  lp:A[;K]+T+T÷(T+.×T+A[;K])×.5
[4]  A[;K+1M]←TT-T÷.×T+.×TT+A[;K+1M]
[5]  →(M≥K+K+1)/lp
      V

```

3. MODEL

In line 8 of *MODEL*, the links variable was originally indexed on *id* and not *il*, amend the line to read as below.

```

      VR←MODEL
[8]  ⚡(0≠NC 'il')/'R+R','Link Function  ','links[il;],' ⚡''

```

4. MUHAT

The function *MUHAT* did not refer to the offset variable. Amend it as below noting the crucial space within the quotes before the word 'offset'

```

      VΔR←MUHAT
[1]  ⚡(0≠NC 'dis')/'START'
[2]  ⚡(inhib)/'ΔR+emess[15;]⚡+0'
[3]  ΔR←⚡'ΔMU',(⚡il),' offset+desmat+.×parest'
      V

```

5. The CHECK function

The *CHECK* function contains a superfluous line in the section pertaining to checking whether a variable is a vector or matrix of positive integers. In APL*PLUS, APL*PLUS II, Dyalog APL, and APL68000 this consists of deleting line 29 of Δ*CHECK*, whereas in APL2 it consists of deleting line 62 of Δ*CHECK*. The amended section, with the line deleted is shown below:

```

[26] ΔCHECK VARIABLES ARE POSITIVE INTEGER VARIABLES
[27] four:TΔ←⚡1+,((DΔ,6)ρ' {0=^/,'}),NAMESΔ,((DΔ,2)ρ'={'}),NAMESΔ,((DΔ,2)ρ'),'
[28] TΔ←TΔ∨⚡1+,((DΔ,7)ρ' {1=∨/0≥'}),NAMESΔ,((DΔ,2)ρ'),'
[29] →(1≠IΔ∨∨/TΔ)/lp1
[30] RΔ←RΔ,IΔ/linefd,emess[5;],,TΔ/NAMESΔ,' '
[31] →lp1

```

A Volume for the Reliability Shelf - Reliability Data Analysis

by Jake Ansell and Mulhim Al-Doori

Introduction

It has always been assumed that software would be added to ASL through the efforts of individuals or groups. However it would not be wise if no opportunity were given for others beyond the authors to comment at an early stage on the material and the functions developed.

In this article a brief description of the functions which are contained within the *Reliability Data Analysis* volume is given. A beta-test version of the software is to be produced in the near future. Before this though it would be helpful to have the views of others on the functions included and those they feel should be added.

The software discussed is part of a larger project which is the development of an Expert System for Reliability Data Analysis. The Expert System is being developed in APL. A subsequent article may describe the details of the Expert System.

APL is a natural language for this volume since most of the algebra is linear in form, either based on raw data or on the ordered statistics. Hence for the most part the routines follow the mathematics without the need for great sophistication. There is though the need for iterative procedures, especially Newton-Raphson-based approaches for estimation.

The data analysis volume is assumed to be part of a larger shelf where there would be routines for Reliability Analysis and Modelling. Fault tree programmes might be written based on APL routines for logic manipulations for analysing systems. Also a Stochastic Model could be added following work done at Swansea under the supervision of Alan Hawkes.

In the future it may be that the Data Analysis volume might share code with related shelves, such as a Survival Analysis Shelf or a more general Medical Statistics Shelf. This emphasises that the software described is only a first attempt, and not the definitive model for the volume. Contributions are

welcomed and as authors we would be delighted to see the shelf grow. The names used for functions might equally transform with suggestions from others.

At present we see the possibility of six sections to the volume. These are:

- Elementary Analysis
- Graphical Analysis
- Distributions
- Covariate Analysis
- Growth Models
- Dependency Analysis

Elementary Analysis

As with the bottom shelf this is seen as a collection of routines which have been found to be useful in the analysis of reliability data, especially in the initial stages of such an analysis. It augments the Bottom Shelf. For example the coefficient of variation (*COEFVAR*) and the index of dispersion (*INDIS*) are added since they indicate whether the data is under- or over-dispersed compared to the Poisson Process. This aids in model selection. Also included is a function to calculate the cumulative mean time to failure, since again many practitioners find it a useful measure.

For chronological data Ascher has frequently suggested the need to test for trend. Hence the volume contains two of the standard trend test measures, the Laplace Trend (*TRENDLP*) and the MIL-HNBK-189 (*TRENDMIL*) test.

Graphical Analysis

Graphical analyses are often used in Reliability. Engineers find them comprehensible and informative. They can give rise to parameter estimates, but more appropriately can indicate the validity of the model. It is assumed that the volume will provide just the values to be plotted and not the plotting routines, which should exist within ASLGRAPH. Hence the volume will produce X-Y coordinates for plotting.

The routines cover the standard plots to assess lifetime distributions such as Weibull (*WEIBPLOT*), Hazard (*HAZPLOT*) and TTT (*TTTPLOT*) plots. These plots often shed light on the appropriateness of a distribution but may also indicate possible problems with the data. A function also exists to produce a Duane Growth Plot (*DUANEPLOT*). This plot is often used to assess whether time between failures is increasing (good) or decreasing (bad).

Beyond these there will be functions to produce plots for more sophisticated analysis, such as residual plots for covariate analysis (see Covariate section) and for examining dependency such as Walls' method (*WALLSPLOT*).

Lifetime Distributions

Obviously looking into the future there should be a separate shelf for distributions, covering the common distributions. Already, though, certain aspects of some distributions have been covered by the Bottom Shelf. In the interim it seems sensible to include some routines associated with lifetime distributions in this volume. (A lifetime distribution is generally taken to be a distribution which takes only positive values.) Hence included in this volume are routines for parameter estimation for a number of lifetime distributions.

Given the reliability context these functions allow for the possibility of censoring, currently right-hand censoring. The distributions considered are the Exponential, both one- and two-parameter forms (*EXPEST*, *EXP2EST*), the Weibull, two- and three-parameter forms (*WEIBEST*, *WEIB3EST*), the Gamma (*GAMMAEST*) and Log Gamma (*LGAMMAEST*).

Since it is also possible to obtain empirical estimates for distributions we have included Kaplan-Meier for reliability function (*KMEST*) and Nelson-Altschuler for cumulative hazard function (*NAEST*).

Covariate Analysis

Cox's Regression Model, referred to as the Proportional Hazard Model (PHM), has led to a resurgence of interest in modelling lifetimes through the use of covariates. There is a wide range of possible models. The current volume contains functions for two models: PHM and Weibull Regression. These functions include estimation of parameters with approximate standard deviations, (*PHMBETA*, *WEIBREGBETA*), likelihood values, (*PHMLIKE*, *WEIBREGLIKE*), and residuals, (*PHMRESID[I]*, *WEIBREGRESID*). (The [*I*] indicates more than one residual available.)

Specific formulations of other models are covered by *ASLREG* either directly or by transformation, for example Gamma, Log Gamma and Log Normal Regression. Hence it seems inappropriate to repeat them in this shelf.

Growth Models

It had been assumed that as technology develops, systems and components will improve especially in the context of hardware. The more pragmatic will realize this is not the case. In Reliability, however, there are a number of models for describing growth which assume positive growth for hardware: the Duane Growth Model, ARINC and others. Most of these models should contain a governmental health warning. For some of the models there is more than one estimator for the parameters of the models. For the Duane Growth model it is possible to use either graphical methods or Crow's estimate for Non-Homogeneous Poisson Process, (CROWEST).

Beyond hardware models there are a stream of software models such as Jellinski-Moranda (JVEST) and Littlewood-Verrall (LVEST). These models equally have drawbacks, but the Littlewood-Verrall model has the advantage that improvement is only expected at every failure, not required.

Dependency Analysis

There is a wide range of possible approaches, for example Engineering yields the beta-Factor model and its extensions, whilst Statistics provides a rich medium for analysis of dependency. Currently there is not a single approach regularly used and hence at this stage we have decided not to include any specific methods except the graphical approach to analysis suggested by Walls (*WALLSPLOT*).

Conclusions and Comments

This article has briefly covered what is contained within the volume. It is hoped to release the volume for beta-test later in the year; it will be supplied with limited documentation. If you have comments on the material so far included, or wish to suggest further functions which might be added, please write to me at the address given in the editorial on page 30.

APL Product Guide

Compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL Visual Display Units and Printers
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 04393-385

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	AWL486	1,950	486 based 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (Inc. 1 year on site maintenance.)
APL People	IBM PCs & compatibles	poa	Includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDPM and the full line of Interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	DYALOG APL DOS 386	poa	Dyadic's PC 386 APL interpreter.
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa 1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.

Cocking/Drury	APL*PLUS PC Rel 10	410	STSC's full featured APL for IBM's and compatibles - Version 10 includes the Quad-NA facility to interface to non-APL software, support for Microsoft Windows and mouse devices. The User-command processor has been built in to the interpreter. Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	175 for 5	Closed version of the interpreter for developers, prevents user exposure to APL.
	APL*PLUS PC Developer System	950	Gives rights to distribute an unlimited number of copies of Run-Time application.
	APL*PLUS II System	1200	High powered APL interpreter for the 80386 chip. Price Includes one year's maintenance and free upgrades - volume discounts VERSION 4.0 NOW AVAILABLE - THE ONLY APL THAT LETS YOU BUILD WINDOWS & DOS APPLICATIONS FROM ONE PACKAGE.
	APL*PLUS II Developer System	3200	Unlimited distribution of APL*PLUS II Run-Time applications!
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyalac	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces I-APL/Mac		13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Products		I-APL is the UK agent for ALL Iverson Software products, including: Sharp APL + enhancements to the ISI-APL standard Versions of J (and the associated texts) for PC, Mac, Arc, Atari. <i>Please call for price list and order form. Please note there is a £3 packing charge on all orders.</i>
IBM (APL Products)	APL Version 2 Release 1	poa	Full APL2 System for IBM 370 and 390. Product No. 5688-228. See your IBM Branch Office.
	APL2 Application Env't Ver2 Rel1	poa	Run-time Environment for APL2 Packages (IBM 370 and 390). Product No. 5688-229. See your IBM Branch Office
	APL2 for the RISC System/6000	poa	Product No. 5765-012. See your IBM Branch Office.
	APL2 for the IBM PC (US version)	\$495	Product No. 5799-PGG. Part No. 6242936. PRPQ No. RJ4B11 (In HONE, RJ0411). (from IBM Direct, order by Part Number; from your IBM Branch Office, order by PRPQ Number)
	APL2 for IBM PC (European vers)	poa	Product No. 5604-250. Part No. 38F1753
IBM UK	TryAPL2	free	Free APL2 for educational/demonstration use. Write to APL Products, specifying diskette size desired.
	IBM PC APL2	348	APL2 for the IBM PC. From all IBM dealers, including MicroAPL

Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.
	APLIPC	\$30	For PC
MicroAPL	APL.68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL.68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL.68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Optima	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
Reuters Ltd	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Univare	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS AND PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Queen-data & Qume etc	500	Daisy-wheel printers
Dyalic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
Dyalic	Printers (Various)	poa	Range of APL printers available.
General Software	Mellordata	poa	
Shandell	HDS Monitors	965-1395	A range of APL monitors. Please call for details.
	X-Terminals	1000-3500	View-stations for use with all X-Windows systems

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system, Stamp & Personal accounts (inc. Run offs)
APL-385	APL-385 FSM-385 DRAW-385 DB-385 GEN-385	50(PC),125(mf)	Including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualedl	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRSII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling

	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	IAC/Graf	15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Klits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Impetus Ltd	<i>Impetus</i>	poa	Corporate Modelling and Reporting System.
INFOSTROY	Russificator	poa	Drivers and documentation for use with APL*PLUS/PC system and other STSC software with Cyrillic alphabet (PC).
Interprocess Systems	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
	AOC	poa	APL2 Optimizing Compiler, translates APL2 functions to FORTRAN programs.
(mainframe)	AFM	\$6500-15300	High performance component and keyed file system (VS APL and APL2)
(PC)	AFM	\$175	Single user component and keyed files for APL2/PC.
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	CALLAP	\$4700	For calling non-APL programs (VS APL and APL2)
	WSORG	poa	Full-screen Workspace Organizer for APL2.
Mercia	STATGRAPHICS 5	635	The old favourite.
	STATGRAPHICS PLUS	895	Now running under APL*PLUS II - so you can have as much workspace as you want!
	LOGOL 90	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self Instruction for APL*PLUS/PC
	STATGRAPHICS Rel 5	590	
Reuters Ltd	GLCS	poa	Global risk management for banks
	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	NEWSFLASH	poa	Real-time message exchange
	VIEWPOINT	poa	4GL with interfaces to DB2, ADABAS, and MVS datasets

UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF22750	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
(for APL*PLUS II)	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES (£)	DETAILS
Active Workspace	Consultancy	poa	PC Based APL system design, programming and implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.

Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	250-500	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
David Crossley	Consultancy	poa	Broad experience in many APL environments
Dyado	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Localization of APL software for the Soviet Union software market
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Uniware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS/APL:		301-384-3672, 300/1200/2400/9600 b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr).
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL In Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)		
		16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
IBM (APL Products)	APL2 Keycaps	poa	Keycaps for the PC and PS/2 (USA and UK standard). Product Number SX80-0270.
	APL2 Keyboard Stickers	poa	Product Number SC33-0604.
Reuters Ltd	MVSLINK	poa	Auxiliary processor complex, providing SharpAPL (Unix & MVS) with facilities to access MVS data, and Invoke non-APL MVS software on remote MVS systems.
	SSQL	poa	Auxiliary processor providing SharpAPL (Unix & MVS) with high-performance SQL interface to DB2.
Iverson Software Inc. J		\$24	Documentation & shareware for many popular machines, including PC, MAC, SUN, & ARCHIMEDES.
	J Source	\$90	Documented source code in C for porting to other machines (Shareware).
	Programming In J	\$15	76pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	USA	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
CPC UG APL SIG (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indlv), 1000AS(corp)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL		ACM, 1515 Broadway, New York, NY 10036 USA Tel: (212) 626-0611
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496648.
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Lewis H. Robinson	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattemann	IBM Germany, Wilckensstrasse 1a, D-6900 Heidelberg, Germany. Tel: (49) 6221-404-243
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
BACUS	Joseph de Kerf	Rooienberg 72, B-2570 Duffel, Belgium.
Anthony Camacho		11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036.
Paul Chapman		7 Queen Court, Queen Square, London WC1N 3BA. Tel: 071-278-5268.
Cocking & Drury Ltd.	Ronilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071-436 9481 Fax: 071-436 0524
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0604-760241
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: 90-66-08-87
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-824741

General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS. Tel: 081-864-9537
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20914, USA.
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL Tel: 0538-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0388-527190. Email: clark.i@applelink.apple.com
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-463-APL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenln Lane, Leningrad 191186 USSR. Tel: 812-238-6392 Fax: 812-319-9709
Interprocess Systems Inc.	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131, USA Tel: (404) 992-8400. Fax: (404) 998-4869
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: (416) 925-6096 Fax: (416) 488-7559
Lingo Allegro USA Inc.	Steven J. Halasz	203 North LaSalle St., Suite 2100, Chicago IL 60601 USA Tel: (312) 558-1342 Fax: (312) 348-9603
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096, Fax: 021-359-0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 8LN Tel: 071-922 8866 Fax: 071-928 1006
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0790-263843
M.T.I.C.	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252 874697
Optima Systems Ltd	Paul Grosvenor	15a George St, Croydon, Surrey CR0 1LA Tel: 081 680-5958 Fax: -5856
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20789. Fax: 0276-683427. Mobile: 0831-307548
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
Reuters Ltd	Laurie Howard	Reuter Nederland BV, PO Box 8230, 1005 AE Amsterdam, Netherlands Tel: +31 20 570 8733 Fax: +31 20 570 8758
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4380. Fax: 716-454-5430
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax: 02407-3118
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131
Sydney APLUG	Rob Hodgkinson	GPO Box 1425, Sydney, NSW 2001, Australia
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel: (1) 45-27-20-61. Fax: (1) 45-27-20-71. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692589
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	53 Shenipsit St, Vernon CT 06066, USA. Tel: (203) 872-7806



RENAISSANCE DATA SYSTEMS

Enlightenment Thru Information Processing

Specializing in Books and Software on APL, J
and other Curiosities of merit.

HAVE YOU GIVEN A COPY OF IAPL OR J TO A TEACHER OR STUDENT?

Name: _____ Date: _____

Street: _____

City: _____ State: _____ Postal Code: _____

Country: _____ Telephone: _____

For a copy of our most recent catalog,
please send a self-addressed, legal-
sized envelope (stamped if from U.S.)
or mail this form to:

Renaissance Data Systems
Park West Finance Station
P. O. Box 20023
New York, New York 10025-1510

Learn APL

APL Shareware/Demos and Related Publications

APL and Mathematics

APL AS A TOOL OF THOUGHT Proceedings

Other Important Sources of APL Information.

APL History

APL References and Techniques

Special Subjects in APL

APL Interpreters and Software

J=: A Powerful Dialect of APL

Other Software for Use with APL

To Russia with Love and for APL92

Other Curiosities of Merit

**I-APL: An international voluntary project of individuals and
companies seeking to share their love of APL.**

Iverson's An Introduction to J

reviewed by Keith Smillie

An Introduction to J is one of several monographs written by Kenneth Iverson on the J language. Its content and structure might best be described by giving the Table of Contents:

Introduction	Defined Adverbs
Mnemonics	Word Formation
Ambivalence	Names
Verbs & Adverbs	Explicit Definition
Punctuation	Tacit Equivalents
Forks	Rank
Programs	Gerund & Agenda
Bond Conjunction	Recursion
Atop Conjunction	Iteration
Vocabulary	Trains
Housekeeping	Permutations
Power & Inverse	Linear Functions
Reading & Writing	Obverse & Under
Format	Identity Functions
Partitions	Conclusion

... and the first two and last paragraphs of the:

Introduction

"J is a general-purpose programming language available as shareware on a wide variety of computers. Although it has a simple structure, is treated completely in a twenty-page manual, and is readily learned by anyone familiar with mathematical notions and notation, its distinctive features may make it difficult for anyone familiar with more conventional programming languages.

This book is designed to introduce J in a manner that makes it easily accessible to programmers, by emphasizing those aspects that distinguish it from other languages.

The following lessons are records of actual J sessions, accompanied by commentary that should be read only after the studying the corresponding session (and perhaps experimenting with variations on the computer). The lessons should be studied with a J system at hand."

An Introduction to J consists then of thirty one-page lessons of worked examples and commentary which cover all of the main features of the language. The

lessons are followed by fifteen pages of exercises which reinforce and extend the concepts introduced in the lessons. Some of the lessons and exercises provide substantial examples. For example, Lesson 13, Reading and Writing, concludes with a sequence of five verbs for barcharts which are used again in the exercises for Lesson 14 in defining verbs for plotting. The exercises for Lesson 13 contain verbs for generating prime numbers and calculating perimeters, areas and volumes and may recall to many readers the author's much earlier APL functions for these problems. Another lesson of interest is Lesson 22 which introduces the conjunctions gerund and agenda and shows their relation to the case statement of conventional languages.

An Introduction to J has the same attractive format as other ISI publications. (One very minor criticism is that the customary Language Summary has been omitted from the outside back cover.) The statement in the Introduction that "This book is designed to introduce J in a manner that makes it easily accessible to programmers ..." should not deter others from reading it. It may be recommended to both the beginner and the experienced programmer as an excellent introduction to the language and also the experienced user of J as a satisfying and stimulating review.

Iverson, Kenneth E., 1992. *An Introduction to J*.
Iverson Software Inc., 33 Major Street, Toronto M5S 2K9, Canada. 44 pp.
\$12.00 + 7% GST (Canada) + Shipping (\$10Canada and US, \$20 elsewhere).

Available in the UK from I-APL Ltd ... see Product Guide entry.

Keith Smillie

Department of Computing Science
University of Alberta
Edmonton, Alberta T6G 2H1

Dyalog APL/W

reviewed by Duncan Pearson

Introduction

This is a version of Dyalog APL that runs under Microsoft Windows 3 in Enhanced Mode and provides a programming interface to the facilities that Windows provides for the user-interface. The language is the same as in the DOS product; it is a true 32-bit application, although using a different C compiler. It is currently only available for machines with an 80387 maths co-processor or an 80486 DX main processor, though Dyadic say that they will release a version for the 80386 by the time that this is published. The review will not cover aspects of the product which are common to both this and the DOS product, reviewed in Vol.7 No.1. Nor will it cover such perennial topics as benchmarks, which are rather meaningless under Windows anyway. The section on the programming interface is more of a worked example as this seems to be the best way to get a flavour of the thing.

Installation

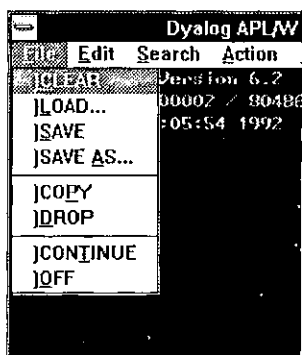
Installing the system is easy provided that you are happy with the standard directory that they specify. All that is necessary is to type install at the A:> prompt, change the disks when asked, and then in Windows create a program item for the DYALOG interpreter program. If however a different directory, or more likely a different drive, is used then the user needs to change some entries in the WIN.INI file.

The keyboard mapping and general I/O tables remain as indecipherable as ever and editing them is as prone to frustration. It seems to be an iterative process whereby all possible key conflicts appear as short messages when trying to load APL explaining why it will not be possible this time. This would not be such a problem if the standard mapping provided with the product was usable. However while Dyadic insist on using the standard movement keys for their own purposes, and use screen colours that hurt the eyes, it will be necessary for we simple souls to brave the intricacies of the .DIN and .DOT files.

General Information

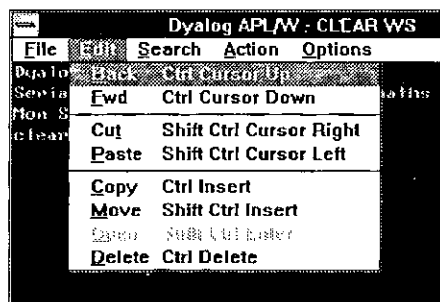
You start APL by clicking on the Dyalog icon a window appears with the normal few words in the top left hand corner and a menu bar along the top. Hanging off

this are most of the standard things that are available as session commands in most APL interpreters, such as)LOAD,)SAVE,)COPY:



Dyalog have adopted the approach of trying to ease their users into the Windows paradigm. The file menu from the session has all the standard Windows file functions but gives them all the traditional APL system command names. Moreover if a function is selected from the file menu the corresponding system command response appears in the session. This seems unnecessary. The functions on the menu do standard Windows things so they should have standard Windows names. It won't take an experienced APLer any time to realise that 'New'

means)CLEAR but it might take a non-APLer with Windows experience rather longer to find out that)CLEAR means 'New'. There are number of other functions that in previous versions of Dyalog APL have been available via control keys:



These are mainly to do with the editing, search and replace facilities of the session and of the function editor. It is here that the value of the labelling approach in the keyboard assignment table comes into its own. Whatever keys you have assigned to a particular session function will appear in the menu entry of that function as a keyboard short-cut.

Given the availability of the menu system and the dialog box facility of Windows one would have thought this a fine opportunity to get rid of such system functions as)FNS,)VARS and)ED. The last thing with which I want to clutter up my session is a long list of all the functions in my workspace. It would be much better to choose a function to edit from a list in the same way that I might choose a workspace to load. They have done the one, why could they not have done the other?

The editor and trace facility are the same as those in the DOS product but the trace facility becomes all the more useful when debugging a Windows user interface as will become apparent in the example later. Likewise the ability to watch a variable, i.e. have a continuously updated window in which the contents

of a variable are displayed, is very useful. In addition a small control window can appear whenever a function is being traced with buttons for all the trace functions such as executing the current line and stepping backward and forward through the function.

One thing that is lacking is support for the standard Windows clipboard. It would be useful to me while writing this review to be able to pull the mouse over a fragment of text in a session or edit window and copy it to the clipboard, ready to paste it into the word processor of my choice. It is a pity also that Dyadic have not followed the example of STSC and MicroAPL in putting the APL manuals onto the help system. In fact there is no on-line help in the APL session at all. Still, with a price tag of £995, the only copy-protection that they seem to have is the necessity of having the manuals to use the system. The two manuals themselves are small, neat and well organised but suffer from poor presentation of the material, when compared with the manuals from many other suppliers of Windows products. The typeface is difficult to read and there is insufficient use of space within the page for clarity. However the chapters in the User Manual on the new Windows interface and on the use of dynamic data exchange are clear and well illustrated.

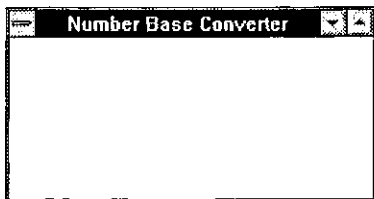
Programming for Windows

The presentation of data and options to the user is done through a number of objects managed by the Windows system. The most basic of these is just a normal window. Within that window can be a number of other objects through which the user can communicate with the APL system. These can be, among others, push buttons, radio buttons, scroll bars and edit fields. For backward compatibility with Dyalog APL for DOS they have included a special SM object which displays the contents of `□SM`. In order for the user to communicate with the system he presses a mouse button while the mouse pointer is over one of the objects of the system or presses a key while one of the objects has the input focus (cursor in or on it). This generates an event, which is really a message that Windows sends to the system to inform it of what has happened. It is then up to the system to react to this message appropriately.

The means of managing the user-interface is sufficiently different from normal practice that the best way to illustrate it will be through a small example. There is a good one in the User Manual which is a temperature converter but perhaps a more appropriate example might be a number base converter, as this is something which every young hacker needs and to which APL is rather well suited.

First we shall create the base window for the application:

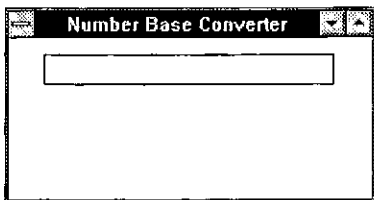
```
'CONV'⌈WC'FORM'('CAPTION' 'Number Base Converter')
```



This puts a window on the screen with all the standard attributes like a border, some sizing icons at the top right, a system menu icon in the top left and title bar along the top with the title we gave. 'FORM' is the basic window type and anything we don't specify, like size and position, will be given a default. 'CONV' is the name of the window and is an object in the APL workspace of class 9. This means that if we `)ERASE CONV` then the window will disappear.

What we need in this window are a field for entering numbers and a button for each of the number bases that we are willing to convert to and from.

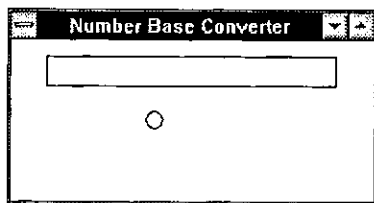
```
'CONV.NUMBER'⌈WC'EDIT'('POSN' 10 10)('SIZE' 20 80)
```



The name 'CONV.NUMBER' marks the NUMBER field as the child of the CONV object. NUMBER does not exist in the workspace and can only be seen by the GUI system functions. An EDIT object is by default a single line field and the size and position are expressed as a percentage of the size of the parent object, in this case the base window.

Next we create a button called BIN for the binary number base:

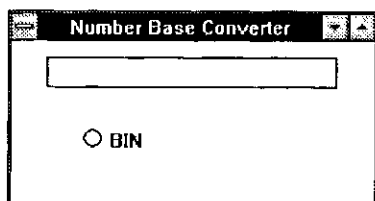
```
'CONV.BIN'[WC'BUTTON']('STYLE' 'RADIO')
```



The STYLE property here makes the button behave like the old waveband selector buttons on what was then called the wireless. This means that if you have more than one of them you can only have one on at a time. Many objects have STYLE and it determines which of a number of variants of the basic type the object is.

Having created it we wish to give it a label and place it in the middle left of the screen so:

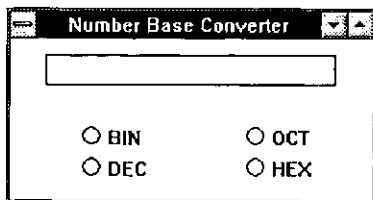
```
'CONV.BIN'[WS]('POSN' 50 20)('CAPTION' 'BIN')
```



The system function [WS] is used to change a property of an object once it has been created.

We now create three other buttons for octal, decimal and hexadecimal.

```
'CONV.OCT'[WC'BUTTON']('STYLE' 'RADIO')('POSN' 50 65)('CAPTION' 'OCT')
'CONV.DEC'[WC'BUTTON']('STYLE' 'RADIO')('POSN' 70 20)('CAPTION' 'DEC')
'CONV.HEX'[WC'BUTTON']('STYLE' 'RADIO')('POSN' 70 65)('CAPTION' 'HEX')
```



Now we have, sitting on the screen, our number base converter, but it doesn't do anything. If you type something in the number field it will just sit there. The radio buttons will behave as radio buttons because Windows handles that aspect of their behaviour. However they are not communicating with APL. For this, two things need to happen. APL needs to 'listen' to the messages that Windows sends it about our converter window, messages such as 'Someone has just pressed the BIN button.', and we need to tell APL in which of those messages we are interested. The first of these is simple; `⌋DQ 'CONV'` will instruct APL to listen to messages about the window CONV. However this gets us nowhere unless we tell APL in which of these messages we are interested. We want to know when the user 'selects' one of the buttons. For APL to know this we must set the EVENT property of each button to react to a 'select' event. For the binary button we do it thus:

```
'CONV.BIN'⌋WS'EVENT' 30 'BASE_CONV' 2
```

30 is the number given by APL to any of the ways in which the user can press the button, `BASE_CONV` is a function that will be called whenever it happens and 2 (the destination number base) is passed as a left argument to `BASE_CONV` when it is called. The right argument to the function is information about what object and event have caused it to be executed, and is made up by the interpreter from the Windows message that it received. This means that you can use the same function for different objects and events but tailor it internally according to how it was called.

We also need to attach a call-back function to the NUMBER field so that you can only enter valid digits for the current base. For this we attach the function to event 22 which is a key press.

```
'CONV.NUMBER'⌋WS'EVENT' 22 'KEY_CHECK' 10
```

If a call-back function returns a 0 result it can in some cases stop the action which caused the event from happening. So our key checking function only needs to test the character against the allowable set and return the result of that test. Movement keys are specified as vectors and normal characters as scalars so we can allow movement keys by checking the rank. We get the information about which key it is from the right argument to the function, and the base against which to check we give as the left argument.

```

    ▽ R←BASE KEY_CHECK ARG;KEY
[1]  A Prevent key press event if the key is not a digit in base BASE
[2]  A it also allows any action keys (returned as vectors rather than scalars)
[3]
[4]  A extract the key from the argument
[5]  KEY←2+ARG
[6]
[7]  A Return 1 if it is valid, otherwise 0. 0 will inhibit the event.
[8]  R←(1-ppKEY)∨(cKEY)∈BASE+'0123456789ABCDEF'
    ▽

```

Having put the base in the EVENT property of the NUMBER field as an argument to the call-back function we can use that to tell the *BASE_CONV* call-back function the base from which it is converting. It can query the events marked for the field and extract from the 22 event the relevant left argument.

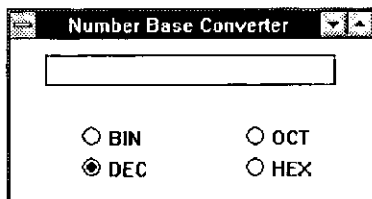
```

    ▽ DEST BASE_CONV ARG;IO;NUM;EVENTS;SOURCE
[1]  A Convert the number in CONV.NUMBER to the base DEST
[2]
[3]  A Index origin 0 is much better for number base conversion
[4]  IO←0
[5]
[6]  A Make sure the button is ON (a select event can turn a button off)
[7]  (≡ARG)IWS'STATE' 1
[8]
[9]  A Get the events logged against CONV.NUMBER
[10] EVENTS←'CONV.NUMBER'IWG'EVENT'
[11]
[12] A Find the key event and extract the number base (left arg to KEY_CHECK)
[13] SOURCE←((1,22←EVENTS)2)≡EVENTS
[14]
[15] A Change the left argument to KEY_CHECK to the new base
[16] 'CONV.NUMBER'IWS'EVENT' 22 'KEY_CHECK'DEST
[17]
[18] A Decode (or encode, I can never remember which) the number in the field
[19] NUM←SOURCE1'0123456789ABCDEF'1'CONV.NUMBER'IWG'TEXT'
[20]
[21] A Convert to the new base and place the new number back in the field
[22] 'CONV.NUMBER'IWG'TEXT'('0123456789ABCDEF'[[(DEST1,NUM)ρDEST]1NUM])
    ▽

```

Finally having set the initial base as 10 we need to turn on the DEC button

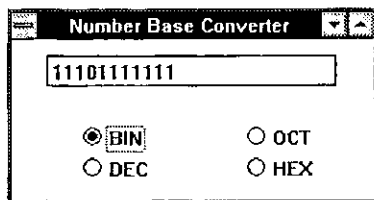
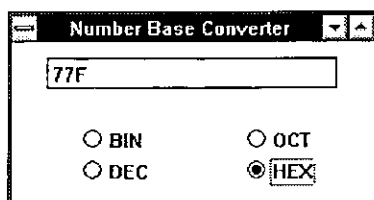
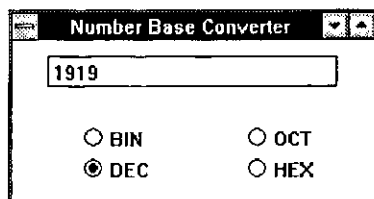
```
'CONV.DEC'IWS'STATE' 1
```



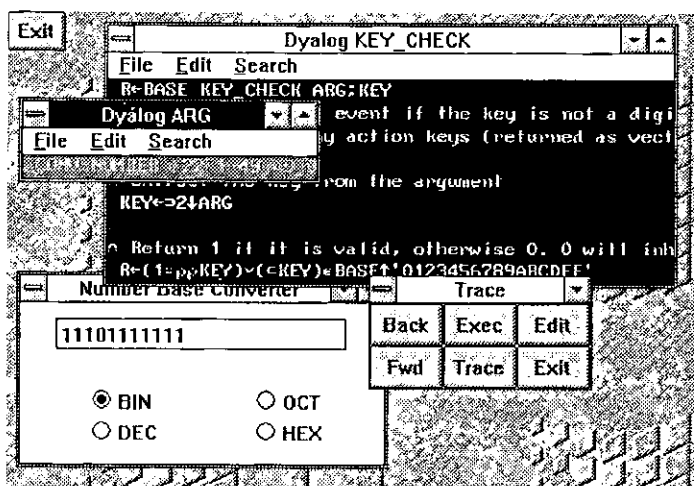
A final

```
□DQ 'CONV'
```

sets APL listening to events from *CONV* which will prime our call-back functions to react to the 'select' and 'key press' events. We enter a number in the field and press a few buttons to see what happens to it.



Now if something, as usual, goes wrong and the system does not work as planned, we can use the Dyalog tracer to step through the functions. When we execute `□DQ 'CONV'` we instead press the trace key (Ctrl Enter) and control passes to Windows as before. However whenever a call-back function is executed it is instead traced. A trace window pops up with the function definition, some buttons with BACK, FWD etc. appear and we can step through the function as we please, watching any interesting variables change in their own windows if we so wish.



General Observations

I find the Windows interface easier to program than any other I have yet used. This is in stark contrast to either of the other methods of programming for Windows that I have seen, namely C and APL*Plus II v4, having said which programming in +IIv4 is effectively interactive C. Here, however, we have something completely different.

In the same way that APL is a language that removes the programmer from the intricacies of the machine and gives him a tool ideally suited to solving numerical problems, so Dyalog APL removes us from the intricacies of the interface but gives us a powerful and simple tool with which to manage it. There are drawbacks to this approach. C and +IIv4 give you access to the full range of windows functions. If you can do it in Windows you can do it in either of these. However in the area of text-based systems I think that Dyalog APL provides everything that I might possibly want to use. Unfortunately it is not possible to comment on the graphics facilities as the current version does not have graphics which we are assured will be in a later release. It is in my mind essential for a serious GUI programming tool to provide graphics and the nature of their implementation seems to be the only question hanging over an otherwise excellent product.

A better comparison would be between Dyalog APL/W and Visual BASIC. VB is the only other programming language that I have heard makes Windows programming easy. It would be interesting to see a VB listing for the example

above. Confirmed APL bigots would perhaps be surprised by how similar they might be.

There is no doubt that an application written in Dyalog APL/W will be slower than the corresponding application written in C. This could be a problem if your users are running Windows on slow 386 machines as the sluggishness of response might seriously affect their acceptance of your systems. On a 486 machine however the response is crisp. A system written in C might be crisper but it is doubtful that your users would notice the difference unless you were doing something pretty complicated with the interface. With the price of fast new machines plummeting, speed will not be a problem unless you have a large installed user-base and a tight budget.

Future Portability

In designing the interface with Windows Dyadic abstracted what was common to the leading GUIs in DOS and UNIX and provided an interface to only those facilities. They state in the manual that it is their intention to implement the same interface to Motif from Dyalog APL/X. This is good news for those of us who live in an uncertain world with the threat (or should that be promise) of UNIX looming a couple of years ahead. If what Dyadic say is true a system written in Dyalog APL/W, provided it does not use any of the auxiliary processors, should port directly to Dyalog APL/X. This would be quite an advantage for anyone who found himself in that unfortunate position, especially when compared with the trouble of taking all the Windows C calls in his +Ilv4 system and replacing them with the appropriate X calls when he ports to +II for UNIX. I'm not saying that it couldn't be done but I'd rather not be the one to do it. Come to that, I wouldn't want to write the +Ilv4 system in the first place.

Postscript

We have found a couple of esoteric bugs in the system which do not affect the running of the code, and the DDE interface does not work with the Oracle DDE server that we run, but otherwise it seems a reliable product. There seems to be a little problem which occasionally causes the Windows session to flash in bright colours when you start APL but this goes away when you shut it down and does not seem to repeat often. Dyadic will produce a maintenance release soon which will fix many of the initial teething problems and also provide a runtime packager. Runtime copies will be free.

RECENT MEETINGS

APL92 AT ST PETERSBURG

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

In this Vector, it is given over entirely to the APL92 Conference at St Petersburg; thanks to Anthony Camacho, Ray Cannon and John Searle for their detailed notes on the sessions and 'events'. Thanks also to (in no particular order):

Anthony Camacho
Phil Benkhard
Ray Cannon
Dieter Lattermann
John Searle

for a superb set of conference photographs. As usual, we have more pictures than there is space to put them in!

APL 92 - Reports on the Conference

notes by Anthony Camacho

A Miracle that it Should have Happened at All?

We reproduce here the Welcome! from the conference final program:



Lynne Shaw and Andrei Kondrashev in the Conference Office

The participants of APL92 in St Petersburg are witnessing a miracle. Two years ago, during APL90 in Copenhagen, when the first aspirations of an APL conference in the Soviet Union were in the air, nobody could quite believe it would be possible in two years, especially not the two SovAPL delegates. They knew that the bureaucracy was unbeatable, travelling from and to the country was next to impossible and international communications were nonexistent or poor. A famous slogan says that the impossible will be done while you wait, miracles will take some more time. Now we know it takes almost two years. On the other hand, we can say that the doubters were right, the APL conference did not take place in the Soviet Union.

We said already in the beginning that APL92 will be much different from any other APL conference. Everybody who has been working for this conference knows already how different it is. It was felt already in the early planning phase. Budgeting an enterprise like this in a country where nobody can predict the economic situation in the next month is a crazy job. The solution was found in separating the hard currency operations from internal operations where the stability is found in established organisations and structures. We wanted to make a truly international conference, but it requires advanced, fast and reliable communications. Someone suggested carrier pigeons. Fortunately, just as the need arose, electronic mail networks reached Moscow and Leningrad. Finally the feared communication problems were not too bad at all. We learned the tricks how to best use telephones, fax and email, and how to send a letter from Moscow to USA in four days for less than one rouble.

This conference was made possible because so many APL friends in so many countries saw the opportunity to strengthen the world-wide

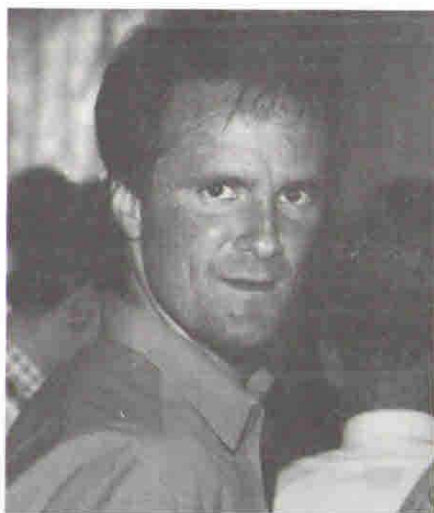


Grigory Amirdjanov (Treas) and Andrei Kondrashev (Chairman)

APL community and took it as a personal challenge. The authors of papers, referees, the international program committee, ACM and SigAPL officers and many other volunteers showed quite exceptional tenacity, flexibility, and understanding in order to overcome problems and to make cooperation work. We can simply say that we have had the most international, most experienced team of the APL history to work on this conference. But ultimately the participants will make the conference. We hope you will make it a unique, unforgettable experience. Remember that we learn most from different friends. You will find lots of different friends in St Petersburg.

Small is Beautiful

Around two hundred delegates seems to be a good size for a successful conference especially if most of them are able to stay in a single building and eat together. Far more interaction is possible and one gets to hear about other people's doings. Although there was only one paper using J and the offer from Bob Bernecky to conduct the proposed J workshop arrived too late for the organisers to take it up, there was a lot of talk about J. Arthur Whitney (the originator of the rank operator and author of 'A') was telling people about 'K', his suggested successor to J. I had one breakfast with Arthur sitting next to me entering expressions on a pad, showing the results and explaining how the interpreter could handle the dialect — it was Special K for breakfast that day even though Kelloggs wasn't on the menu! Later he decided that as what he proposed was not really a successor to J the name 'K' was inappropriate. He may call it 'R' for reduced instruction set but Paul Chapman thinks R is for Arthur. Arthur agreed to try to write a fullish description and send it to Vector; a treat worth joining the British APL Association for in my opinion.



Arthur 'K' Whitney

Some Difficulties were Marvellously Overcome

The food! We were told it was all imported from Finland. Amazingly we ate well; no-one went hungry. We had fresh milk and cornflakes at breakfast if we wanted. The buffets at all the special events had almost too much luxury food. By the end of the conference we would have been glad of some plain bread and butter, but no-one could complain at caviar, sliced beef, smoked ham, smoked salmon, mixed vegetables rolled into bite sized balls, cheeses, savoury pastries, cherries, strawberries and bananas.

The drink too was good. Champagne at every buffet with white wine and sometimes beer and sometimes red wine, plus pepsi and carbonated water for the non-alcoholics.

Tea, coffee and fresh milk for breakfast; beer and coca-cola for lunch and always boiled water to clean one's palate.

Some Easy Tricks were Missed

For the first day or so there were problems finding one's way about; there were no signs or they were inadequate. Announcements were not always made when a paper or workshop was cancelled because the author had not been able to come. Room changes were not announced and this caused some papers to begin very late — in fact the timing of the conference took its cue from the unexplained twenty-seven minute delay in starting. I suspect this is a consequence of the way Russia has run for years: motto — when things go wrong don't make waves or it may get worse!

There were one or two Disasters

For the first two days the water heating was out of action; the hot taps produced water the colour of black coffee. The relief when it did work was considerable and the colour changed to pale Chinese tea so we neither smelled nor looked dirtier at the end of the conference than we were at the beginning.

I suppose the worst disasters were the papers that were not given and the workshops that were not held because the speakers did not come. This was a surprise to me because as the Russian visa procedures are so inflexible the organisers knew a considerable time beforehand just who would be there. Usually when one knows of a speaker's absence one gets the speaker to nominate someone who is coming to do the presentation.

The Best Tours and Entertainment Yet

More trouble was taken with entertainment than ever before. The five evenings were completely used up with a Welcome reception, an Exhibitors Reception, a trip to Kronstadt, a picnic at Lomonosov and the banquet. Here are a few notes on the events:

The Welcome Reception

This was held in Shuvalov's palace. The rooms were extremely ornate; we were fed with a champagne and caviar buffet and treated to speeches from various people, with interpreters when they spoke only Russian. This was a good chance to get to meet people and talk. As I had numerous commissions to execute for I-APL and ASL and had been asked to sound out people about the British APL Association's proposal to run APL 94 at York, I welcomed this and the other opportunities.



Elena Korovina & Sylvia Camacho



Nikolai Puntikov
(Social Program and Tours)

The Exhibitors' Reception

The exhibitors' reception was held mainly in the large corridor outside the exhibition, the souvenir shop and the posters.

The Trip to Kronstadt

The Kronstadt cruise was a highlight of the conference. We were taken by bus to board three large hydrofoil launches and driven at high speed across the sea to Kronstadt which has been a closed city for seventy years. Even citizens of the USSR could not go there unless they worked there. Kronstadt is the naval base that used to guard the approaches to St Petersburg from the Baltic. It is an island now connected to the mainland by two huge long causeways (still under construction) with great bridges to allow ships access to St Petersburg. On disembarkation we were entertained by a naval band. Then we walked through a small park and along a road to the hall of the officers' mess (which

the program tells us is in Menshikov's Palace) where the buffet supper was laid out for us. The band came with us and re-assembled on the stage under a sign which read "OFFICER — PROFESSIONAL HEROES", where it played to us in between the speeches. The trip back was by coach along the causeway and overland. It was a long trip and we noticed for the first time that we had a police escort at the head of the cavalcade. Once off the causeway we drove through some of the northwestern suburbs of St Petersburg — vast dreary blocks of flats which seemed endless. One of our hosts told us "The centre may now be St Petersburg again but this is still Leningrad".

The Picnic at Lomonosov

After Kronstadt had been so good we didn't expect too much of the "Picnic". The coaches were led by a Police escort which confidently went through red lights and drove in the centre or even the wrong side of the road when the right side was not free. Even when the police had a green light, it was often red by the time the eighth coach reached it, but the cavalcade was not to be split and carried on undaunted. Russian drivers must be prepared for this sort of thing as there were no accidents or sounding of horns, even when the last coach crossed a red light with the police car quite out of sight. The journey was a long one as Lomonosov is on the other side of St Petersburg to the South East.

As we milled around the coaches on the scrubby parking lot a band struck up and our hosts told us to follow the band. To our amazement, out of a wood marched the band

dressed in eighteenth century costumes with wigs and braid on their uniforms and followed a zigzag path through the wood to one side of the lake where it paused for a couple of numbers to allow the 250-odd picnickers to catch them up and regroup. Then off we went again round the lake in front of the Chinese palace and through a pergola to find trestle tables with champagne for us all.



After the champagne we were off again to the courtyard in front of the house where the band left us. There a flute, violin and cello played some eighteenth century music and a man in eighteenth century costume declaimed some poetry by the Russian poet and chemist whose house it had been. We were encouraged to walk round the back of the house where we found a Russian folk dance group; a group of dancers in bright costumes and a group of musicians including two piano-accordions and a bass balalaika. These gave a spirited performance with much whooping and stamping.

Then we were encouraged to return to the building by the car park where the buffet supper was served. Upstairs and in other rooms there were souvenirs on sale. After supper we could go on a tour of the Chinese palace; it was very richly



... much Whooping & Stamping

decorated and the marquetry floors were quite outstanding — we had to wear felt pads tied on under our shoes so as not to mark them. Then we had another escorted ride back to the Education Centre where we arrived at 11.30 pm. Of course at this time of night in July St Petersburg is still light. Except when it was very dull or raining I could read at any time, even indoors at midnight.

The Banquet and Floor Show

After another exciting coach ride we reached the Pribaltiskaya Hotel, built (to a very high standard) by the Swedes — or so I was told. After a glass of champagne we were let into a huge ballroom with decoration that I now think of as 'standard international luxury hotel', found a table and sat down to eat the first course which was already laid out. The food was even more luxurious than usual. As well as the usual wines there was red wine and vodka and the first course was followed by fillet steak with duchesse potatoes and fruit and ice cream.



Erkki Juvonen

Erkki began the announcements with a sequence of thanks to the many people who helped to make the conference such a success, telling us that a small miracle has occurred in St Petersburg this week. Everyone thanked got a cheer or a round of applause and the wives who had helped also got a bouquet. Erkki himself and the hardest workers of all had standing ovations.

Larry Moore (chairman of APL 93) gave a short slide show of the beauties of Toronto and we were all urged to attend and to find ways to make it possible for our Russian friends to come too. Then we had a floor show! Talk about over the top; there were pop singers eating their microphones, a contortionist (not quite entirely naked), a chorus line of dancers with one leg

sheathed in gold lamé and the other not, and best of all a juggler. Apart from the juggler this was not at all to my taste but in this I defer to other people's judgement. If we had had this in New York I would not have been surprised but in St Petersburg it was amazing.

The Tours

I only went on one of the tours. If they were all as good the accompanying persons must have had a marvellous time. It was billed as St Petersburg from above — to climb the highest buildings for the sake of the view from the top. There was both a guide and an interpreter who didn't always agree with the slant on things that the guide offered. They kept asking how many of us were from the USA and as it happened none of the

party was. I felt the emphasis on the spiritual side of life was put in to please transatlantic tastes and that the guide was too inflexible to change it on the fly.

We went up the Petropavlovskaya cathedral tower, and apart from the views were allowed to inspect the clock which can play tunes on an extensive carillon. One of the drums is programmable but is not at present in use: the clock is being refurbished by some Belgians, we were told. The tower is not usually open to tourists or Russians. We were told we were the first tourists allowed up. Then we went up the Admiralty tower, which is similarly restricted. We had to wait to be admitted and for naval personnel to lead us up and unlock as we went. The view from the top is excellent, and the way up dusty and ill-lit enough to believe that we really were the first westerners allowed in.



The Winter Palace

The Program

Highlights were the papers by Langlet, Tollet with Harrison and Spunde. As Dieter Latterman remarked in introducing Christoph Popsiech "Christoph has used APL2 of course: otherwise he wouldn't be allowed to talk here". Maybe there was not too much APL2, but there definitely was too little J and Sharp and other implementations. I certainly felt that Dieter had hit the nail on the head even though he immediately corrected what he had said.

In general the program allowed one to attend more of the sessions one wanted because there was less going on elsewhere, but in spite of this I missed Bernecky's last wind tunnel, Curtis Jones on APL plotting, Johann Mitlöhner on porting via ASCII and all but one of the workshops.

The Software Library

R G Brown ran the software library. When we arrived there were sets of disks and a 20 page descriptive manual. By the Wednesday there were an additional seven megabytes of contributions, including some updates to his public domain libraries from Dick Holt of HRH systems, the workspaces of APL Traceguide (this was a poster of APL*PLUS/PC utilities from Zalman Rubinstein of Israel), J version 5.0 from ISI, Peter Lewis' spreadsheet, Adrian Smith's Adobe type 1 APL font with samples (almost indistinguishable from the 2741 font for 2741 characters), the Ctalk material, the material from bulletin boards on J edited to work in the latest version of J from Alain Delmotte and the 1991 index of APL articles by Rosemary Snow. It was the largest software library ever and the descriptive manual a welcome innovation. I look forward to receiving my copies of the extra seven megabytes.



Bob Brown



The End of the (Chorus) Line

Conference Diary

Opening Plenary

This was billed to start at 10.00 — at 10.27 the conference Chairman, Andrei Kondrashev went to the podium and announced the conference open. No explanation of the delay was offered.

Erkki Juvonen, program co-chairman, made the initial announcements. Sessions were to be 60 minutes long to allow for language difficulties. Workshops were to be in whatever format the convenor chose. He suggested that each of us aims to contact five names new to APL as the first step in spreading the word. Then he welcomed us to the conference and repeated the welcome in Russian, French, German and what I guess was Finnish.

Viktor Ivannikov, program chairman, also welcomed us. He wants to keep APL linked to the main stream of computer science. He suggested that we copy the twin town idea in computer science. He told us that APL was growing in Russia — from this year there will be APL courses in Moscow State University (to be taught by Professor Popselov and Andrei Kondrashev) and as the leading institution it is likely that others will



Viktor Ivannikov

follow suit. Russia will introduce a new copyright law based on the European tradition so the current practice of using pirated or public domain software can end and vendors can find a market in Russia. He ended by telling us that there is a real opportunity to make APL popular in Russia and that we should talk of this as the first ACM conference in Russia.

Erkki then introduced Lynne Shaw as the representative of the ACM to welcome us to this ACM conference, which she did and then Erkki introduced Josef de Georgi, the manager of the IBM office in St Petersburg.

Josef de Georgi told us he was an expatriate Austrian with the difficult task of introducing IBM into St Petersburg. The problem is that nobody knows how to get from where the parts of the old USSR now are to where they want to be. Mayor of St Petersburg Sobchuk has many grand projects: a highway to Helsinki, a Free Customs Port, a highway to Moscow. Such projects will take ten years and meanwhile we struggle and our difficulties strengthen and are exploited by the enemies of perestroika. Matters are made worse by the ethnic problems — worst at present in Yugoslavia but likely to break up Czechoslovakia and who knows what they will do in the USSR? Nobody knows how to stabilise the rouble, or how to get farmers to produce enough to make Russia self-sufficient in food.

IBM opened an office in Moscow in 1974 but the trade embargo on computers following the Russian invasion of Afghanistan put them almost back to square one again. The embargo was only lifted in 1989. Now IBM Russia has 85 employees; in St Petersburg there are two foreigners and three locals. In spite of the problems IBM believes in the potential of the CIS; it supports APL2 as the strategic interactive language. Finally Josef de Georgi urged us to read the welcome address in the final program.

Gérard Langlet: Towards the Ultimate APL-TOE

This was not billed as a plenary session but as there were no workshops or competing papers in effect that was what it was. APL-TOE stands for the APL theory of everything. M. Langlet believes that one can discover laws of nature without experimentation.



Gérard Langlet: a very long Difference Scan

The argument runs something like this: things must, in the last resort, be made of simple components. At the lowest level of all, models made of simple components are likely to match the real world because both are fundamentally simple. Gérard Langlet has spent great efforts over the years on algorithmic compression and, as a result, has found many algorithms in their most compressed form are made up of difference scans.

He then dazzled us with fascinating simple matrices. Some were self inverse, some, when inverted, changed the signs of a symmetrical group of the elements. He showed us how to generate a Sierpinski triangle. He showed us the pariton, a matrix generated by repeated difference scans which had one of the properties of a hologram in that many different parts of it contain enough information to reconstruct the whole. He found analogies to Pauli's exclusion principle, ways of magnifying or shrinking a pariton matrix which preserve its essential characteristics. A fractal is a matrix operator: the smallest fractal is a single bit.

M. Langlet asserted that his next example was not artificial but NATURAL intelligence. Newton found gravitational attraction but physicists could not easily reconcile the gravitational attraction with the repulsive forces that apply at the smallest scales. The difference scan on vectors does something very like this. By the time M. Langlet had reached helicons and robys I had lost the ability to take notes as well as follow the argument. But I am convinced that there is something in this. If it is possible at all to build mathematical models of the physical world, we assume that the equations are simple enough for us to discover. Surely this is as arrogant or even more arrogant than M. Langlet's proposal that we should work the other way and look for instances of the simplest suitable mathematical models in the real world.

If the model is simple and elegant enough then, he hopes, there will be something physical that corresponds at some scale — at some level of generalisation. That is all we are doing when we say two apples plus two apples makes four apples — it is true even if no two apples are alike.

This paper was the best of the conference for me. After this there was only one way for it to go — down. But, to compensate, in every question time that M. Langlet attended after this, he asked some question such as whether the speaker had tried simplifying his expressions by using difference scans. It became the running joke of the conference.

There is a recommended text for the Theory of Everything — by John D Barrow of the University of Sussex ISBN 0-19-853928-2.

Gustav Tollet and Peter Harrison: A Bézier Curve Builder

The Drawmaster product from IBM contains about 20,000 lines of APL code. It is all the work of three people, Gustav Tollet in Helsinki, Tom Springel in White Plains and Peter Harrison in Hursley. It now serves several thousand users.



Gustav Tollet

It was begun in 1984 when it was common to represent curves by polylines — sequences of short straight lines which were shorter where the curve changed direction rapidly. M. Bézier worked for Regie Renault and invented his representation of a curved surface to make it easier to describe the curves of a car body. The two-dimensional version is a simplification. The formula is a parametric cubic and is transformable, which is to say it can be reflected, rotated, transposed and displaced by the same procedures as

polylines. B-splines and conics are not transformable in this way. Bézier curves, however, cannot exactly represent conic sections and the problem of finding the point of intersection of two Bézier curves has not yet found a general solution. The Bézier curve is much more economical in space; typically a 95-segment polyline might be a 21-segment Bézier. And a straight line is just a limit of a Bézier, so the whole of any curve can be approximated by Béziers. Such a reduction in the storage needed for a drawing, which might be a megabyte or more, is a prize well worth working for.

To convert a polyline to Bézier one must first identify the end points and then match the curves. The IBM team had a great deal of help from Pat Bierne of Corel Systems (one of the authors of CorelDRAW!) who challenged them with polylines of 100 points forming a circle (represent it as four symmetrical quadrant Béziers) and a rectangle with rounded corners (represent it as four straight lines and four corner quadrants).

Gustav Tollet explained some of the techniques used. They were fascinating but your reporter couldn't write fast enough. You can buy the proceedings to read the paper. Vector hopes to scoop the world by printing the complete code (which is Gustav Tollet's copyright) — but we don't yet have a date!

And the advantages? Not only is it very economical of space but the conversion is really fast and completely reliable. In answer to Ray Polivka's question Peter

admitted that "Yes Pat Bierne was impressed". By the way the words "of the sum of squares of deviations (called the error value below)." were omitted from the end of the last paragraph on page 261 of the proceedings.

Tien Chi Chen: Globally Convergent Polynomial Iterative Zero Finding Using APL

Your reporter did not understand the mathematics; the essence of this paper was that the new iterative method of finding solutions (zeros) is much faster than Newton's or Laguerre's methods and Mr Chen has found how to avoid successive iterations oscillating between two or more values.

The presentation plotted values graphically and showed how the iterations approached the solutions even in hard cases such as several symmetrical clusters of points, symmetrically arranged.



Tien Chi Chen

Martin Gfeller: Walks Into the APL Design Space

Martin began with a survey of where APL is today. Iverson's book is 30 years old; there are several incompatible dialects of APL; there is stagnation rather than commercial growth; APL is not accepted as part of computer science; little university time is spent on teaching or using APL; the APL community hasn't chosen a niche or decided to be a universal language. He then showed us some diagrams categorising the market and applications. He suggested that APL could survive only if it adapted itself to the future; then he listed the strengths of APL and considered how to add to the paradigm, which is based on functions and arrays, without inconsistency. The suggestion is that APL could include an Object Oriented paradigm without damaging its fundamental characteristics. Since attending the presentation I have read the paper and feel that Martin's arguments deserve more careful attention than I gave them at his presentation. Too many people, disappointed at the poor rate of growth in the use of APL, decide that what is wanted is a re-design of the product: nearly all re-designers fail to preserve the consistency of the syntax and elegance of the primitives. On this occasion I did NOT feel that I was listening to a poor salesman re-designing the product as an excuse for his failure.

Igor Pospelov: Russia on the Way from Plan to Market: the Experience of Modelling.

Professor Pospelov told us he has used APL for 15 years. It has the advantage for him that the code contains nothing but the model. He proposed to show us some real economic models of the Russian economy. The demonstration is not specially arranged but is a real run of the current version of the model. Regrettably, in order for the audience to see the large screen projections of the computer output, the lights were turned down and note-taking became impossible. The system is written in APL*PLUS/PC and is in the software exchange as system GRAND. Interested people can try it out for themselves.



Igor Pospelov

Professor Pospelov paid tribute to the work of Andrei Kondrashev, our conference chairman, for creating the system that displayed APL, Cyrillic and Roman alphabets all on the screen at once.

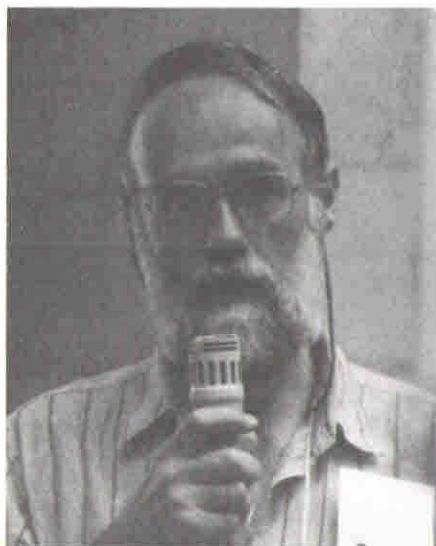
Eke van Batenburg: Porting and Optimising STAR

Mr. van Batenburg began with a short lesson in biochemistry. RNA is the template formed on stretches of the unzipped DNA which is required to be replicated. When the RNA comes off the DNA it does not stay straight but becomes a tangled string and some parts of the string become stuck to other parts. It may even become knotted. These connections of one part of the RNA strand to another are the secondary structural characteristics of the RNA and are important in determining its further behaviour. The program STAR does STructural Analysis of Rna.

The most famous program for this work is by Zuker; it is very computing-intensive because it works out all possible attachment points and makes all the connexions in one go. Runs take a whole night on a VAX. STAR looks for the points which attract each other most strongly and then those which are next strongest and so on. Because the number of cases is much reduced by this procedure it is very fast. Because the likeliest connexions are chosen first, which matches the way the RNA strand gradually gets more and more places stuck, it

sometimes produces a better prediction of the secondary structure which an RNA strand will form. The run which took the VAX a night using Zuker takes 11 seconds on a 386/33.

The program took 9 months plus a further 9 months to produce a nice user interface. For the lessons learned while porting the program to APL68000 (on Atari and Macintosh), APL2 and Dyalog please see the paper.



Eke van Batenburg

There were some delightfully easy things (such as porting the engine to APL68000) and some problems (such as APL2 refusing to share the machine with QEMM or how to get a nice user interface out of API24).

I liked the advice on optimisation:

1. Don't do it.
2. If you are an expert, don't do it yet.
3. Don't do it everywhere: make an execution profile and an "irritation" profile (the first to show where the bulk of time is used; the second to show where time is used that spoils the user interface). Only improve what these profiles show to need it.

R Bernecky and G Osterburg: Compiler Tools in APL

Bob Bernecky was introduced as the Potentate of Snake Island Research. He gave the advantages of compilers as broadening the range of applications where APL was competitive, simplifying the creation of applications, improving reliability and improving performance (especially where code must be iterative).

A compiler can be built of a finite state machine and a table of transitions processing an input string. Such a compiler cannot recognise languages which require backtracking. Bob gave a nice example of the table of transitions for validating input of a real number.

Gert Osterburg of Technoma showed us how the specification of an application went through the stages: input; tokenise; parse; evaluate. From the token

definitions the FSM generator creates the FSM table which converts the input into a token list. From the grammar the parser generator creates the parsing table which converts the token list into a parse string. From the semantic definitions of the primitives the parse string can then be evaluated to produce the result.

Sharp APL\370 contains facilities to incorporate functions written in machine code; they appear as locked functions. The FSM and PARSER functions were produced this way.

J-J Girardot: The CTalk Programming Language: a Strategic Evolution of APL

It irritates APLers to have the supposed weaknesses of APL (those that prevented it from taking over the world) listed again and again. Jean-Jacques began by doing this; how else could one introduce an new language to APLers as an improvement? His view of the weak points is: the line structure of defined functions and branching, the late binding of identifiers hence the ambiguity of some instructions (they depend on what is localised) and the character set.

CTalk is designed to look like C with all the usual control structures, with lexical binding, nested definition of operations and functions as first class objects. Of course it uses the ASCII character set.

CTalk is at present an unfinished experiment. As a benchmark J-J ran a Fibonacci function and whereas APL is 800 times slower than C, CTalk is only 20 times slower. The biggest drawback of the current implementation is that it doesn't cater for arrays of more than one dimension which makes porting APL systems to it non-trivial. CTalk is in the software library.

R G Willhoft: Petri-net Evaluation using APL2

I didn't know what Petri-nets were and was surprised to find that I had used something similar in the 1960s when working on interactions in real-time systems. They were first published in 1962. A Petri-net consists of 'places' and 'transitions'. Each place may be connected to one or more transitions by an arrow; each transition may be connected to one or more places by an arrow. When a place is 'occupied' or switched on then each transition it is connected to is partially or wholly enabled. When every such input to a transition is enabled then it may 'fire'. The firing may be delayed for a minimum time or forced to happen within a maximum time, but in general there are no constraints. When a transition fires the places with arrows to it become 'unoccupied' and those with arrows to them from it become 'occupied' thus perhaps enabling further firing.

This scheme is used for analysing such mechanisms as communication networks, multi-access storage mechanisms in computers and even neural nets. Analysis may reveal the possibility of a deadlocked state where no transition is wholly enabled, or that there are states which are unreachable.

Mr Willhoft showed us the 'coverability tree' which is valid when there are no time constraints and shows the possible states, the 'firing diagram' which adds time constraints and may make some states unreachable and the 'GRID' — a graph of reachable instantaneous descriptions. The functions used to perform these analyses and others are in the paper and the software library. The representation of the Petri-net is explained thus allowing us to devise further analyses if we wish. The system has been used to avoid deadlock and obtain maximum reliable performance from several hardware designs.

Elod Polgar: Europe without Borders: New Dimensions in APL

Dr Polgar showed a nice series of slides with a message of hope that APL has many opportunities to grow in a Europe newly enlarged by the opening of the east. He asked some interesting questions and told a story about an exam where the examinees had three hours to answer the question "Is this a question?". He asked the audience what they thought the shortest answer was and your reporter, who had heard it before, said it was "If this is an answer." and won a prize — a spoon with a telescopic pointer as a handle; it can be extended according to the devilishness of my supping companions.

SigAPL Meeting [Wednesday 14.00-15.00]

Rosemary Snow has produced the 1991 index of APL articles and appealed for people to send her material to be indexed. Bob Bernecky said that the utility of the index on disk would be much improved if properly chosen keywords were included; people won't browse as there is too much material. R G Brown spoke for Quote-Quad. Subscribers should now have the December 1991 issue and the following issue has gone to press. The next issue should get to Jon McGrew, who is production editor, in August and the fall '92 issue in September or October. It is hoped that Quote-Quad will be up to date again by the end of the year or the beginning of next year. After enquiries with ACM SigAPL have discovered that Quote-Quad is not prohibited from taking advertisements and will now do so. The rate will be \$500 (U S Dollars) for a full page or approximately pro-rata, but check with SigAPL before sending money. Quote-Quad is short of good material.

In answer to Anthony Camacho's questions Lynne Shaw told us that the current membership was around 1300 and the circulation of Quote-Quad nearly 2000.

Education Workshop: convenor Professor Howard A Peelle

Professor Peelle asked everyone to introduce themselves with a two minute statement. After this introduction anybody could say what they liked. Per Gjerlov said his daughter (a crystallographer) suggested that the average age of APLers at conferences could be brought down if, in each country, the APL group ran competitions with a ticket for two to the next conference as a prize. Walter Spunde wanted to return ownership of the learning process to the student — I-APL helps to make this possible but is short of graphing tools. Ed Shaw feels we need teaching material in a series of workbooks each on a specific topic.



Oleg Luksha and Rolf Levenbach at the Education Workshop

Eke van Batenburg wanted some marketing guidelines, saying "How do I justify teaching APL?". Anthony Camacho asked people to submit such material for publication in Education Vector. R G Brown offered copies of a course outline he had taught in New York financial institutions for some years. Phil Benkard felt that some people would like to know more about the theory of APL and how it works. Sasha Skomorokhov arrived and said he would like a book of problems and solutions in many languages so that people could see how short and elegant the APL2 solutions were. Someone reported that the annual ACM programming contest did not allow APL entries. Lee Dickey offered Waterloo University's FTP (email file transfer protocol) site.

Professor Peelle summarised:

- 1 Encourage APL organisations to get young people to conferences
- 2 Produce workbooks
- 3 Provide a FULL (with graphics) APL cheaply
- 4 Publish material in Education Vector
- 5 List the universities and colleges where APL is used
- 6 Enter an APL team for the programming olympiad
- 7 Complain to the ACM about the exclusion of APL & enter their contest
- 8 Publish APL solutions to problems
- 9 Send material for circulation by email to Lee Dickey
- 10 Use email more for communication
- 11 Adopt a teacher and help to get APL used in school

Ed Shaw complained that this (two-hour) session had been a talking shop. There were no actions assigned; nobody had undertaken to do anything except Lee Dickey and the value of this meeting would be lost as soon as the people here forgot what was said. Ed is a go-getter and I agree with him; at each conference we begin again the job of considering how to get APL used in education and all we do is consider it. Readers of Vector can prove me wrong by taking up some of the suggestions and acting on them. I wish I had gone to hear the presentations on the Last Wind Tunnel, Jogging with APL and Cannibals and Missionaries which were on during this workshop.

Alexander Kalinin: Information Security in the former Soviet Union

Colonel Kalinin worked for the KGB for 17 years and described it as a monster but its staff were just soldiers doing their jobs. He described the structure of the new federal agency which was linked to the major intelligence gathering department (part of the army). It has three sections: radio monitoring, government signals and cryptography. The cipher equipment is used by both the latter sections and computer analysis is used for special purposes in radio monitoring. After a survey of cryptography Colonel Kalinin told us that Russia has the world lead in cryptography and that private companies cannot compete with the government agencies.

Alexei Skurikhin: Neural Networks in J

Regrettably the majority of the Program Committee took the view that J is not APL and that J papers should not be included in the conference. When a J paper by a Russian was submitted they amended their view to 'J is not APL except when written by a Russian'. I look forward to papers from McIntyreski, Iversonovich, Huisov and McDonnellanov at Toronto. This presentation began with a partial history of neuro-engineering and the reasons for choosing J (its

power and extensions beyond any enhanced APLs made implementation easier). The difference between explicit and tacit definition was explained; all the code uses explicit definition. An illustration in the paper builds the learn verb which incorporates a version of back propagation from twenty-nine subordinate functions. Mr Skurikhin explained how, as J does not have a ready-made menu generator, he used quickC and CScape to handle inputs and link-J to connect with J. The work was done in J version 3, but it is intended to convert to version 5.0 and to keep up to date with later versions as they are issued.

Alexander Skomorokhov: Do Russian Children Like APL2?

Sasha's foils were all 'intentionally blank'. He told us that those that can, do: those that can't, teach and those that can't teach, teach teachers.

He strongly supported the use of an enhanced APL as an exercise of the mind. APL2 is like playing chess but much, much better. He would like to see an experiment where two classes were matched and one taught APL2 and the other not. He suspects that the APL2 class would become cleverer at non-APL2 subjects because the exercise in symbolic manipulation would be valuable in all thinking. Many computer languages can be learned and impart valuable lessons, for example



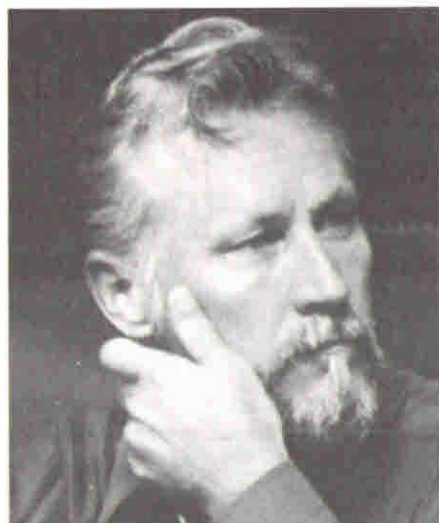
Alexander Skomorokhov

Prolog, but the special quality of APL is that the pupil can look at the function and understand how the computer does it. The pupil can apply the function in new situations. APL kills the mystery surrounding the computer; instead of a magic black box whose results are not to be questioned it becomes a controllable machine for which we can take responsibility.

At the conference there was a daily class in which Sasha and others taught APL2 to a collection of 13 and 14 year-olds from local schools. He said he didn't have to teach them; they taught themselves; they already knew Pascal and were too strong in computer science to need much guidance. He gave examples of the sort of things that they were doing by the end of the week.

He wished it was less expensive to equip a school to teach APL.

Walter Spunde: Challenging the Maths Curriculum



Walter Spunde

After a slide of an ancient woodcut showing Boethius and Pythagoras, Walter suggested that the Abacus held up mathematical development for a long time because it replaced symbolic manipulation by rote operations. (Your reporter was surprised to find his bill in a St Petersburg bookshop added up on an abacus.) It is regrettable that many mathematics faculties look down on calculation. Many mathematical education packages simply fail to teach — they hide the material to be taught from the student and were probably wonderful mathematical educations for the authors.

A teacher has to think through the activity demanded of the student and

how the student is to find a way to learn from that activity. Walter then asked us to try to find a way to show that the sum of the distances from the generators of an ellipse to a point on the ellipse is constant, and then by the aid of a picture of the cone with the ellipse and a sphere resting in the cone made it obvious. He seeks to achieve a better balance between geometric insight, symbolic manipulation and calculation.

His equivalent to the worksheet is a storyboard, on which exposition, problems, hints and suggestions for exploration can be mixed. Examples showed how the weaknesses of poor algorithms can be demonstrated. "In no other language could one afford the luxury of implementing so poor an algorithm to demonstrate a basic idea." Thus he aims to return ownership of the curriculum to the student.

He paid tribute to I-APL without which he might never have made the early steps in this work, but had found moving to APL*PLUS (with the aid of an educational grant from STSC) a marvellously liberating experience. His conclusion was that APL is a very good tool to use in the classroom.

The presentation was a superb example of professional exposition. I urge anyone interested in education to read the paper; it has more commonsense in it about how to teach than I had heard at any APL conference.

Professor Howard A Peelle: Education and the Future of APL

After listing the points in his summary from the education workshop (see above) and an extensive quotation from Tama Traberman's address to APL 85, Professor Peelle listed for us one word for each letter (26) of the Roman alphabet which he felt was appropriate to APL and then one word for most (some of them cannot begin a word) of the (31) letters of the Cyrillic alphabet which he felt was appropriate to APL. The Cyrillic exercise was less successful owing to the limited vocabulary in Professor Peelle's Russian phrasebook. This vapid exercise seemed to take hours — Professor Peelle went on so long that the educational meeting which he had called to follow-up the educational workshop was aborted.

Per Gjerlov: the Song of the Whale



Per Gjerlov

As Per said, this was a marvellous slide show of a holiday he has had since retiring from IBM, on a boat in the Azores with an English team researching a group of whales. It also had a serious purpose. The sounds made by the whales are recorded and the aim is to find out what meaning they contain. It would be nice to be able to identify whales from the sound each produces, for example. The sounds are mainly clicks and these are extracted from the recordings and plotted.

The data has been used several times in a course Per teaches at the laboratory for Applied Mathematical Physics at the Technical University of Denmark. There are other options which mostly use languages with which the students are already familiar, such as Fortran.

The whale option attracted more than half the students this year and they surprised and delighted the authorities by the keenness they showed, the hours they worked in the laboratory and the excellent results some of them produced. As the option is only marked PASS or FAIL the motivation Per obtained from them was clearly excellent.

Farewell [Friday 13.00-13.10]

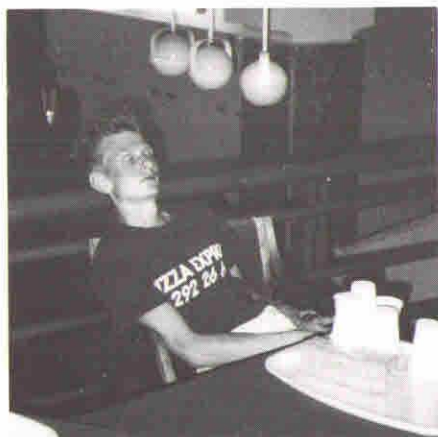
Erkki closed the conference by changing his shirt to one with a holiday look and feel and telling us he thought he deserved a holiday! The noise the audience produced was evidence that they agreed.



Hydrofoils on the Trip to Kronstadt

APL92 — BEKTOP at St Petersburg

notes by Ray Cannon



Pizza Express Staff worked 20-hour days

The slogan of Pizza Express (the Finnish firm who provided most of the catering for the conference) is "We deliver in Leningrad". Well, they almost did not.

As is now usual, Vector was to be given out to the delegates. The problem was, how to get it hot off the press from the printer's in the UK to the conference site. The solution was to send it to Helsinki by air mail, and then overland (via a Pizza Express lorry) to St Petersburg. Sounds fine in theory but... the Vectors (or BEKTOPs) were put on the P.E. lorry bound for St Pete's but

did not arrive in time to be included in the delegates' handouts on Sunday. They still had not arrived on Monday. By the end of Tuesday, I was giving up hope of ever seeing them. Finally on Wednesday, someone found them (in Pizza Express's deep freeze storage?) and they were delivered to the Conference. All's well that ends well.

IBM had hoped to provide all the delegates with a copy of the IBM Systems Journal which featured APL. Unfortunately, they appeared to have used a more conventional mode of transport. They never arrived.

I am sure there's a moral in there somewhere.

Evening Seminars

Summer evenings in St Petersburg are long, and the bar at the conference centre stayed open till about 1:30am. The normal beverage consumed was a Finnish canned beer called "Lapin Kulta". This was quickly renamed Rabbit Beer. There was also available "APL 92 Vodka" which has the slogan "Cool before discussion" printed on it.

By the time the bar closed, it would be starting to get light again (not that it really got that dark) and it would be time for "Evening Seminar". This ritual meeting of minds normally occurred in Sasha (Alexander Skomorokov)'s room. Often the participants would be adding the final touches to their paper (to be given the next day at the conference). These meetings lasted till about 3 or 4 am.

I used to wonder why most Russians walk round looking so glum; now I know, it's just a permanent hangover.



James Wheeler and John Scholes

Modern User Interfaces

James Wheeler (Manugistics STSC APL) and John Scholes (Dyadic Dyalog APL) hosted an excellent workshop on APL and Windows. Both vendors were offering a new version of their own product running under MS Windows. Surprisingly, at the workshop there appeared to be no rivalry, also neither product was shown or promoted. (This was reserved for the vendor forums.)

The workshop explained the different approach needed to program systems to run under Windows, and how the vendors had implemented the Windows

interface. The whole talk was rounded off with an excellent slide of "A piano designed by Bill Gates".

Manugistics APL under Windows. This product seems to provide a complete interface to MS Windows, by providing low-level access to every Windows procedure, thus allowing full Windows capabilities at the cost of a lot (in APL terms) of coding. As someone who has written Windows programs in C++, this APL Windows environment makes Windows programming a lot simpler and easier to debug.

DYALOG APL for Windows. Dyadic's Windows product is quite different. It is designed as an interface which is GUI-implementation independent. It provides a high-level control and a simple Windows interface. Although (at present) it does not provide access to all types of Windows resources (e.g. icons and bitmaps), it is VERY easy to use. (It took me about 20 minutes to produce a window containing "Bill Gates' Piano". Prior to that I had only had a 5 to 10 minute introduction to Dyalog for Windows.)

APL & WINDOWS. Dyadic and Manugistics products are very different and satisfy very different requirements. In the future they will both come closer together. In the meantime, some enterprising independent APLer should be able to write a set of high-level windows drivers for them, so that the same APL code can run under both vendors' products.

APL92 and the KGB

At APL92 we were very lucky to have as one of the invited speakers a former Colonel in the KGB, Alexander Kalinin. He was correctly introduced as "perhaps the most unusual speaker at our Conference".

Although his talk was interesting in its own right, as Andrei Kondrashev pointed out, it was more important to the Russian audience merely because it was now possible for such a talk to take place at all.

Alexander Kalinin worked in a special department that was responsible for information security and cryptography. Needless to say, he gave away no "trade secrets".

CTalk, an Acceptable Alternative to APL?

Jean-Jacques Girardot's paper "The CTalk Programming Language: A Strategic Evolution of APL" presented a theme that has been bandied about APL

conferences for many years now; that of producing a new language with all the strengths of APL but packaged to look more like C, Algol, PL/1, Pascal etc.

However, what made this admirable paper different, not only was the new language CTalk *proposed*, it had already been implemented, and was available (in several machine formats) at the software exchange. Well done.

(Unfortunately for me, when I got home and tried to decompact my copy of CTalk, I found that I have got the wrong version of the utility PKUNZIP to extract the files. Has anyone else out there got a copy of CTalk (for IBM PC) I could have?)

The Ultimate Question.

G.A.Langlet "Towards the Ultimate APL-TOE" (TOE=Theory of Everything)

As is well known to fans of The Hitch Hikers Guide to the Galaxy, the Answer to Life, the Universe and Everything, is 42. However, if you attended APL92, you might be forgiven for believing it to be "Difference Scan". In his excellent paper (and in many of the question and answer sessions following other papers) Gérard postulates that the NotEqual Scan is the ultimate (and possibly only required) algorithm. If you enjoyed reading Douglas Hofstadter's book "Gödel, Escher, Bach: An Eternal Golden Braid" read Gérard's paper.

Water Joke

Question. What's the difference between tap water at the Conference centre and its coffee?

Answer. The coffee is hot sweet black water, the tap water is cold bitter black water (and there's no milk in either of them).

St Petersburg is built on islands, hundreds of islands so water (and bridges) played a large part in the conference visits.

First the cruise to Kronstadt by high speed hydrofoils, to visit Menshikov's Palace at the Russian Naval base. I took several photos of Russian military hovercraft, and naval vessels. It was a very strange feeling openly walking around a former USSR naval base with a camera in my hand! On the trip back (by bus convoy) we had a police escort complete with flashing blue light. Heaven knows how many red traffic lights we passed.

Then the fireworks over the lake outside the Chinese Palace at Lomonosov, after an evening of Russian Music, Food, Culture, Dancing and Mosquitoes. (Followed by an exciting bus ride, with escort back home.)



The Chinese Palace at Lomonosov

Not forgetting the pond and fountain in the Hotel Petersburg before the Closing Banquet; the entertainment at the banquet reminded me of Las Vegas. Again the evening was ended by a high-speed dash through the streets of St Petersburg.



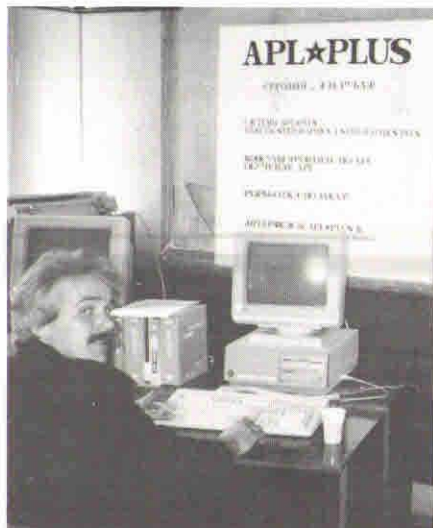
Poetry at the Chinese Palace

However my lasting memories of St Petersburg will not be the conference, the architecture, the bridges, the food, or the trips, but the stalls which surround the Metro stations like small shanty towns, selling almost anything from books to beer, and toothpaste to Vodka, but never windscreen wiper blades.

Pictures at an Exhibition



Peter Donnelly and Dyalog/W



Alexei Miroshnikov and APL*PLUS



The IBM Stand

Quotable Quotes from APL 92

by John Searle

1. An ASCII keycap set is available for APLers who want to learn J — Bob Brown
2. I wanted to divide this pie chart into 7 categories, but my drawing package couldn't do that, so I have 8 categories. I'll tell you later which one I didn't want — Martin Gfeller
3. It's difficult to understand that APL is so easy — a Russian student at the young APL world classroom



John Searle



The BEKTOP Shop!

4. I have one more slide, and then I'll ask questions — Bob Bernecky
5. Speed is not important until you've seen something go faster — Walter Spunde
6. In no other language could one afford the luxury of implementing so poor an algorithm to demonstrate the idea so clearly — Walter Spunde
7. If APL is so bad, why are so many people using it? — Cory Skutt
8. Please do not judge me by my lecture; I have many other good qualities — Elod Polgar
9. A good composer does not imitate, he steals — Igor Stravinsky quoted by Elod Polgar

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

Smalltalk, APL and J

An Exercise in Comparative Programming

by Dave Ziemann

Introduction

This idea was conceived when I started to learn Smalltalk/V Windows, the object-oriented programming language and environment. I wanted to find a small programming problem that I could use as a learning exercise. It had to be something well-defined, and preferably without any complicated inputs or outputs (I didn't want to get embroiled in user-interfaces quite this early), just something pure and algorithmic. I was also looking for a problem that would suggest the use of a distinctive feature of the language.

For some reason, I think someone was talking about them in the office, Hilbert Matrices sprung to my mind. I'm no mathematician, but I remembered that they were something to do with fractions, and that there are people who think that they are very useful. A Hilbert Matrix is a symmetrical matrix of fractions, each with a unit numerator. An order three Hilbert Matrix looks like this:

$$\begin{array}{ccc} 1/1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{array}$$

My problem would be to write a Smalltalk program to find the determinant of a Hilbert Matrix.

After solving this problem, which did help me to start learning Smalltalk, I wondered how easy it would be to do the same thing in APL. After I'd coded the problem in APL*PLUS II, my favourite symbolic dialect of that particular language, a further thought arose. Surely this problem could also be expressed very naturally in J, Ken Iverson's ASCII based rationalisation and extension of APL.

This article documents a solution of the Hilbert Matrix problem in those three languages, and makes some comparative observations. In each case the code is written in a manner appropriate to the particular language and environment being used.

The text of this article is written in a tutorial style for the benefit of those who are not familiar with Smalltalk or J.

The Smalltalk Program

It would have been possible to program a quick-and-dirty solution, just to get the job done, but I wanted a solution that would do reasonable justice to the object-oriented paradigm in general, and the Smalltalk/V Windows class hierarchy in particular. Furthermore, I resolved to find out about inheritance, polymorphism and encapsulation.

Let us look at a few examples of Smalltalk syntax:

```
42 sign
2 + 3
2 raisedToInteger: 3
chars at: 3 put: 'a'
```

In the first example we say that the "message" "sign" is sent to the number 42, the "receiver" of the message. The result will be 1 because 42 is positive. In the second example we would say that the message "+" is sent to the number 2, with an argument of 3. The result, or as they say in Smalltalk, the "answer", is 5. The third example shows a "keyword" message, which always contains a colon. Again the argument is 3 and the "receiver" of the message is 2. In the last example a keyword message is sent to a named object. This time the message is the `at:put:` message, which takes two arguments, one per colon. Any number of arguments can be passed into an appropriately named keyword message.

The three possible message types are known as unary, binary and keyword messages. Smalltalk expressions are executed left to right, but with unary messages taking precedence over binary, which in turn take precedence over keyword messages.

Smalltalk code is partitioned into "classes" which are distributed within a hierarchical structure, a small part of which follows:

```
Object
  Magnitude
    Character
    Date
    Number
    Float
    Fraction
    Integer
      LargeNegativeInteger
      LargePositiveInteger
      SmallInteger
```

Smalltalk provides a Fraction class, which directly supports operations on fractions, such as $3/4$, which are represented as pairs of integers.

A class is a template for creating objects, and each object is an instance of a class. The class "Object" sits at the top of the Smalltalk class hierarchy — all other classes being subordinate to it. As an example, we can say that the number 42 is an "instance" of the class SmallInteger, which is a subclass of Integer, which in turn is a subclass of Number, and so on.

A class whose name appears indented is a subclass of a less indented class higher up in the list. Each class in the hierarchy has a private collection of functions, or "methods" as they are called in Smalltalk. When a message is sent to an object the code that is executed depends upon the class of the object.

So when we sent the message "+" to the object 2, which is a SmallInteger, the system looked for a method called + in the SmallInteger class. In fact there is no such method in this class, so Smalltalk searches up the class hierarchy until it finds one. If it doesn't find one, then an error occurs. In our example a + method is found in the Integer class, and it is this code which is executed when the statement $2+3$ is evaluated.

In effect, the subclass inherits all of the methods in its superclass. This is an example of what is meant by inheritance. It is also possible to shadow a superclass method by redefining the method in a subclass. An example of this can be seen in the Fraction class, a subclass of Number, which has its own definition of the + method. This is a very powerful idea. It means that objects of different class can respond to the same message in different ways. This behaviour is known as polymorphism.

In Smalltalk, integer arithmetic is performed to arbitrary precision, so for example, a one-hundred digit number formed by successive multiplications is represented exactly. This precision is automatically extended to fractions, because the methods used in the class Fraction are themselves defined using integer arithmetic.

The Fraction class supports exact fractional arithmetic using a similar set of messages to that used by the Number class. Consider the following two statements:

```
2 + 3
5
1/3 + (1/6)
1/2
```

In the first case the + message is sent to the SmallInteger 2 so the + method in class Integer is executed, performing a simple integer addition. In the second example the + message is sent to 1/3, an instance of class Fraction. The Fraction class has its own definition of the + method, which knows how to add fractions. In a similar way the other common numerical operations have been subclassed in Fraction.

Smalltalk's direct support for the Fraction class will prove useful for the Hilbert Matrix problem. All we need to do is to create a matrix of Fractions. Well almost, except that Smalltalk doesn't know what a matrix is. So we need to tell it.

Let's look at another part of the class hierarchy:

```
Object
Collection
Bag
IndexedCollection
  FixedSizeCollection
    Array
    String
    OrderedCollection
```

An Array is a kind of FixedSizeCollection which is a kind of Collection, which in turn is a kind of Object, as everything eventually is in Smalltalk. Instances of the Array class therefore inherit the properties and methods of its superclasses. Thus a Smalltalk array is a fixed length, one-dimensional structure that can contain other Smalltalk objects.

Many of the methods in the superclasses of Array will be useful to us, but the Array class as it stands is not convenient for our purpose. Rather than attempt to modify the Array methods to meet our needs, we will add a new subclass to Array. In fact we will add three new classes to Smalltalk, as indicated by the bold class names in the following:

```
Object
Collection
Bag
IndexedCollection
  FixedSizeCollection
    Array
    Matrix
    SquareMatrix
    HilbertMatrix
    String
    OrderedCollection
```

The new classes can be defined in Smalltalk/V Windows either by writing program code to do it, or more easily, by using the mouse. If the mouse is used, all you have to type is the new class name.

The new structure recognises the fact that a Hilbert matrix is indeed a kind of square matrix, which in turn is a kind of matrix. Each class can be defined to include a number of "instance variables", variables which will be associated with all instances of that class. We will define Matrix to have two instance variables, called 'rows' and 'columns', to represent the number of rows and columns in the matrix. The SquareMatrix class will have an instance variable called 'order', which will represent both the number of rows and columns, the order of the square matrix. Finally, class HilbertMatrix will have no additional instance variables, but will support methods for initialising and displaying Hilbert matrices.

The use of three distinct new classes allows us to distribute code to those classes where it most naturally resides. For example, a method to index elements of a Hilbert matrix can be written for the class Matrix instead of the class HilbertMatrix, so that all types of matrix can use it. The code for calculating the determinant of a matrix can be placed into the SquareMatrix class, so that all square matrices, not just Hilbert matrices can understand the "determinant" message. (Actually one could define determinant for any shape matrix, but this is an additional generalisation that has not been made here.)

Let's look at the code for the class Matrix.

```
new: row by: column
    "Answer a new instance of Matrix with size row by column"
    ^ (super new: row*column) rows: row; columns: column
```

This is called a "class" method because the new:by: message is sent not to an instance of the class Matrix, but to the class itself, in order to create and initialise a new Matrix. The semicolon is a shorthand for sending another message to the same receiver object. We use this method to create a new 3 by 3 Matrix as follows:

```
M33:=Matrix new: 3 by: 3
```

The new:by: method in turn uses a superclass new: method, which is used to create an Array of appropriate length. A Smalltalk Array is a linear structure analogous to the APL vector. The circumflex indicates the result to be answered. We also define the methods rows: and columns: in the Matrix class in order to specify the number of matrix rows and columns:

```

rows: anInteger
    "Set the number of rows in the receiver Matrix"
    rows:=anInteger.

columns: anInteger
    "Set the number of columns in the receiver Matrix"
    columns:=anInteger.

```

In this way the instance variables `rows` and `columns` are defined for an instance of the class `Matrix`. There is no problem using method names that are the same as instance variable names. The next step is to provide access methods for these two instance variables, which are simply:

```

rows
    "Answer the number of rows in receiver matrix"
    ^rows
columns
    "Answer the number of columns in the receiver matrix"
    ^columns

```

Matrix elements can be indexed by the `at:at:` method, which takes the row number and column number as arguments:

```

at: i at: j
    "Answer receiver Matrix element i,j"
    | index |
    (i<1 or: [i>self rows] ) ifTrue: ['self error:'INDEX ERROR'].
    (j<1 or: [j>self columns]) ifTrue: ['self error:'INDEX ERROR'].
    index:=self loc: i with: j.
    ^self at: index

```

To make us feel at home, the method reports an APL-like error message when an index is out of range. The word `self` refers to the receiver of the message, in this case the instance of the class `Matrix` being indexed. The square brackets `[]` group blocks of Smalltalk code which are used here as arguments to messages such as `or:` and `ifTrue:`.

The `loc:with:` method converts the row number and column number pair into a single index value which is used to index the linear Array which represents the matrix. The method uses the existing `at:` method inherited all the way from the class `Object`.

```

loc: i with: j
    "Locate the Array index of receiver Matrix element i,j"
    ^(i - 1)*self columns+j

```

The specification of matrix elements is achieved with the `at:at:put:` method, which employs the existing `at:put:` method from the `Object` superclass.

```

at: i at: j put: data
    "Put data into receiver Matrix at element i,j"
    ^self at: (self loc: i with: j) put: data.

```

These methods define the message interface by which a Matrix object can be accessed. This "encapsulation" protects the instance variables of an object from access by the outside world.

Finally, although not strictly necessary for the problem, we will be polite and provide Smalltalk with a method to display matrices. When Smalltalk tries to display something, it looks for a method named `printOn:` in the class of the object to be displayed. We will see this method in action later:

```

printOn: aStream
    "Send formatted Matrix to aStream"
    1 to: self rows do: [:i|
        aStream nextPut: $(.
        1 to: (self columns)- 1 do: [:j|
            aStream nextPutAll:
                (self at: i at: j) printString, ' , '.
            aStream nextPutAll:
                (self at: i at: self columns) printString, ') ' ;cr].

```

Now for the code in the class `SquareMatrix`. An instance of this class is created by sending the message `new:` to the class itself, in a manner similar to the creation of a `Matrix` used above:

```

new: size
    "Create new SquareMatrix of order size"
    ^(super new: size by: size) order: size
order: anInt
    "Set the order of the receiver SquareMatrix"
    order:=anInt.
order
    "Answer the order of the receiver SquareMatrix"
    ^order

```

The `order:` method establishes the order instance variable, which is used to track the order of a `SquareMatrix`, i.e. the number of rows and columns. The heart of the algorithm is the following three methods, which calculate the determinant of a `SquareMatrix` by recursively computing the determinants of the imbedded minor matrices, until a 2 by 2 matrix is obtained, when the determinant is directly found:

```

determinant
    "Answer determinant of receiver SquareMatrix"
    self order=2 ifTrue: [^self determinant2by2].
    (1 to: self order) inject: 0 into: [:answer :j|

```

```

j odd
ifTrue: [answer+
  ((self at: 1 at: j) * (self minor: j)determinant)]
ifFalse: [answer-
  ((self at: 1 at: j) * (self minor: j)determinant)].
determinant2by2
  "Answer the determinant of the receiver 2 by 2 SquareMatrix"
  ^((self at: 1) * (self at: 4)) - ((self at: 2) * (self at: 3))

minor: anInteger
  "Answer a minor matrix in the receiver formed by deleting
  row 1 and column anInteger"
  | result ir |
  result:=self class new: order - 1.
  ir:=1.
  order+1 to: order*order do: [:is|
    (is- 1\\order)=(anInteger- 1)
    ifFalse: [result at: ir put: (self at: is).
      ir:=ir+1]].
  ^result

```

The polymorphic nature of the arithmetical messages such as * will apply integer arithmetic to integers, and fractional arithmetic to fractions.

Finally the code for the class HilbertMatrix. Again, we have a class method which is used to instantiate a HilbertMatrix, and an initialize method to populate the matrix with fractions:

```

new: size
  "Create new HilbertMatrix of order size"
  ^(super new: size) initialize
initialize
  "Initialize the receiver HilbertMatrix"
  1 to: self order do: [:i|
    1 to: self order do: [:j|
      self at: i at: j put: (1/(i+j- 1))]].
  ^self

```

Smalltalk is largely written in Smalltalk, which is great because it means you can see how it works, learn how to program, and re-use existing code. It also means that you are bound into the North American spelling system, as indicated by the above method name. Of course you can stand firm and spell "initialise" with an "s", but unfortunately this inevitably leads to confusion and error.

We can write another HilbertMatrix class method to produce a report, which will be run in a few moments:

```

reportMany: orders
    "Show a report of the order and determinant for
    Hilbert matrices of order orders"
| report det |
report:=WriteStream on: String new.
report nextPutAll: 'Order   Determinant' ; cr.
orders do: [:order|
    det:=(HilbertMatrix new: order) determinant.
    order printOn: report.
    report nextPutAll: '      '.
    det printOn: report.
    report cr].
Transcript nextPutAll: report contents.

```

Interestingly, the HilbertMatrix class itself has very little code in it. As you can see from this design, most of the code is utility code, useful in a more general context than just for finding the determinant of a Hilbert matrix.

We are ready to create a Hilbert Matrix and find the determinant:

```

HilbertMatrix new: 3
(1 , 1/2 , 1/3)
(1/2 , 1/3 , 1/4)
(1/3 , 1/4 , 1/5)

(HilbertMatrix new: 3) determinant
1/2160

```

Notice how the printOn: method, defined in the Matrix class, is inherited by its subclasses, and quite happily displays a HilbertMatrix. The Fraction class also has its own printOn: method, which knows how to display the fractions themselves.

Let's now run the reportMany: class method, to produce a list of Hilbert Matrix determinants:

```

HilbertMatrix reportMany: (1 to: 7)
Order   Determinant
1        1
2       1/12
3      1/2160
4     1/6048000
5    1/266716800000
6   1/186313420339200000
7  1/2067909047925770649600000

```

The arbitrary precision arithmetic has done its stuff — the determinants are exact. This is not something we can expect to happen in APL or J.

The APL Program

The APL version was written in the second-generation APL*PLUS II system, using a direct definition handler that I wrote some years ago, just for this kind of development. Index origin zero is used throughout.

APL has no special datatype for fractions, so we will have to use an existing data structure to represent a fraction. A vector of fractions could reasonably be represented by a 2-column matrix, the first column for the numerators and the second for the denominators. In general an array of fractions would then always have an additional trailing dimension of length 2.

But I have chosen to represent an array of fractions as an array of enclosed integer pairs, because it makes the application of certain operations simpler, it results in an array whose shape does not artificially include a 2, and because the default display of such arrays will always juxtapose the numerator and denominator in each fraction.

A 2-element integer vector is used to represent the numerator and denominator of the fraction. The function *fraction* is used to form vectors of enclosed fractions as follows:

```
fraction: a,"w
2 fraction 3
2 3
1 fraction 2 3
1 2 1 3
```

A fraction formatter will also be useful. Here it is:

```
fmt: fmt"ω : 1=ω : (⍺1↑ω), '/' , (⍺1↑ω)
fmt 2 3
2/3
fmt 1 fraction 2 3
1/2 1/3
```

The *fmt* function uses the three-segment form of direct definition, where if the middle segment evaluates to 0 then the left segment is executed, otherwise if the middle segment evaluates to 1, then the right segment is executed. The recursive definition extends the domain of the function to arrays of enclosed fractions.

A Hilbert matrix of any order can then be generated by using the function *HM*, which is defined as follows:

```

HM:HM: 1 fraction (1ω)•.+1+1ω

HM 3
1 1 1 2 1 3
1 2 1 3 1 4
1 3 1 4 1 5
    fmt HM 3
1/1 1/2 1/3
1/2 1/3 1/4
1/3 1/4 1/5

```

Now that we can represent fractions in APL we also need to be able to take their product and difference, in order to compute determinants. The product of a pair of fractions is just the product of their corresponding denominators and numerators. But we will also need to simplify the resulting fraction, which may not be in its reduced form. So we have:

```

times: simplify α×ω
simplify: ⌊ω÷GCD/ω
GCD: z : 0≠v+|ω Δ z+|α : v GCD v|z
Δ: α

2 3×3 4
6 12
    simplify 2 3×3 4
1 2
    2 3 times 3 4
1 2

```

The *simplify* function works by dividing the numerator and the denominator by the Greatest Common Divisor of the two. A reduction is performed on the integer pair using the recursive *GCD* function. (The direct definition handler automatically localises the variable *v* to the function *GCD* because it is assigned therein.) *Δ* is a statement separator utility.

Fraction subtraction and addition are analogously defined as follows:

```

minus: simplify(-/α×φω),(1>α)×1>ω
plus:  simplify(+/α×φω),(1>α)×1>ω

```

The *plus* function is shown here for interest only — it is not required for our purpose.

As in the Smalltalk solution, the general determinant is found by a recursive application to each of the minor matrices in the argument matrix. A minor matrix is produced by dropping the first row and *n*th column of its argument, where *n* is the left argument of the function *minor*:

```

minor: 1 0+(α-1+ρω)/ω

fmt 1 minor HM 3
1/3 1/4
1/4 1/5

minors: (1+ρω)minor"ω

```

where the function *minors* generates every possible minor matrix. So the determinant functions are as follows:

```

determinant: >minus/ω[0;]times"determinant"minors
              : 2=1+ρω
              : determinant2by2 ω

determinant2by2 :>minus/times/0 1ω

```

where the second function produces determinants for matrices of order 2 or less. Let's try it out now:

```

fmt determinant HM 3
1/2160

```

Finally we can write a reporting utility as we did in Smalltalk:

```

report: (↑ω){fmt determinant HM ω}
reportMany: 'Order' 'Determinant' ↑report"ω

report 3
3 1/2160

reportMany 1+17
Order Determinant
1      1/1
2      1/12
3      1/2160
4      1/6048000
5      1/266716800000
6      0/1
7      0/1

```

Notice that APL's 4-byte integer precision is insufficient for the exact computation of Hilbert matrix determinants beyond order 4. The order 5 result is correct "by accident", thanks to the trailing zeros in the result. We can expect to encounter the same behaviour in the J program, to which we now turn our attention.

The J Program

At first sight one might think that the J version of this problem would be very similar to the APL solution, and in many ways it is.

The representation of a fraction is however, subtly different in J. Instead of a 2-element pair of integers, a *boxed* 2-element pair of integers is used. But isn't that just the same as enclosing the fraction in APL? Yes, but in the APL solution a fraction was carefully defined as an integer pair, not an enclosed integer pair. A major reason for this decision is that in APL2-type systems, the reduction operator applies the function between the disclosed elements of its argument data and not between the elements themselves. So by defining an APL fraction as an integer pair, we could use reduction to apply functions such as minus and times to vectors of enclosed fractions. The alternative would have been to apply a somewhat artificial enclose-each operation prior to any reduction on vectors of fractions.

J does not perform implicit operations when its analogous insertion operation is performed, so we can represent a fraction as a scalar, a boxed integer pair, which is constructed by the following verb called `fraction`. This verb is defined for clarity in terms of the utility adverb `each`, which is used again later.

```
each=. . &.>
fraction= . ,each
```

```
1 fraction 2 3
```

1	2	1	3
---	---	---	---

Programming in J encourages an even greater degree of modular decomposition than in APL, particularly if you want to develop code "on the fly", or if you want to be able to read your code later.

The denominators of the fractions in a Hilbert matrix can be generated with the `HMden` verb, which as before builds an addition table. The verb demonstrates the application of the monadic hook `+/>:` in which the verbs `>:` (increment) and `+/` (table) are a monad and a dyad respectively.

```
0 1 2 +/ 1 2 3
```

```
1 2 3
2 3 4
3 4 5
```

```
(+/>:) 0 1 2
```

```
1 2 3
2 3 4
3 4 5
```

```
HMden=. (+/>: )@i.
```

```
HMden 3
```

```
1 2 3
2 3 4
3 4 5
```

The Hilbert matrix verb is formed by composing the verbs `fraction` and `HMden`, as follows:

```
HM=. 1&fraction@den
```

```
HM 3
```

1	1	1	2	1	3
1	2	1	3	1	4
1	3	1	4	1	5

The J fraction formatter is built from two components, one which formats a number as an integer, and one which sticks a slash between two character vectors, as follows:

```
f=. 0&":
c=. , '/'&,
f 2
2
'2' c '3'
2/3
2 c&f 3
2/3
c&f/ 2 3
2/3
```

The dyads `c` and `f` are composed and inserted between the numerator and the denominator of the fraction. But the J fractions are boxed, so the expression must first be modified with the `each` adverb, as follows:

```
fmt=. c&f/each
```

```
fmt 1 fraction 2 3
```

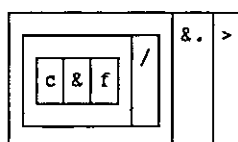
1/2	1/3
-----	-----

fmt HM 3

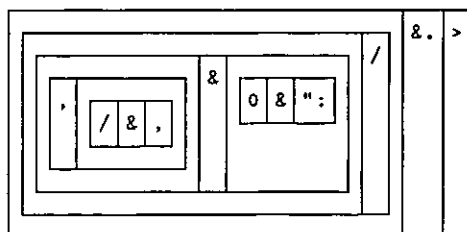
1/1	1/2	1/3
1/2	1/3	1/4
1/3	1/4	1/5

The verbs `c` and `f` were used to simplify and clarify the development process, and need not be referenced in the definition of `fmt`. These references can be removed by "fixing" the verb with the `f.` adverb, as follows:

fmt



```
fmt=.fmt f.
fmt
```

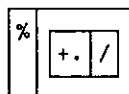


The next job is to write the verbs for multiplying and subtracting fractions. A `simplify` verb similar to the APL function is used, but this time we can exploit the fact that J provides us with a primitive GCD operation, the `%.verb`:

```
2 6 % +./ 2 6
1 3
```

```
simplify=.%. +./
```

```
simplify
```



```
simplify 2 6
1 3
```

The monad `simplify` is a monadic hook, where the GCD-insert prong `+/` is a monad, and the divide prong `%` is a dyad. The `times` verb is formed by composing `simplify` with `multiply`, as follows:

```
times=. simplify@:*each
```

Note the use of the `@:` conjunction (at) rather than `@` (atop). The use of `@` would cause the rank of the derived verb to be the same as the rank of `multiply`, i.e. zero, whereas the infinite-rank `@:` ensures that `simplify` is applied once to the entire argument, in this case a two-item list of numbers, rather than to each of the two scalars. The `each` adverb is used to open the boxes before the operation is applied to the corresponding pairs of boxed fractions.

```
fmt (2 fraction 3) times (3 fraction 4)
```

```
1/2
```

```
fmt times/2 3 fraction 3 4
```

```
1/2
```

Subtracting two fractions is a little harder: the resultant numerator is the product of the first numerator and the second denominator, minus the product of the first denominator and the second numerator. Put more concisely, it is the difference of crossed products of pairs, as follows:

```
dcp=. -/>(*|.)
```

```
2 3 dcp 3 4
```

```
_1
```

The hook `*|.` takes the product of the left argument with the reverse of the right argument. The product of the two denominators yields the denominator of the result, as in the following verb `pot`, product of tails:

```
pot=. *&{:
```

```
2 3 pot 3 4
```

```
12
```

These two verbs can be composed under the "append items" verb `(.)`, using a dyadic fork, as shown in the completed verb `minus`:

```
minus=. simplify@(dcp,pot)
```

As before, the analogous verb `plus` is not required here, but is included for interest:

```
scp=. +/>(*|..)
plus=. simplify@(scp,pot)
```

The Dictionary of J reveals that a determinant primitive is provided; the dot conjunction used with `-/` and `*` as follows:

```
i.2 2
0 1
2 3
-/ . * i.2 2
_2
```

This being J of course, we can substitute our defined verbs `minus` and `times` for the primitives in the above definition. Furthermore, the J primitive is already defined recursively for higher order matrices. So we can say:

```
determinant=. (minus each) / . (times each)
fmt determinant HM 3
```

1/2160

Again, the verbs are modified with the `each` adverb in order to open the boxed pairs before applying the operation, and to box them again afterwards. Finally we can write the reporting utilities:

```
report=. ;fmt@determinant@HM
reportMany=. ('Order';'Determinant')&(<report"0)
```

```
report 3
```

3	1/2160
---	--------

```
reportMany >:i.7
```

Order	Determinant
1	1/1
2	1/12
3	1/2160
4	1/6048000
5	1/266716800000
6	1/186313420308152800
7	1/2067909088097816940000000

As with the APL solution, the order 6 and 7 results are not correct, as can be verified by comparing them with the exact Smalltalk answers.

Comparing the Solutions

Development

It is difficult for me to estimate objectively the development time for each program; the Smalltalk code was produced while on a relatively steep part of a learning curve, as was the J code, to a lesser extent.

Furthermore, the development time would depend on how much of the utility code already exists in each case. In the case of J, it was not necessary to code some of the hard algorithmic parts (GCD, determinant) because they are primitives in the language. With Smalltalk, a lot of useful code becomes naturally available via inheritance when the new classes are inserted at an appropriate place in the class hierarchy.

In any event, the problem bears little resemblance to a typical commercial program, where a large proportion of the effort is normally associated with user input, output and data validation issues.

Nevertheless, I was impressed by the short time it took to get the Smalltalk solution working; although there is a larger code volume it was straightforward and quick to write, and the testing and debugging facilities are a dream compared with either APL or J.

Execution Speed

The following table summarises comparative execution speeds. Each entry is the time in seconds to generate a Hilbert Matrix and find its determinant (but not to output the result).

order	Smalltalk	APL	J
1	0.0	0.0	0.01
2	0.0	0.03	0.04
3	0.01	0.13	0.14
4	0.05	0.60	0.55
5	0.25	3.1	2.7
6	1.7	19.7	16.5
7	13.4	142	114

We must be careful not to infer generalisations from these timings; for example, the highly modular APL model is certainly not optimal as far as execution speed is concerned. Nevertheless, the Smalltalk timings were a pleasant surprise, particularly when you remember that it is performing arbitrary precision arithmetic.

The use of the `f`. "fix verb" primitive did not result in a significant performance improvement to the `J` determinant verb.

Code Re-use

It is interesting to review the situation from the point of code re-use. There are two aspects to this: first, how much code can be raided from existing sources in order to develop the application; and second, how much code which might be re-used elsewhere did the development generate.

This was my first excursion with Smalltalk. Although I had no existing code to raid, inserting the new classes beneath the Array class meant I inherited all of the superclass code for free. In APL I was able to raid the `GCD` function and the trivial `␣` statement separator utility. In J, only the `each` adverb was needed.

What about the prospects for re-using any of the Hilbert Matrix code? We can attempt a crude re-usability metric by scoring each module as follows:

- 0 — module is specific to this application
- 1 — module could be re-used in a similar application
- 2 — module could be re-used in any application

In Smalltalk, we might expect the module re-usability index to increase as we climb higher up the class hierarchy, as the classes become more and more general. In this application however, I have rated the 9 methods in the Matrix class and the 6 methods in the SquareMatrix classes at 1, as I feel that the difference in usability between the two classes is slight. The 3 HilbertMatrix methods get a score of 0, producing an index of 15 for a total of 18 modules.

It's interesting to see what a small fraction of the Smalltalk code is specific to this problem; it implies that levels of re-use will be particularly high when raiding code for the development of similar applications, something that is not necessarily the case in APL or J, where stronger programmer discipline is required to generate re-usable code. (Modifying a copy of a function is a very poor form of code re-use.)

I rate the APL function Δ at 2 points, the functions *HM*, *determinant*, *determinant2by2*, *report* and *reportMany* at 0, and the remainder at 1, giving a re-usability index total of 10 for 14 modules.

I rate the J each adverb as 2, the verbs *HM*, *HMden*, *determinant*, *determinant2by2*, *report* and *reportMany* at 0, and the remainder at 1, giving a total of 9 for 13 modules.

Dividing through gives a code re-use index per module of .8, .7 and .7 for the Smalltalk, APL and J solutions respectively. I'm not sure how meaningful this is for such a small application, or even if this is an appropriate scoring method for comparison across languages and environments. Perhaps further studies will reveal if Smalltalk's claim for high re-use can be supported.

Configuration

The developments and timings were made using an Olympic 486 33Mhz machine, running Windows 3.1. The application software was Smalltalk V/Windows version 1.1, APL*PLUS II/386 version 4.0 and J PC version 5, all running under Windows. The original of this document was created using Windows Write, and Adrian Smith's TrueType APL font. Thanks to Jonathan Barman and Richard Green for feedback during the preparation of this article.

References

- [1] *Smalltalk V/Windows Tutorial and Programming Handbook*, Digitalk, 1991
- [2] *There & Back Again - A Quick Trip To ObjectLand*, Gene Korienek & Tom Wrench, Artifact Inc, 1991
- [3] *ISI Dictionary of J*, Kenneth E. Iverson, Iverson Software Inc., 1991
- [4] *An Introduction to J*, Kenneth E. Iverson, Iverson Software Inc., 1992

Object Oriented Extensions to APL and J

by Robert J. Frey

Introduction

This paper is a preliminary investigation, motivated by a series of simple examples rather than theory, into how object-oriented (OO) extensions can be made to APL in a parsimonious manner, i.e. in a way which maintains compatibility with the existing language, which introduces few new language constructs and which, most importantly, does not change those characteristics which make APL unique.

The first section contains a brief introduction to OO programming concepts. This is followed by a two-stage description of our proposed extensions. The paper next gives a short critique and suggestions for future research. It concludes with several examples where OO capability yields simpler and more robust designs than are possible in APL. The work here is preliminary, but we have tried to be as complete as possible in the development of our ideas.

Object-Oriented Programming and System Design

Software *modularity* and *reusability* have proven to be elusive goals in computer programming. By modularity we mean software elements which: correspond to syntactic units in the language, have explicit interfaces and provide a high degree of information hiding. A modular language allows us to decompose complex problems into subproblems which can then be solved separately. Because such modules tend to be relatively simple and focus on the solution of a single functional requirement, modularity makes software more understandable and encourages the construction of new software elements by combining existing ones in new ways. Finally, modularity restricts the scope of both design changes and execution side-effects to one (or at least a few) modules [Meyer 88].

Reusability imposes certain requirements on modules. First, modules should have a measure of generality: e.g. if we implement a linked list module, then linked lists composed of different types of elements should not require different versions of the module. Second, similar types of module should behave in similar ways: e.g. ordered lists and unordered lists should have many common aspects of behaviour. Third, related functions and data must be grouped together. Fourth, users of a module must be able to use it without knowing how it is implemented. And fifth, related groups of modules should be able to share common parts of their implementations [Meyer 88].

Object-oriented programming languages seek to achieve modularity and reusability through four characteristics: *abstraction*, *encapsulation*, *inheritance* and *polymorphism*. Abstraction means that the user only needs to know what a module does, not how it does it. Encapsulation means that the details of both the interface and implementation are grouped together into a unit. In fact, objects are encapsulations of abstractions [Pinson 88]. Abstraction and encapsulation focus on achieving the goal of modularity.

Inheritance and polymorphism on the other hand provide mechanisms for managing objects in ways which enhance their reusability. Inheritance refers to the ability of one object (the child) to inherit some of its behaviour from another (the parent). If a child can inherit from more than one parent, then this is said to be multiple inheritance [Meyer 88].

Polymorphism is the ability to take on more than one form. This is manifested in two ways. First, the same name may mean different things depending upon the type of object being referenced. Thus, if we have a list of polygons, then asking a "square" what its area is calls a different routine than the same request of a "circle". Second, in strongly typed languages it represents a relaxation of type checking to the extent that if a routine calls for an instance of a particular type, then it is also valid to use a type which is a child of it [Meyer 88].

Although the details vary from language to language, objects are typically referenced by an `<object><message><arguments>` triple. The `message` generates a call to one of the methods located in `object` with the `arguments` given.

The form is often `object.message(arguments)` as it is in Eiffel, C++, Objective-C and Turbo Pascal. This notation is an extension of structure or record field referencing. For more information readers are encouraged to consult the many good introductions to OO programming such as [Cox 87], [Meyer 88], [Pinson 88] and [Stroustrup 86] in the bibliography.

Extending a Non-OO Language

While there exist languages which were envisioned from the start to be OO, e.g. Smalltalk and Eiffel, many OO languages have been developed by extending an existing language to directly support an object paradigm. Turbo Pascal, LISP, Objective-C and, the best known, C++ are all examples. In fact, the oldest and still one of the best is SIMULA which grew from ALGOL [Meyer 88].

Many of the features of OO design can be used in a non-OO language by establishing a disciplined approach to object definition using a metaphor native

to the language at hand. For example, we can bind data and functions together in C by building structures which contain pointers to functions. In C++ this *programming-discipline-plus-object* metaphor has been replaced by extensions to the language itself. Nevertheless, the motivating metaphor remains: C++ objects look very much like structures. Structures are themselves just the simplest objects, ones containing only data [Stroustrup 86].

We want to extend APL to support an OO paradigm in a way which both preserves the character of the language and still provides abstraction, encapsulation, inheritance and polymorphism. Given those objectives, we must begin with some native metaphor which gives us most of the capabilities of OO programming in the language as it is now. There are many ways support of OO programming can be added to APL, but most of the obvious ones are precluded by these requirements. For example, adding some form of typing or fixed object definition would change the language in ways we would find unacceptable. Alternatively, binding of data and functions by permitting functions to exist as array elements (or gerunds in J) is extremely cumbersome: arrays do not permit us to reference their elements by name.

What structure exists in APL which can be exploited in this way? APL contains variables, functions and operators (in J, nouns, verbs, adverbs and conjunctions) bound together in a workspace. Elements in a workspace are defined through assignment augmented by a collection of structure functions, such as *reshape* and *transpose*. Thus, in APL the most natural mechanism for binding together disparate elements into groups within which they can be referenced by name is the workspace itself.

We propose that the creation of objects in APL be accomplished by binding together variables, functions and operators into a subspace — a local workspace in which all name references are strictly internal and in which communication with the “outside” is accomplished primarily by passing arguments to and receiving results from a set of public functions. Such a metaphor is parsimonious; yet, allows us to introduce a full complement of OO features into the language.

Subspaces will be described in two parts. The first will introduce a single new notational convention into APL. In this preliminary form subspaces provide a facility somewhat more powerful than, say, packages in APL2 or modules in Modula-2, but somewhat less than true classes in a full OO language such as Smalltalk. We have chosen the J dialect of APL [Iverson 91] as the basis for our exposition because it is relatively new and perhaps more amendable, because it appears to be extremely portable, and because its developers have generously released its source code.

Extending J – The In-Object Paradigm

By way of an example, we define an association list subspace. The `list.` subspace (the period is significant) uses a global noun `.data`, which is a two-column table of boxed elements. Each row (*TAG*; *ITEM*) of `.data` contains the association tag and data for the list. The noun `.data` is global only within the `list.` subspace, and as is true for all objects within a subspace it cannot be referenced directly outside the subspace.

Names can be referenced by identifying the subspace, using the form `subspace.name`. In the case of `.data` the `"."` prefix precludes this form of reference, forcing private usage of the pronoun. Note the conventions introduced so far:

<code>subspace.</code>	subspace reference
<code>subspace.name</code>	reference to name within subspace
<code>.name</code>	strict internalization of name within subspace

Obviously, subspace names must contain at least two characters; otherwise, there would be conflicts with existing J primitives (such as `i.` or `e.`). We assume the session manager has been extended so that the subspace context can be set and that all definitions immediately below occur within the context of the `list.` subspace.

We will need the following proverbs. Unless otherwise noted they return the final value of `.data`; this is for purposes of exposition only and is represented schematically by `LISTDATA`. The non-J character `"←"` (left arrow) is used below to indicate the type of value returned. The first is a dyadic proverb which creates or re-initializes the list.

```
usage:      LISTDATA ← TAG list.create ITEM
definition:  create =. ' ' : ' .data=:1 2$<x.;y.'
```

The next is a monadic proverb which tests for the presence of a tag in the list, returning a 1 if found and a 0 otherwise, represented schematically by `BOOLEAN`.

```
usage:      BOOLEAN ← list.contains TAG
definition:  contains =. '(<y.)e. 0{"1 .data' : ''
```

This monadic proverb returns the item from the list given its associated tag.

```
usage:      (TAG; ITEM) ← list.fetch TAG
definition:  fetch =. '>{(0{"1 .data)i.<y.){1{"1 .data' : ''
```

The last is a slightly more complex example. The dyadic proverb `list.insert` will add a new item or update an existing one. It uses two private (i.e. private to `list`, not private to `list.insert`) proverbs, `.add` and `.replace`. Note that the reference is to `contains` and not `list.contains` because `list.insert` executes within the context of the `list`. subspace.

```
usage:      LISTDATA ← TAG list.insert ITEM
definitions:
    .add =. ' ' : ' .data=: .data, x.;y.'
    .replace =. ' ' : ' .data=: (x.;y.)((0{"1 .data}i.<x.)) .data'
    insert =. ' ' : 'x. ((.add' .replace)@.contains x.) y.'
```

Here is a sample session using the `list`. subspace:

```
'abc' list.create 10 20 30
```

abc	10 20 30
-----	----------

```
'def' list.insert 100 200
```

abc	10 20 30
def	100 200

```
list.contains 'abc'
```

```
1
```

```
list.fetch 'abc'
```

```
10 20 30
```

```
'abc' list.insert 50 1000 2 10 3 5
```

abc	50 1000 2 10 3 5
def	100 200

```
list.contains 'yz'
```

```
0
```

```
'yz' list.insert 1 1 2 3 5 8
```

abc	50 1000 2 10 3 5
def	100 200
yz	1 1 2 3 5 8

```
list.contains 'yz'
```

```
1
```

Note that `.data` is not directly accessible outside the subspace, although later we will provide a general mechanism for by-passing the interface to reference private names. Users of `list.` do not need to concern themselves with implementation, only usage. If we wanted to give a user read only access to `.data` we could define a proverb:

```
getdata =. ' .data' : ''
```

We can assign one subspace to another; "inheritance" for subspaces is handled as inheritance for other elements in J, by "assignment" suitably augmented by "structure functions":

```
anotherlist. =. list.
```

This sentence would duplicate the elements of one subspace in the other. If we executed `anotherlist.create`, the `create` proverb would execute in the context of the subspace from within which it was called. Thus, the `.data` pronoun that was re-initialized would be the copy in `anotherlist.` while the copy in `list.` would remain unchanged.

As it does in J normally, a simple assignment of one subspace to another results in inheritance by reference for verbs. Invoking a proverb in `anotherlist.` causes the associated proverbs in the original `list.` subspace to be called, but that execution would occur in the context of the `anotherlist.` subspace. If we wished to accomplish inheritance by value, then we would apply the fix adverb `f.` as follows:

```
anotherlist. =. list. f.
```

This would result in an actual copy of verb values from one subspace to another.

The difference is important. In the first case changes in `list.` would be reflected in `anotherlist.` and in the second the two subspaces would have independent existences. The precise mechanisms of inheritance would be different for different dialects of APL, but we would certainly always want to be able to control which sort of inheritance, by reference or by value, we were applying. This allows us to set up subspace prototypes without introducing strict typing into the language. Multiple inheritance could be supported by having the "methods" of a particular subspace refer back to more than one prototype.

Subspaces in their current form give us a capability somewhat more than simple modules as in Modula-2, Ada and some Pascals because they function as more than simple libraries of procedures and functions with a well-defined interface. Each subspace is an independent context, and one would anticipate that multiple

instantiations would be the rule. Also, real inheritance is provided for in this scheme.

As a matter of fact, subspaces alone add a number of capabilities to APL and J that are extremely useful. For example, we can have verbs with static nouns (nouns with values that persist between calls), record structures (subspaces consisting only of nouns), program modules and libraries and simple objects such as our `list.` subspace above. All this goes a long way towards satisfying our original goals of modularity and reusability even if it falls short of full OO capability.

Extending J – The On-Object Paradigm

To see where subspaces fail to give us full OO capability, we'll return to our simple example. Assume we have two lists, `alist.` and `blist.`, which we wish to merge and assign the result to the `alist.` list. A natural expectation would be to define a monadic proverb `merge` in the `alist.` subspace which is used:

```
alist. =. alist.merge blist.
```

This is a more general use of subspaces because it involves an operation *on* a subspace rather than *in* one. To accomplish this extension we introduce one new conjunction `::`, which returns the entire subspace upon completion. There are also two prospaces, `L.` for the left subspace argument and `R.` for the right. And we must have a conjunction `s.` to force a reference in one subspace to execute in the context of a specified subspace. Then `merge` can be defined:

```
merge =. 'insert"1 R. s. .data' :: ''
```

Definitions made with the `:` conjunction use the “in” paradigm and definitions made with the `::` conjunction use the “on” paradigm. It is through the `s.` conjunction that inheritance from prototypes is accomplished. Thus, in our original example if the `anotherlist.` subspace inherits from the prototype-parent subspace, then a verb method in the child becomes:

```
list. s. method
```

... in the child.

We must also be able to treat subspaces as we would any other first-class object. For example, if we have a subspace `employee_record.` containing only the pronouns `name`, `id_number` and `address`, then

```
employee_record.name list.insert employee_record.
```

... would insert that subspace as an element in the `list. subspace`. Note that what is stored is not a reference to `employee_record`, but its value. If we subsequently deleted the named subspace then the list would still contain those values. Thus, subspaces do not need to be named explicitly, any more than, say, the elements of a numeric array need to be individually named.

It is clear that unless we can use subspaces as software elements in their own right, and not simply as a way for grouping references, we do not have a true OO programming language.

Review and Critique

Have we achieved the four capabilities required for OO programming: abstraction, encapsulation, inheritance and polymorphism?

Subspaces are true abstract data types. Users need only concern themselves with the interface and not the implementation. We could change our `list. subspace` in some very fundamental ways, e.g. making it file-based rather than using an in-memory array, and, as long as we did not change its interface, *not a single line of user code would need to be changed*.

Encapsulation is achieved by clearly separating public and private names. The interface between a subspace and the outside world is clearly defined. A limited form of access to private references is granted via the new `s.` conjunction.

Inheritance for subspaces is handled as it is for any other object in APL or J: by assignment augmented by appropriate structural operators. We further assume the session manager is extended so that a user can "enter" and "leave" a subspace so that the construction of a subspace can also proceed in the same way a user creates a workspace now. Multiple inheritance is supported.

Finally polymorphism, it turns out, is easy. APL and J are naturally polymorphic. Because they are not statically typed, there is no need to employ special mechanisms which allow for type generalization.

One concern one may have is that subspaces encourage the use of side-effects. Clearly, if one's ideal of programming is to use tacit programming in J to produce pure functions, this will be of some concern. However, properly constructed subspaces will encapsulate their state in private names and, hence, will not have side-effects.

Anyone who has worked on large APL-based systems has had the experience of seeing a clear and coherent design become obscured in implementation by the

large number of global variables and low-level support functions that inevitably seem to clog up the workspace. It is little comfort that we have avoided side-effects in our functions when our workspaces are littered with them.

The ability to group variables and functions into packages, as is possible in some APLs, is a major advance, but falls short of providing all the capabilities we need to begin to build modular and reusable software in a disciplined manner. Packages do not go far enough and are too difficult to use. They are not really part of language.

Another issue is that of efficiency and complexity. As the depth and size of the inheritance tree grows, there is a real possibility that the overhead required to call functions, and the difficulty of understanding the system *in toto*, will become too great to be tolerated. Experience with Smalltalk, however, has shown this not to be the case. It should be possible to design an OO-extended APL or J interpreter which both performs well and is adequately supported by tracing and browsing tools. In fact, because many operations will occur using ordinary APL and J primitives (unlike Smalltalk where everything is an object including integers, characters, etc.) it should be possible to produce interpreters which have significantly better performance than found in Smalltalk.

Some Applications

Here are some possible applications. Consider static nouns in verbs: there are many instances where some notion of state must be encoded in one or more global variables. For example, in a pseudo-random number generator, the next number in the sequence is calculated by applying some function to the previous one.

Subspaces consisting only of nouns are record-like objects which nevertheless do not impose static typing on the language and maintain APL's characteristic flexibility. A vector, each element of which is a boxed subspace of "employee records", is a much easier thing to work with than a matrix in which we must know the convention that column 1 is the name, column 2 the address, etc.

Standard libraries or software modules can be implemented with subspaces. For example, a time series "library" subspace can be designed so that it expects input in the form of some "record" subspaces which have "fields" describing the start and end dates, the frequency (daily, monthly, etc.) and the time series data themselves. It would be easy then to define a library function that could add together two time series, producing as a result a new time series containing the sums at their intersecting dates.

Another example where software modules would be useful would be as interfaces to external processes or devices. Graphics is a good example in which a module interface which hid global variables and low-level functions would be welcome. A database interface is another.

OO languages are inherently extensible. SIMULA, although it is often characterized as a simulation language, is actually an OO language which has been extended using its native facilities to support discrete event simulation [Meyer 88]. The same could be done for an OO-APL. Artificial intelligence is another area in which language extensibility could be exploited.

Conclusion

Subspaces are a simple and parsimonious way to introduce many useful extensions into APL; the most significant of which is to support a full object-oriented model. As experience with Smalltalk has shown, efficient and usable interpreters and environments can be implemented, and there is every reason to expect that an OO-extended APL or J could be implemented even more efficiently.

APL is extremely powerful, but it is not particularly extensible and does not adequately support information hiding. These shortcomings can make it unnecessarily difficult to design, implement and manage medium to large systems. It is time we rectified these deficiencies.

Robert J. Frey, Ph.D
Renaissance Technologies Corporation
100 North Country Road
Setauket, NY 11733, U.S.A.

Bibliography

- [Cox 87] Brad J. Cox, *Object Oriented Programming: An Evolutionary Approach*, Addison-Wesley Publishing Company, Reading, Massachusetts, 1987.
- [Iverson 91] Kenneth E. Iverson, *ISI Dictionary of J (Version 4)*, Iverson Software, Toronto, 1991.
- [Meyer 88] Bertrand Meyer, *Object-oriented Software Construction*, Prentice-Hall, New York, 1988.
- [Pinson 88] Lewis J. Pinson and Richard S. Wiener, *An Introduction to Object Oriented Programming and Smalltalk*, Addison-Wesley Publishing Company, Reading, Massachusetts, 1988.
- [Stroustrup 86] Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley Publishing Company, Reading, Massachusetts, 1986.

APL92 Caption Contest



Polivka and De Kerf



The Unknown Bandsman



Time for a # Exit

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Shared Variables and Windows DDE	Adrian Smith	128
Three Combinatoric Puzzles	Roger K.W. Hui	139

Shared Variables and Windows DDE

by Adrian Smith

Introduction

You are on holiday in Marbella. A nice young lady approaches, and asks if you would like to share your variables with her. Do you:

- a) Explain that you are English, that you have a wife and two delightful children back at the villa, etc. etc.
- b) Pretend you haven't heard;
- c) Engage her in a detailed technical discussion about the merits of concurrent processing, co-operating tasks, hot-links and cold-links, etc. until she wanders away in complete confusion.

Answers on a postcard please.

This was going to be a simple, practical guide to the use of shared variables and DDE under Dyalog/W and Windows 3; however it occurred to me that any APLers who have grown up since the mid 80s may never have encountered `⌈SVO` and `⌈SVC` at all. Consequently, I will start with a brief introduction to the philosophy behind concurrency in APL, and (hopefully) show how Windows DDE fits closely into the same pattern.

In part-2 (the bit subtitled "Practical Stuff (3)") you will learn:

- how to use Excel as a simple report formatter for APL
- how to use APL to supplement existing Excel formulae (e.g. to fit a Fourier Series through a column of sales data)
- how to drive Excel 3-D graphics from an APL numeric array
- how to exchange text between Winword paragraphs and APL nested vectors.

The point is that DDE is a very general mechanism for exchanging data between Windows applications. The new 'OLE' protocol that Microsoft are making such a fuss about still uses DDE underneath, so with a bit of digging in the manual, you should find that you can 'hot-link' APL data to almost anything. The possibilities are limited only by your imagination!

Historical Stuff

The original motivation for shared variables is well explained by Wheatley in "Extending the Domain of APL" in the recent APL Systems Journal (Vol.30 No.4, pp. 446-456). They were first postulated as a means of modelling the channel architecture in the APL representation of the IBM/360; subsequently they were implemented in APLSV as a generic way of managing I/O facilities. The original concept was somewhat diluted by a series of rather unwieldy implementations, which failed to exploit the intrinsic symmetry and elegance of the basic idea. Consequently, many APLers regard $\square SVO$ more as a tacky way of reading VSAM files than as a generalised communication channel between co-operating tasks.

IBM were also very inconsistent in the provision of true 'user-to-user' shared variable processing, and unfortunately failed to implement the necessary event messages to allow efficient programming of concurrent tasks. Only VSPC came close to providing a usable system, although there is an Interprocess product which supports user-to-user $\square SVO$ under VM and TSO.

IP Sharp did rather better, partly because they have always operated with very tight restrictions on workspace size, and partly because they provided a good event handler in $\square TRAP$. If you want to go digging, there are several good papers in IPSA conference proceedings which describe the use of background tasks for data management, workspace documentation and so on.

Meanwhile, the focus shifted to the PC world with its strictly single-tasking approach. IBM persisted with $\square SVO$ in APL2/PC, but simply as a way of porting a mainframe style of file access and screen I/O. It was not until the arrival of limited multi-tasking under Windows-3 that a genuine use of $\square SVO$ again became possible; by a useful coincidence Windows already defines a reasonable shared-variable manager (the DDE protocol), and Dyalog had chosen to use the IP Sharp approach to event handling. With both these crucial elements in place, it is not surprising that Dyalog/W has the first truly effective use of shared variables since APLSV.

Practical Stuff (1): Shared Variable Basics

I had better begin with a very basic tutorial, aimed at those APL*PLUS/PC users who never tangled with *SVO* in their mainframe days.

Suppose you have two APL users sitting at their VSPC terminals, connected to the same mainframe; Adrian is user 2001, Alice is 2014. Here are excerpts from their session logs:

<pre> Adrian (2001) ----- X+16 2014 SVO 'X DATA' 1 </pre>	<pre> Alice (2014) ----- 2001 SVO 'QQQ DATA' 2 QQQ 1 2 3 4 5 6 QQQ+'Hello Adrian' </pre>
---	--

X
Hello Adrian

This illustrates the simplest possible use of a shared variable: to move data quickly between two APL users. For a time, Rowntree shared production of the Dairy Box assortment between York and Edinburgh, and a shared variable link (very like the above) was used to allow both planners to see (and change) the same data at the same time.

To explain briefly what is going on:

- Adrian creates *X* and offers it to Alice with the (optional) alias *DATA*. The result indicates that his offer has not been taken up (possible results from *SVO* are 0: offer not made, 1: offer made but not matched, 2: offer accepted).
- Alice offers *QQQ* to Adrian with the same alias name of *DATA*. This time she gets a result of 2, the number of links to the object known to the shared variable manager as *DATA*. On referencing *QQQ* she gets the value that Adrian put into it.
- Alice re-assigns *QQQ*. Some time later, Adrian looks at the content of *X*, and finds that it has taken the new value.

There are several potential snags with this simple arrangement. One obvious one is that there is no way for Adrian to tell that Alice has sent him a new copy of *X*. In many systems, this may not matter; if Alice is the counter on a Kit Kat wrapping machine, she will re-assign *QQQ* several times per second, but the

plant manager (Adrian) will only be interested in the cumulative total a few times per day. The fact that several thousand intermediate values go to waste is quite immaterial. On the other hand, if Alice is an ATM machine at your bank, it is pretty crucial that Adrian (the bank's central computer system) gets to know every transaction!

In order to manage the interchange of data under many different circumstances, Adrian and Alice need to make use of Shared Variable Control, which allows them to impose interlocks on the communication channel:

```

Adrian (2001)                Alice (2014)
-----
      1 0 0 0  □SVC 'X'
1 0 0 0
X+'Hi, Alice'
X+'Goodbye'      # Adrian's session now hangs until ...

                                □SVC 'QQQ'
                                0 1 0 0
                                QQQ
                                Hi, Alice

      2+2                # Adrian is alive again!

4                                QQQ
                                Goodbye

```

The left argument to □SVC is a boolean vector, inhibiting you/your partner from setting/using the shared data according to the unravelled 2x2 array:

	You	Partner
Set	0	0
Use	0	0

A 1 in the top left corner stops you setting *X* until your partner has used it (when Alice checks the value of □SVC, she sees a 1 in the top right corner, as her partner can't set *X* until she has used it). Consequently, Adrian's terminal locks up as soon as he assigns a second new value into *X*. Eventually, Alice comes back from lunch and examines *QQQ*; the second assignment is released, Adrian's terminal unlocks and Alice can reference *QQQ* again to see the final value.

This simple protocol works fine if you can rely on your partner not going out to lunch! In most mainframe IBM systems the partner is an Auxiliary Processor (such as AP126 for managing GDDM screen access); it will always be there, and it will always respond more or less straight away. Consequently you will

normally find `SVF` set to 1 1 1 1, as you want to ensure GDDM has processed your format request before you send it your field data.

Things get harder when you want to manage genuine concurrency between two processes which mostly just want to get on with life, but occasionally may need to inform each other of a change to some shared data. You could imagine a market monitor, which takes a constant feed of share information, and is triggered to set an 'alert' flag when certain pre-set conditions arise. You cannot have the trading system suspended until an alert arrives, but you do want to be informed instantly when it does. Mainframe IBM APL never provided the tools to implement such a system; it was left to IP Sharp to devise a consistent definition of 'events' (see Vector 6.1 and 6.3 for the background to event trapping) which is the final piece in the jigsaw.

Practical Stuff (2): DDE Basics for Windows

DDE stands for **D**ynamic **D**ata **E**xchange; it allows co-operating Windows programs to access the same piece of data at the same time. For example, you might calculate some figures in your Excel spreadsheet, and create a link to the table from Winword. The figures get printed as if they were part of your document, but any changes you make in the spreadsheet are automatically reflected in the Word report.

You can see DDE in action even within a single application: enter some data into an Excel spreadsheet and open a Chart. Whenever you change the numbers, the chart moves, and in some cases you can drag the chart with the mouse to change the data in the spreadsheet cells. If you look under File, Links in the chart menu you will see that Excel has set up a DDE channel from the spreadsheet to the chart, and that the two are actually running as concurrent tasks.

DDE is an excellent protocol as long as you are working on a single machine. However consider what happens when you send your Winword document (and Excel worksheet) to a friend who does not own Excel! When he loads the document, Word will try (and obviously fail) to re-establish the link to the spreadsheet. The data is there alright, but in order to access it, Word needs to fire up Excel to act as the DDE Server. OLE (Object Linking and Embedding) overcomes this problem by giving you the extra option of embedding the spreadsheet data into your document. Word now saves the spreadsheet as part of the final document (with some hidden information on where it came from), but in order to update it you must explicitly select it (which starts Excel for you with the data already loaded).

The picture is further confused by the tendency of modern Windows applications to make life as easy as possible for the user!! In Winword-1 you simply selected Insert,Field and were offered 'DDE' and 'DDEAUTO' as possible field types. You then had to type the source of the data, e.g. 'DDE(AUTO) ExcelWorksheet Sheet1 R1C1:R5C2'. In Winword-2, you are expected to copy the worksheet range into the clipboard, and use Edit,PasteSpecial in Word. Word examines the clipboard for you, and inserts the DDEAUTO field automatically. This is fine, except that the details of how to do the job by hand have been removed from the manual, and are only to be found in the on-line help file!

In summary, DDE is now, and for the foreseeable future will be, a widely supported channel for data sharing between Windows applications running on the same machine (and shortly between machines attached to the same network). However, some persistence might be needed to establish the syntax needed to set up (by hand) DDE links to your particular Windows application. The examples below will work for Excel-4 and Winword-2; in extremis you could always link APL to one of these, and employ the newer Paste,Link syntax to make a second link from there to your final target!

Practical Stuff (3): DDE from Dyalog/W to Excel and Word

Scenario-1: Excel as a report formatter

It is 11.03am. You have just completed a company reporting system in APL, which does all sorts of hard sums and comes up with a simple table of consolidated results. These have to be included in the Company Report, which must go to the printers at 11.15 sharp; outside you can hear the motorcycle courier revving his engine impatiently. You have just been told that you have to use the official corporate typefaces (18 point Souvenir Demi-Italic for the heading and 18 point Optima-Bold for the data); here is what you do ...

```

      □WSID
SALESWS

      REPORT      ⍝ A nested array with your data
                Home      Export
Sales          12          6
Profits        2           3

      ρREPORT
3 3
'DDE:' □SVO 'REPORT data'
1

```

Load Excel, select cells A1..C3, and type:

```
=Dyalog|salesws!data
```

... and hit CONTROL+SHIFT+ENTER to enter an array formula. You will see your report appear in the selected cells. Select row-1, click the right mouse button, choose **Font** and set the top row to **Souvenir Demi-Italic 18pt**. Repeat for the first column, and the body of the report (only this time select **Optima-Bold**). Double-click the border to get the cell widths set correctly, right align the headings in row-1 and print it:

	<i>Home</i>	<i>Export</i>
<i>Sales</i>	12	6
<i>Profits</i>	2	3

Dash downstairs and hand the paper to the courier. It is 11.12, and you have 3 minutes to spare.

Scenario-2: APL as an Excel calculator

You have just set up a really nice sales forecasting system in Excel. This makes use of all the gee-whiz features your user could possibly want (macros attached to buttons, a custom toolbar with your own design of icons, 3-D tower charts with rotatable perspective), but unfortunately the forecaster has just read a paper recommending **Fourier Series** as the best way of fitting a mixture of trending and seasonal data. You know of a nice way of fitting a **Fourier Series** in APL, but a rapid search through the list of Excel add-ins leaves you stumped. Here's what you do ...

Select the column of Excel data, and use **Formula, DefineName** to call it something sensible (say 'series'). Start APL, and enter:

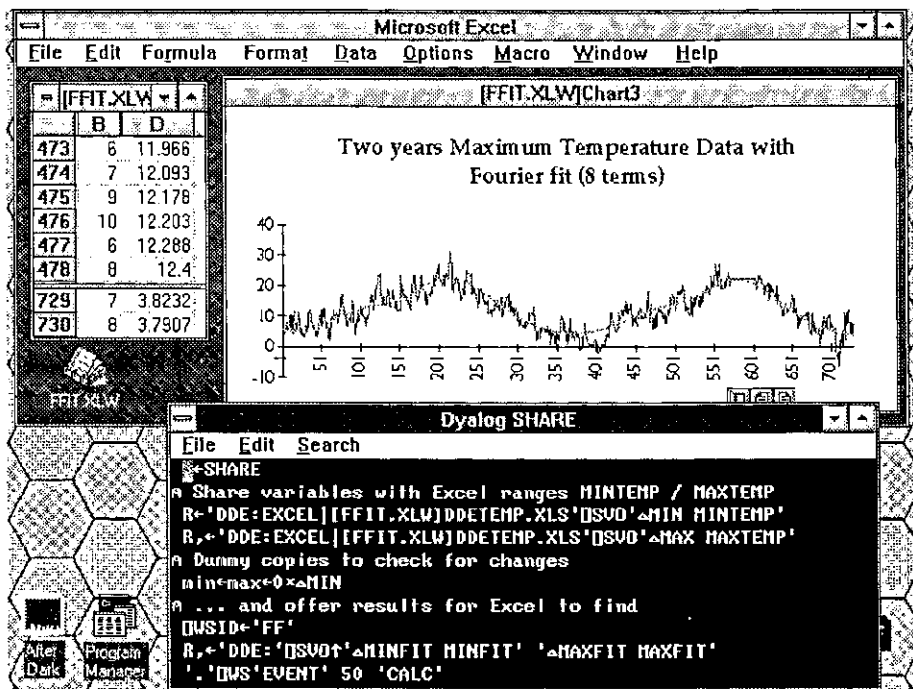
```
'DDE:ExcelWorksheet|ddesales.xls' ⎕SVO 'ΔXXX series'
2
ρΔXXX
104 1
```

You may like to check some values in **ΔXXX**, modify them in Excel and check them again. You will notice that **ΔXXX** is 'hot-linked' to the range called 'series', and that any changes in the spreadsheet are applied automatically in your APL workspace. Apply your **Fourier** function to **ΔXXX**, and return the result to Excel as described in Scenario-1.

This is fine for a one-off exercise, but you would like to make the process fully automatic. The full details are well described in the Dyalog APL User Guide, but there are a couple of snags along the way! Dyalog APL provides you the option of trapping a 'DDE' event; if you set up an event trap on the root window:

```
⋮ WS 'EVENT' 50 'CALC'
⋮ DQ ⋮
```

your workspace will detect any change to the data in 'series', run *CALC* to calculate a new Fourier fit, and bounce the result back to Excel automatically.



The snag is that *CALC* gets triggered on any DDE event, including your return of the completed data to Excel! You need to ensure that it only bothers to run the (quite lengthy) calculation when the data has changed. In theory, you can use *SVS* (Shared Variable State) to check whether 'series' has altered since you last referenced it; in practice this frequently reports a change when nothing has actually happened!

This may be Dyadic's problem (in which case it will doubtless get fixed), or it may be something odd at the Excel end (in which case it would be as well not to hold your breath while waiting). The best plan is probably the simple-minded approach of copying the data, checking for a mismatch and exiting otherwise ...

```
▽ CALC
[1] →(xxx≠ΔXXX)+Run
[2] →0
[3] Run:FIT
[4] xxx+ΔXXX
▽
```

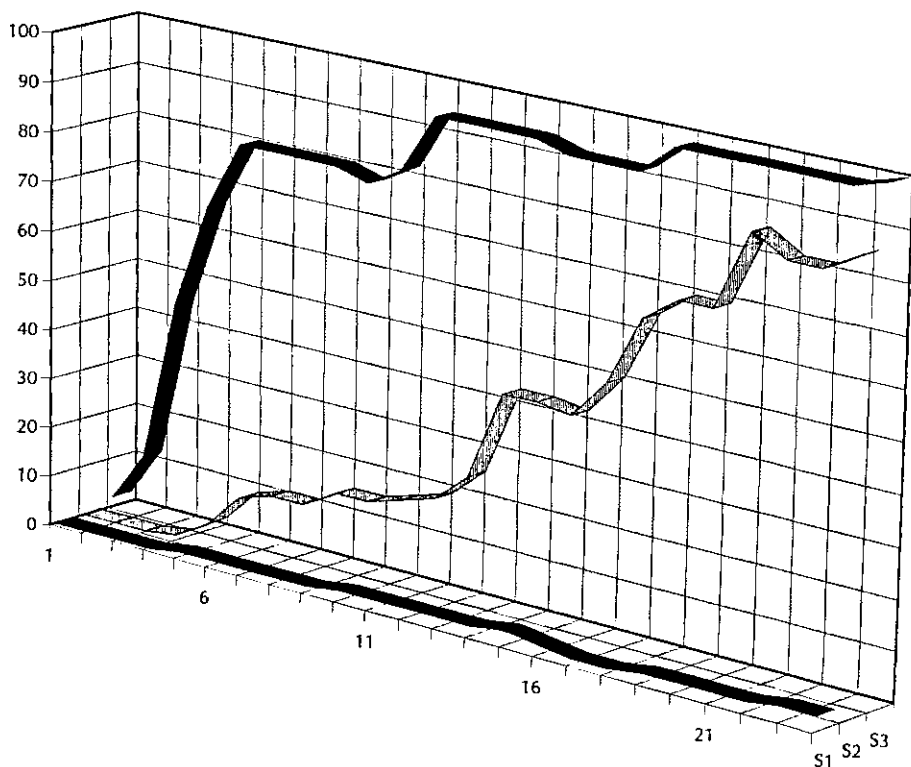
If you ensure that DYALOG.EXE is in a directory on your search path, Excel will start APL automatically as soon as you open the worksheet, load your *FFIT* workspace, and (assuming you have defined a sensible *ΔLX* to share the variables and initialise *xxx*) your APL code will be up and running with no user intervention.

Health Warning: when playing with shared variables and DDE you should *)SAVE* as often as you can. Unfortunately, Dyalog cannot save the workspace when there are active shared variables; unfortunately (again) one of the best ways of killing APL (and sooner or later the whole of Windows) is to retract the share at the APL end (with Excel still running) and then try to *)SAVE*. If you do get into this state, don't try to rescue anything, just hit Ctrl-Alt-Del while you still have the chance.

Useful Advice: beg, steal or otherwise obtain a copy of SPY.EXE from the Windows Software Development Toolkit (this also requires HOOK.DLL). With SPY running, you can watch the DDE messages flying backwards and forwards, which is invaluable in debugging anything more complex than the simple examples described above.

Scenario-3: Excel as an APL graphics server

You have just written a little workspace to explore the performance of simple genetic algorithms with APL. You would like to see the outcome of each generation plotted as it happens, rather than waiting for the APL run to finish and then drawing the entire history. You rather fancy a 3-D ribbon chart



Proceed as per Scenario-1. Start your results array the right shape, then as you index into it generation by generation, you will see the chart plot your results as they happen. Once you have set the run going, you can minimize APL and simply look at the moving chart.

It is interesting to look at the links, both in the chart and the worksheet: the chart is linked to the worksheet, and the worksheet is linked to APL. If you can fit all three on the screen together, you will see the sheet update fractionally before the chart!

Scenario-4: Text transfer between APL and Word

Suppose you have a stack of documentation in an APL workspace, which you need to access in Word, so that you can compile it into a Windows Help file. However you would rather like to continue maintaining it in APL. Here is what you do ...

```

      ρTXT                                ⋈ 25 paragraphs
25
      ρ"TXT
123 28 257 ....                        ⋈ made up of characters
      [WSID+'APL2MSW'                  ⋈ ensure WSID is set OK
      'DDE:' [SVO 'TXT']               ⋈ as normal
1

```

In Word, select Insert,Field from the menu, ignore the choices offered, and type:

```
DDEAUTO Dyalog APL2MSW TXT
```

... you will see your text inserted at the appropriate point in the document. If you modify *TXT* in the workspace, the updates are immediately reflected in the Word document.

To effect a similar transfer in the reverse direction (say from a document called 'manual'), select the text in Word and name it with a bookmark (e.g. Insert,Bookmark 'word2apl'). Then in APL:

```

      'DDE:WordDocument[manual] [SVO 'XXX word2apl'
2
      ρXXX
12                                ⋈ 12 paragraphs

```

As before, any changes in the document are instantly apparent in *XXX*.

Summary

DDE is a good general-purpose mechanism for moving data 'in real time' between concurrent Windows tasks. It is philosophically very close to the existing APL ideas embodied in `[SVO]`, `[SVC]` and `[TRAP]`. Dyalog/W uses shared variables in a simple and consistent way to allow APL to co-operate with other Windows programs. This opens up the possibility of exploiting the power of Excel graphics as part of an APL application, or of using APL simply as a calculation engine for an existing spreadsheet system.

APL Multi-media, anyone? I wonder if I can share a Bach Canon with my SoundBlaster card ... perhaps I should digitize the dog and profile his bark in APL The End.

Three Combinatoric Puzzles

by Roger K.W. Hui

1. Being involved in the development of a programming language has its rewards, none more pleasant than receiving gems like the following (EEM is Eugene E. McDonnell):

No. 6122659 filed 2.13.00 Sun 5 Apr 1992
 From EEM
 To KEI RHUI
 Subj Josephus Problem

With n people numbered 1 to n in a circle, every second one is eliminated until only one survives. For example, for $n=10$, the elimination order is 2 4 6 8 10 3 7 1 9, so 5 survives.

The problem: Determine the survivor's number $J(n)$.

$J = .1\&|.8. \# :$

Try $J^{''0} 1.50$ for a nice pattern to emerge.

See also Graham et al, *Concrete Mathematics*, Section 1.3.

J can be derived by observing that:

```

] y=.2^?10$10
2 128 16 32 4 1 64 64 512 8
J''0 y
1 1 1 1 1 1 1 1 1 1
] y=.>:710$1000
520 831 35 54 530 672 8 384 67 418
(J''0 >:y) - J''0 y
2 2 2 2 2 2 2 2 2 2

```

That is, $J \ 2^k y$ is 1, and $(J \ 1+y) - J \ y$ is 2 if $1+y$ is not a power of 2. As McDonnell asserts, the pattern is made evident by applying J to the first few integers:

>:1.5 10	$J^{''0} >:1.5 10$
1 2 3 4 5 6 7 8 9 10	1 1 3 1 3 5 7 1 3 5
11 12 13 14 15 16 17 18 19 20	7 9 11 13 15 1 3 5 7 9
21 22 23 24 25 26 27 28 29 30	11 13 15 17 19 21 23 25 27 29
31 32 33 34 35 36 37 38 39 40	31 1 3 5 7 9 11 13 15 17
41 42 43 44 45 46 47 48 49 50	19 21 23 25 27 29 31 33 35 37

The statement of J in J is interesting in its own right. $1\&|. \&. \# :$ is in the form $f\&.g$, defined as follows:

```
f    &.g  y    is    g^:_1 f    g  y
1&|. &. #: y    is    #.    1&|. #: y
```

That is, convert to the binary representation ($\# :$), rotate by one ($1\&|. \& .$), and invert by computing the binary value ($\# .$).

As a matter of historical interest, binary representation and binary value (denoted by the monads τ and $\downarrow$) were once defined in APL (see Falkoff, Iverson, and Sussenguth [1964]), but were later removed due to space limitations. Such draconian measures are understandable; at the time, APL was running on a S/360 Model 50 with 256 Kbytes of main storage (nevertheless supporting 24 users simultaneously with sub-second response).

2. Compute the $((2^n), n)$ sorted table of the n -bit binary numbers.

```
bin0 =. #&2 #: i.@(2&^n)
bin1 =. (1 0&$) '({0&,. , 1&,. )@$:@<: ) @. *
grow =. 0&,. , 1&,.
bin2 =. 'grow^:y.i.1 0' : ''
```

$bin0$ exploits the fact that $encode(\# :)$ does almost exactly what is required. $bin0$ is a *fork*:

```
(#&2 #: i.@(2&^n)) n
(#&2 n) #: i.@(2&^n) n
(n#2) #: i.2^n
```

If $0 < n$, $bin0$ can be simplified to $\# :@i.@(2&^n)$.

$bin1$ is recursive: n -bit numbers are formed by prefacing $(n-1)$ -bit numbers by 0 and by 1. Agenda ($@.$) applies signum ($*$) to the argument, and selects 1 0&\$ or (0&,. , 1&,.)@\$:@<: for execution according to whether the result is 0 or 1. The word $\$:$ denotes self-reference.

	$] t=.bin1\ 2$	$0\&,. t$	$1\&,. t$
0 0	0 0 0	1 0 0	
0 1	0 0 1	1 0 1	
1 0	0 1 0	1 1 0	
1 1	0 1 1	1 1 1	

(0&,. t),(1&,. t)	(0&,. , 1&,.) t
0 0 0	0 0 0
0 0 1	0 0 1
0 1 0	0 1 0
0 1 1	0 1 1
1 0 0	1 0 0
1 0 1	1 0 1
1 1 0	1 1 0
1 1 1	1 1 1

bin2 uses the same idea as bin1, but replaces recursion by explicit definition (:) and the power conjunction (^:). That is:

```
bin2 n
'grow^:y.i.1 0':'' n
grow^:n i.1 0
grow grow ... grow i.1 0
```

Generating Gray codes is a minor variation on generating binary numbers: Given n , compute the $((2^n), n)$ table of the n -bit Gray codes, with each row differing from a neighbour by one bit. As with the binary numbers, Gray codes can be generated in many ways; the following algorithm illustrates that it is a *reflected* code. Note the use of |. (reverse), and compare with bin1.

```
gray =. (1 0&$)'((0&,. , 1&,. @|. )@$:@<:) @. *
gray 0      gray 1      gray 2      gray 3
      0      0 0      0 0 0
      1      0 1      0 0 1
              1 1      0 1 1
              1 0      0 1 0
                  1 1 0
                  1 1 1
                  1 0 1
                  1 0 0
```

The same general strategy can be used to compute the following enumerations:

- all size m combinations of $1..n$
- all size m compositions of $1..n$
- all permutations of $1..n$
- all self-upgrading permutations of $1..n$
- all self-downgrading permutations of $1..n$
- all size m partitions of n
- all partitions of n

3. Ackerman's function is defined on the non-negative integers as follows:

```

ack = . c1'c1'c2'c3 @. (#.@(, &*))
c1 = . >:@]          NB. 1+y
c2 = . <:@[ ack 1:    NB. (x-1) ack 1
c3 = . <:@[ ack [ ack <:@]  NB. (x-1) ack x ack y-1

      2 ack 3          3 ack 2
9          29

```

In the definition of `ack`, the phrase `#.@(, &*)` computes the base-2 value (`#.`) of the catenated signs of the arguments (`, &*`), and the resultant number is used to select `c1`, `c2`, or `c3` for execution.

Lemma: If $x \text{ ack } y$ is $f\&.(3\&+) y$, then $(x+1) \text{ ack } y$ is $f\wedge:(1+y)\&.(3\&+) 1$.

Proof: By induction on y .

$(x+1) \text{ ack } 0$	Basis
$x \text{ ack } 1$	Definition
$f\&.(3\&+) 1$	$x\&ack$
$f\wedge:(1+0)\&.(3\&+) 1$	$\wedge:$
$(x+1) \text{ ack } y$	Induction
$x \text{ ack } (x+1) \text{ ack } y-1$	Definition
$f\&.(3\&+) (x+1) \text{ ack } y-1$	$x\&ack$
$f\&.(3\&+) f\wedge:(1+y-1)\&.(3\&+) 1$	Inductive Hypothesis
$_3\&+ f \ 3\&+ _3\&+ f\wedge:(1+y-1) \ 3\&+ 1$	$\&.$
$_3\&+ f \ f\wedge:(1+y-1) \ 3\&+ 1$	$+$
$_3\&+ f\wedge:(1+y) \ 3\&+ 1$	$\wedge:$
$f\wedge:(1+y)\&.(3\&+) 1$	$\&.$ QED

Using the lemma (or otherwise), it can be shown that:

$0\&ack = >:\&.(3\&+)$	NB. $>:$
$1\&ack = 2\&+\&.(3\&+)$	NB. $2\&+$
$2\&ack = 2\&+\&.(3\&+)$	NB. $3\&+@(2\&+)$
$3\&ack = 2\&+\&.(3\&+)$	
$4\&ack = \wedge/ @(\# \&2)\&.(3\&+)$	
$5\&ack = \wedge/ @(\# \&2)\wedge:(1+y.)\&.(3\&+) 1' : '1'$	

References

- [1] Graham, R.L., D.E. Knuth, and O. Patashnik, *Concrete Mathematics*, Addison-Wesley, 1989, Section 1.3.
- [2] Falkoff, A.D., K.E. Iverson, and E.H. Sussenguth, *A Formal Description of System/360*, IBM Systems Journal, Volume 3, Number 3, 1964, Table 1, page 200-201.

Index to Advertisers

Dyadic Systems Ltd	2
MicroAPL	6
APL Booklist (Renaissance Data Systems)	52

All queries regarding advertising in VECTOR should be made to Gill Smith, at 04393-385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 0488-648575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 92/93

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting student membership (UK only)	£6	☐

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____

The British APL Association will take the completion of this form as evidence that you do not object to your membership details being held on our computer. For details of the British APL Association's exemption from the Data Protection Act please see Vector, the Journal of the BAA, Vol.8 No.4

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Send the completed form to:

British APL Association, FREEPOST (SG923), 8 Cardigan Road, LONDON, E3 5BR

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1992/93 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0272-730036	11 Auburn Road, Redland BRISTOL, BS6 6LS.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 0488-648575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Duncan Pearson 0904-603510	c/o Operational Research Nestlé-Rowntree, YORK YO1 1XY
Projects:	George MacLeod 0732-885126	Rusper, Sandy Lane, Ivy Hatch, Sevenoaks, Kent TN15 0PD
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Dr Peter Branson 081-848 8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Jonathan Barman	0488-648575
Secretary:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	Ray Cannon (0252-874697), Richard Weber, Sylvia Camacho, Duncan Pearson, David Ziemann (071-267 8032)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St. BATH, BA1 5NS
Tel: 0225-462602
Fax: 0225-444552

Compass R&D Ltd
15 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483-302249
Fax: 0483-302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel: 0256-811125
Fax: 0256-811130

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 071-353 4212
Fax: 071-353 3325

Cocking & Drury Ltd
180 Tottenham Court Road
LONDON W1P 9LE
Tel: 071-436 9481
Fax: 071-436 0524

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8866
Fax: 071-928 1006

Reuters Ltd
Reuters Nederland BV
PO Box 8230, 1005 AE Amsterdam
Netherlands
Tel: +31 20 570 8733
Fax: +31 20 570 8758

Peter Cyriax Systems
22 Hereford Road
LONDON W2 4AA
Tel: 071-229 5344

Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel: 0732-885126

Dutch APL Association
Postbus 1341, 3430BH Nieuwegein
Netherlands
Tel: 03474-2337

Manugistics
2115 East Jefferson St
Rockville
MARYLAND 20852
USA

I-APL News for Vector Volume 9 No 2 October 1992

J Version 5.1

We are now distributing J version 5.1 in several versions:

- For PC
- For 386 with extended memory access
- To run under Windows
- For Atari
- For Archimedes (see below)

There other versions (eg for Macintosh and various Unix machines) available from ISI, 33 Major St, Toronto, Ontario, Canada M5S 2K9, or we can get them for you.

Problems with new J Archimedes port

There is a problem with the Archimedes operating system. It traps such expressions as divide by zero which are not errors in J (the result is infinity). Paul Chapman has not yet found a good way to implement J so that its infinities work properly. We did not want to delay issue of version 5.1 so have put a warning on the disk. At least now you can have version 5.1 which works in nearly all respects.

Atari ST Manual now ready

The manual for the Atari ST version of I-APL is now printed and on sale. All back orders have been supplied. So few orders have been received that it seems that the world of Atari ST users has not heard of APL or I-APL. Please will anyone who can help to spread the news do so.

Russian I-APL/PC manual corrections received

The draft version of the Russian I-APL/PC manual, translated by Elena Korovina and Alexei Miroshnikov, was circulated at APL 92. Some improvements to the translation to make it more suitable for use in schools have been suggested by Oleg Luksha and Vyacheslav Kuprianov and Alexei and Elena have rewritten some sections. We are very grateful for the excellent work done by all concerned. The British APL Association has agreed to pay for typesetting and printing of a supply of these manuals and for shipping them to the I-APL support groups in Russia. The printed version should be ready before the end of the year. We will send you one for five pounds, but you must order from the enquiry address at once because we won't keep back any when we send them to Russia.

Guidance in choice of cheap APLs

Dick Holt of HRH Systems has produced a guide for the Capital PC User Group to circulate on a PC disk or via his bulletin board. I-APL would like to provide similar advice, but we would have to cover a range of machines with different disk formats so must do it in printed form. For each machine we should list the interpreters available for it in price order with their advantages and disadvantages. We should cover the drivers and workspaces which make it possible to display and print APL. The guide should cover I-APL, ISI products, Sharp APL/PC, MicroAPL products, TryAPL2, Pocket APL and any obsolete APLs such as VIZ::APL which were put into the public domain. Maybe we should briefly review the fully commercial APL interpreters such as Dyalog, APL2/PC and APL*PLUS products. I need help in compiling such a guide; please write to me if you would like to join in.

Anthony Camacho 19 October 1992

I-APL Ltd order form 1 November 1992

Address for enquiries: 11 Auburn Rd
Redland
Bristol BS6 6LS

Address for orders:

56 The Crescent
Milton, Weston-super-mare
Avon BS22 8DU

Disks are 360K PC disks except where specified. Please supply:

Prices include UK postage & VAT

Part No	Item	No. of copies	Price £	Total
I-APL and related Products				
PC12	PC clone/Nimbus 360K disk & manual	—	8.00	—
AT12	PC clone/Nimbus 720K 3.5" disk & manual	—	11.00	—
BBC12	BBC 80T disk Master (& B with 32K sideways RAM) & manual	—	8.00	—
A12	Archimedes 3.5" disk and manual	—	11.00	—
Mac12	Macintosh 3.5" disk and photocopied manual	—	13.00	—
ST12	Atari ST 3.5" disk and photocopied manual	—	11.00	—
M2	I-APL Manual for PC clones, BBC Master/B & Archimedes	—	3.00	—
ST2	I-APL Manual for Atari ST (photocopied)	—	4.00	—
T3	Tutorial by Thomson & Alvord (45pp)	—	3.00	—
E4	Encyclopedia by Helzer (228pp)	—	6.00	—
S5	APL in Social Studies by Traberman (36pp)	—	3.00	—
P6	APL programs for maths classroom, Thomson (185pp)	—	16.00	—
Disks for I-APL				
FEMC	Disk of Functions for Thomson's book by F Espinasse	—	5.00	—
FEK12	Maths kits 1 and 2 by Francis Espinasse	—	5.00	—
FEK34	Maths kits 3 and 4 by Francis Espinasse	—	5.00	—
MK	Morten Kromberg's GRAF workspaces	—	5.00	—
ZVJ	Zdenek V Jizba's Lessons workspaces	—	5.00	—
FINN	FinnAPL workspaces	—	5.00	—
ISI Products				
SAPL	Sharp APL/PC manual [system] (350pp), disks & Tang. Math	—	42.00	—
DSAPL	Sharp APL/PC disks only (shareware version 17)	—	10.00	—
APLIPC	Sharp APL/PC version 20 for PC & AT clones (3.5")	—	55.00	—
DAPLIPC	Disks only version of APL/PC (3.5")	—	21.00	—
APLI386	Sharp APL/PC version 20 for 386/486 in real mode (3.5")	—	60.00	—
DAPLI386	Disks only version of APLI386 (3.5")	—	26.00	—
APLIWIN	Sharp APLI386 for Windows 3.0 enhanced mode (3.5")	—	60.00	—
DAPLIWIN	Disks only version of APLIWIN (3.5")	—	26.00	—
APLISW	Sharp APL/PC V 17, (+ISI help & fullscreen editor) (3.5")	—	8.00	—
SAR	Sharp APL Ref. Manual [language] (qto 349pp) by Berry	—	18.00	—
JPC	'J' for PC, (APL dialect): Dictionary & Intro to J (3.5")	—	16.00	—
J386	'J' for 80386 PCs with Dictionary of J and Introduction to J (3.5")	—	16.00	—
JMAC	The Macintosh version of JPC above: Mac 3.5" disk	—	16.00	—
JARC	The Archimedes version of J: Arc 3.5" disk	—	16.00	—
JAST	The Atari ST version of J: Atari ST 3.5" disk	—	16.00	—
PIJ	Programming in J (75pp) by K. E. Iverson	—	10.00	—
ARI	Arithmetic (118pp) by K. E. Iverson	—	12.00	—
IJ	An Introduction to J (47pp) by K. E. Iverson	—	8.00	—
TM	Tangible Math (in J 36pp) by K. E. Iverson	—	8.00	—
TryAPL2 — Many people can get TryAPL2 from IBM free. If YOU can, don't buy from us.				
TRY720	TryAPL2 720K disk (contains the manual on disk)	—	8.00	—
APL Press Books				
AAT	Algebra: an Algorithmic Treatment, Iverson (361pp)	—	8.00	—
SAAT	Solutions to foregoing by J Iverson (42pp)	—	2.00	—
EA	Elementary Analysis by K Iverson (218pp)	—	6.00	—
CNK	Calculus in a New Key by D Orth (286pp)	—	5.00	—
RCT	Resistive Circuit Theory by R Spence (qto 279pp)	—	8.00	—
TEY	APL Quote-Quad: The Early Years (qto 465)	—	11.00	—
SBA	A Source Book in APL (qto 140pp)	—	7.00	—
AI	APL and Insight by Berry et al. (89pp)	—	4.00	—
FCP	The Four Cube Problem by McDonnell (qto 27pp)	—	4.00	—
SM	Star Map by Berry & Thorstensen (41pp)	—	3.00	—
PA	Probability in APL by L Alvord (120pp)	—	5.00	—
IA	Introduction to APL by K E Iverson 1984 (110pp)	—	6.00	—
AL	APL Language Reference for foregoing (128pp)	—	6.00	—
IAT	Introducing APL to Teachers (25pp) by Iverson	—	3.00	—
IASE	Intro to APL for Scientists & Engs, Iverson (26pp)	—	3.00	—
AE	APL in Exposition by K E Iverson (61pp)	—	3.00	—

Please add 3.00 per parcel for packing

3.00

VAT Regd No 478 1634 13 (VAT @ 17.5% is 87.5 pence per disk: books are zero-rated)

Total

Name _____ (please write legibly)

Address _____

Postcode _____

Please pay "I-APL Ltd" Note that
we do not accept credit cards;
please send cheque with order.

OFFICE USE